# Data Representation and Analysis Using Graph Models

Prof. Carlos Henrique Gomes Ferreira

UNIVERSITÀ DEGLI STUDI DI TRIESTE

DECSI
DEPARTAMENTO DE COMPUTAÇÃO E SISTEMAS

UFOP
UNIVERSIDADE FEDERAL DE OURO PRETO

# Who am I?

**Bachelor's Degree in Information Systems from the Federal University of Viçosa (2010 - 2014)**

**Thesis:** Performance Evaluation of Virtualizers Applied to Cloud Computing
**Topics:** Distributed Systems, Cloud Computing, and Workload Modeling

**Master's Degree in Computer Science and Computational Mathematics from the University of São Paulo (2014-2016)**

**Thesis:** PEESOS-Cloud: A workload-aware architecture for performance evaluation in SOA
**Topics:** Distributed Systems, Cloud Computing, and Workload Modeling

**Ph.D. in Computer Science from the Federal University of Minas Gerais (2016 - 2022)**

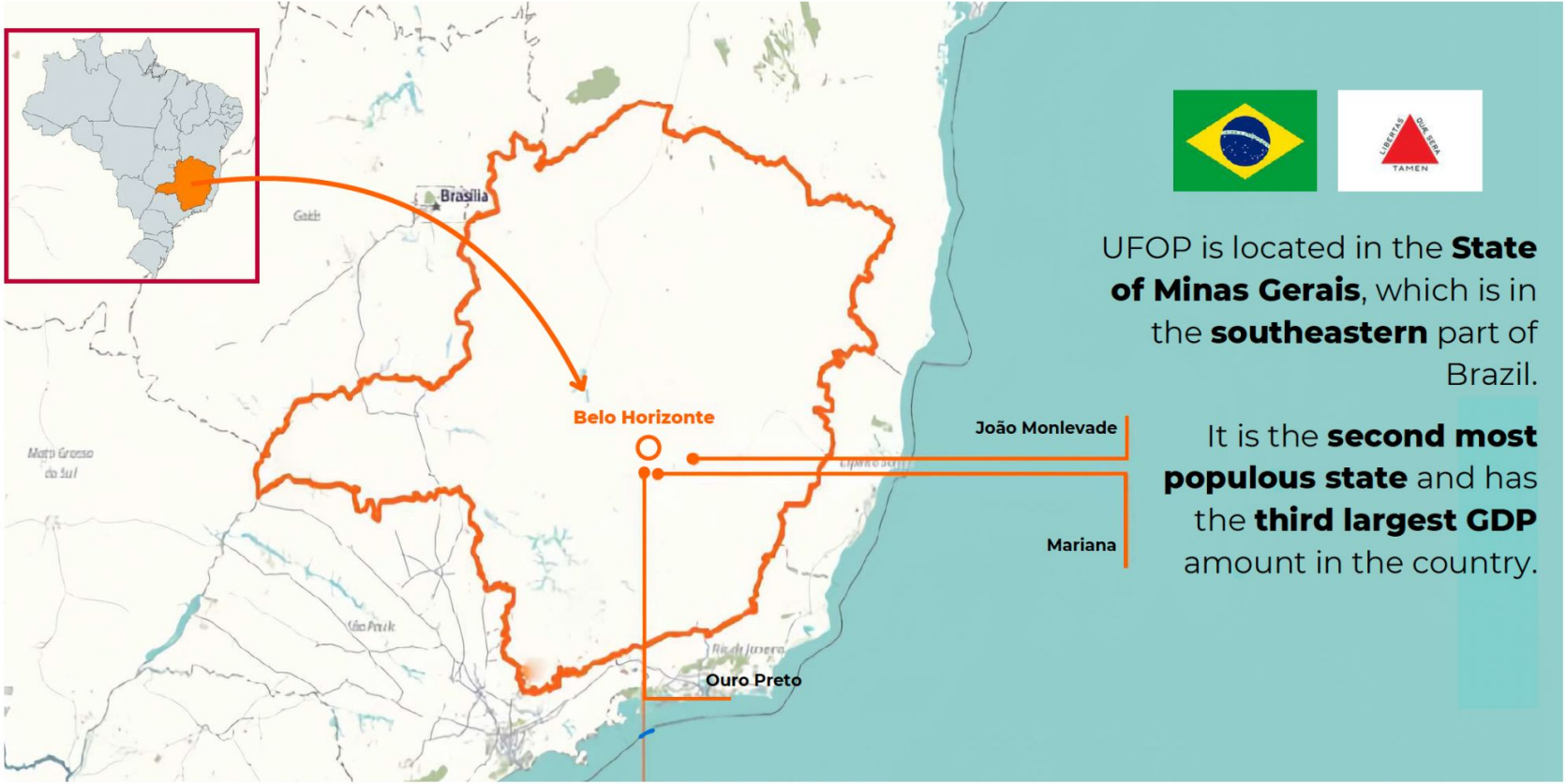
**Dissertation:** Modeling and Analyzing Collective Behavior Captured by Many-to-Many Networks
**Topics:** Network Science, Data Science, Online User Behavior

**Ph.D. in Electrical, Electronic, and Communication Engineering from the Politecnico di Torino, Italy (2019 - 2022)**

Double Degree Agreement - *Cum Laude*

**Assistant Professor in the Department of Computing and Systems at the Federal University of Ouro Preto - Currently**

# Where am I from?



# Where am I from?

## CAMPI




# Where am I from?

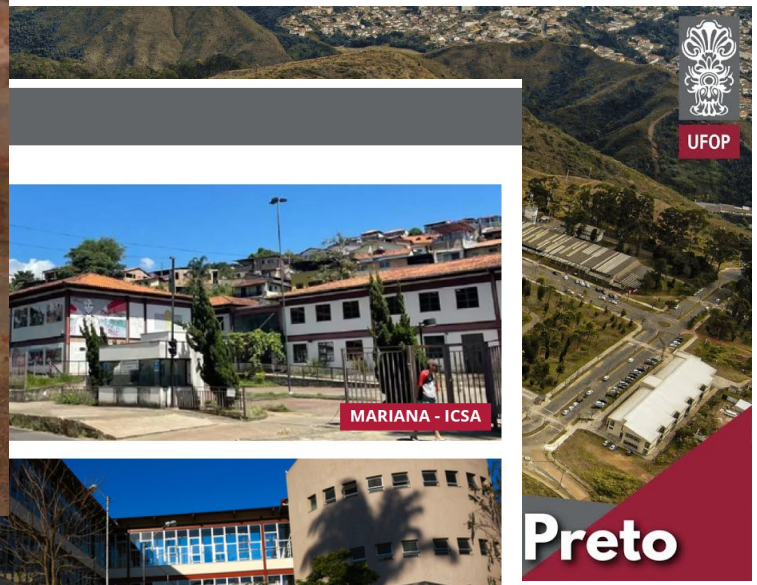
# Where am I from?


# Where am I from?

Industrial hub in the Médio Piracicaba region, in Minas Gerais, **João Monlevade** is a reference in the extraction and processing of steel.

Aerial view - João Monlevade

# Where am I from?








# Where am I from?


# Where am I from?

# Where am I from?



# Where am I from?

# Roadmap

Graph Modeling Fundamentals

Structural and Centrality Analysis

Community Detection

Final Considerations

# Roadmap

**Graph Modeling Fundamentals**

Structural and Centrality Analysis

Community Detection

Final Considerations

# What is a graph?

A graph G is formally defined by two sets and is represented by the tuple G=(V, E), where:

- **V** represents the set of **vertices (or nodes):** components, elements, parts of the modeled system

- **E** represents the set of **edges (or links):** these are the connections between vertices, indicating relationships/interactions between nodes
    - Each edge in E is formally defined as a pair of vertices (u, v), where u and v are elements in V

# What is a graph?

Graphs can be classified according to the:

**Direction of edges:**

- **Undirected:** Edges are bidirectional
  - The order of the nodes/relation does not matter

- **Directed (Digraphs):** Edges have a defined direction from the source node to the destination node

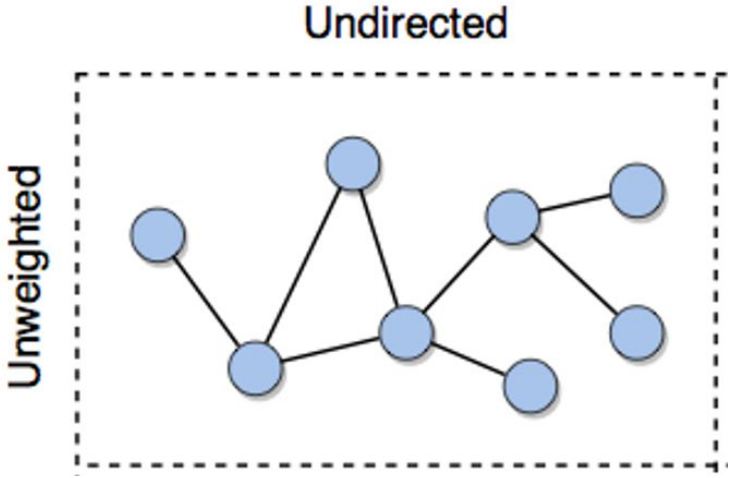
# What is a graph?

Graphs can be classified according to the:

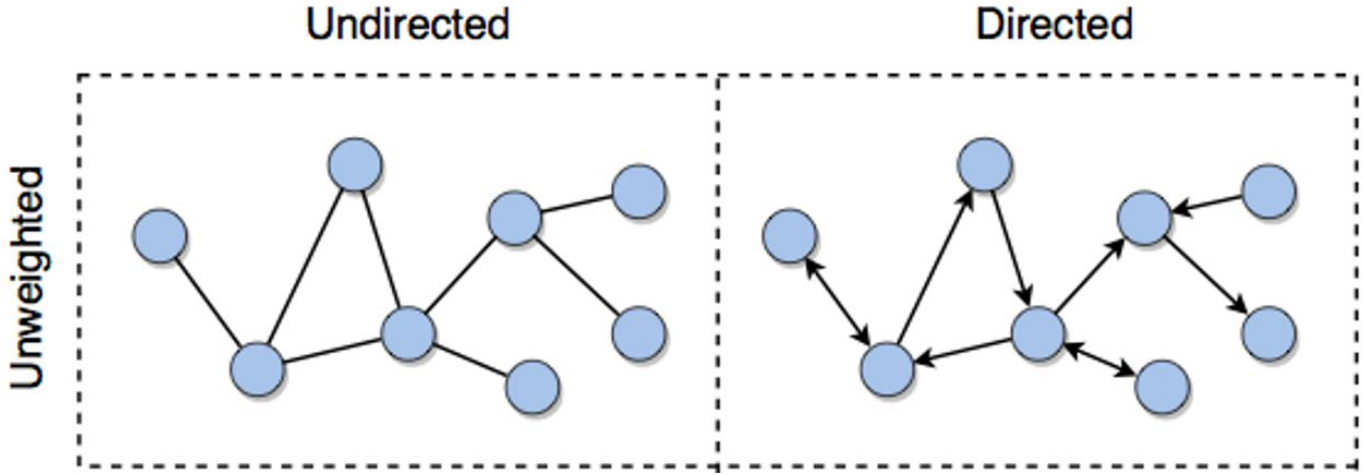
**Weights of edges:**

- **Unweighted:** All edges are considered to be of equal importance
- **Weighted:** Each edge has a weight $w$. The graph is defined as G = (V, E, $w$)
  - A weighting function $E \rightarrow R$ assigns a weight to each edge
  - Weights are costs, distances, or another quantitative measure of the connections between vertices

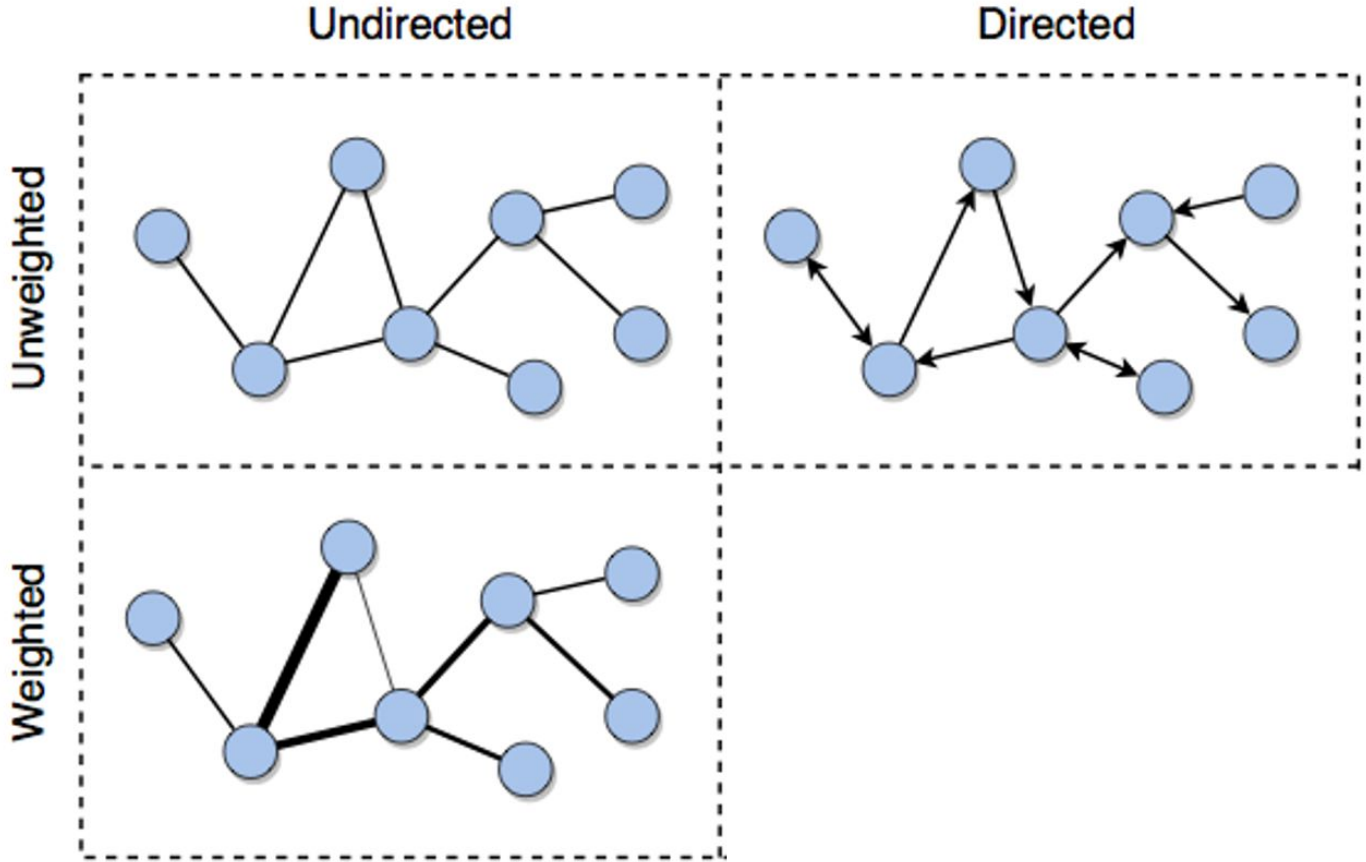
# Graphical representation



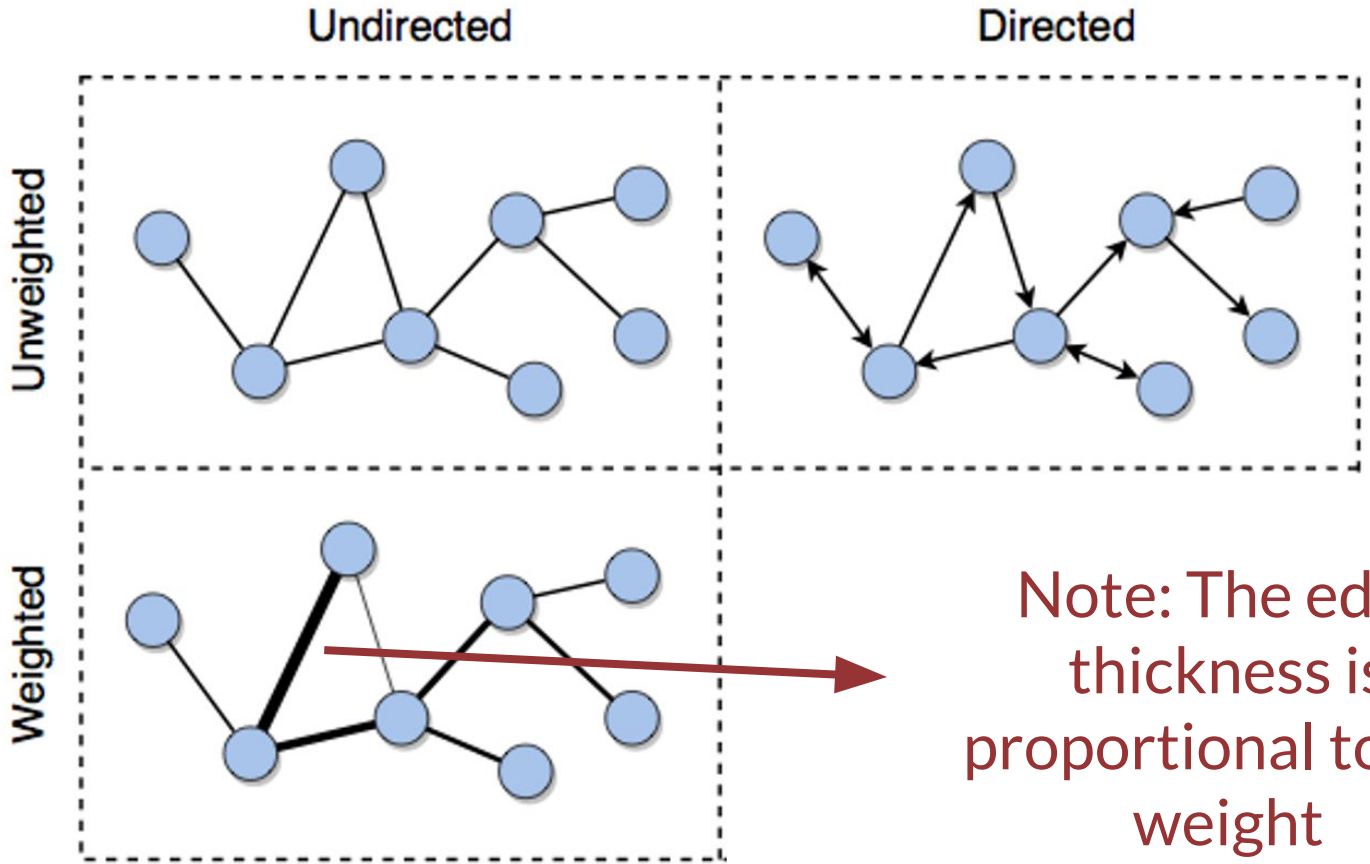
# Graphical representation



# Graphical representation

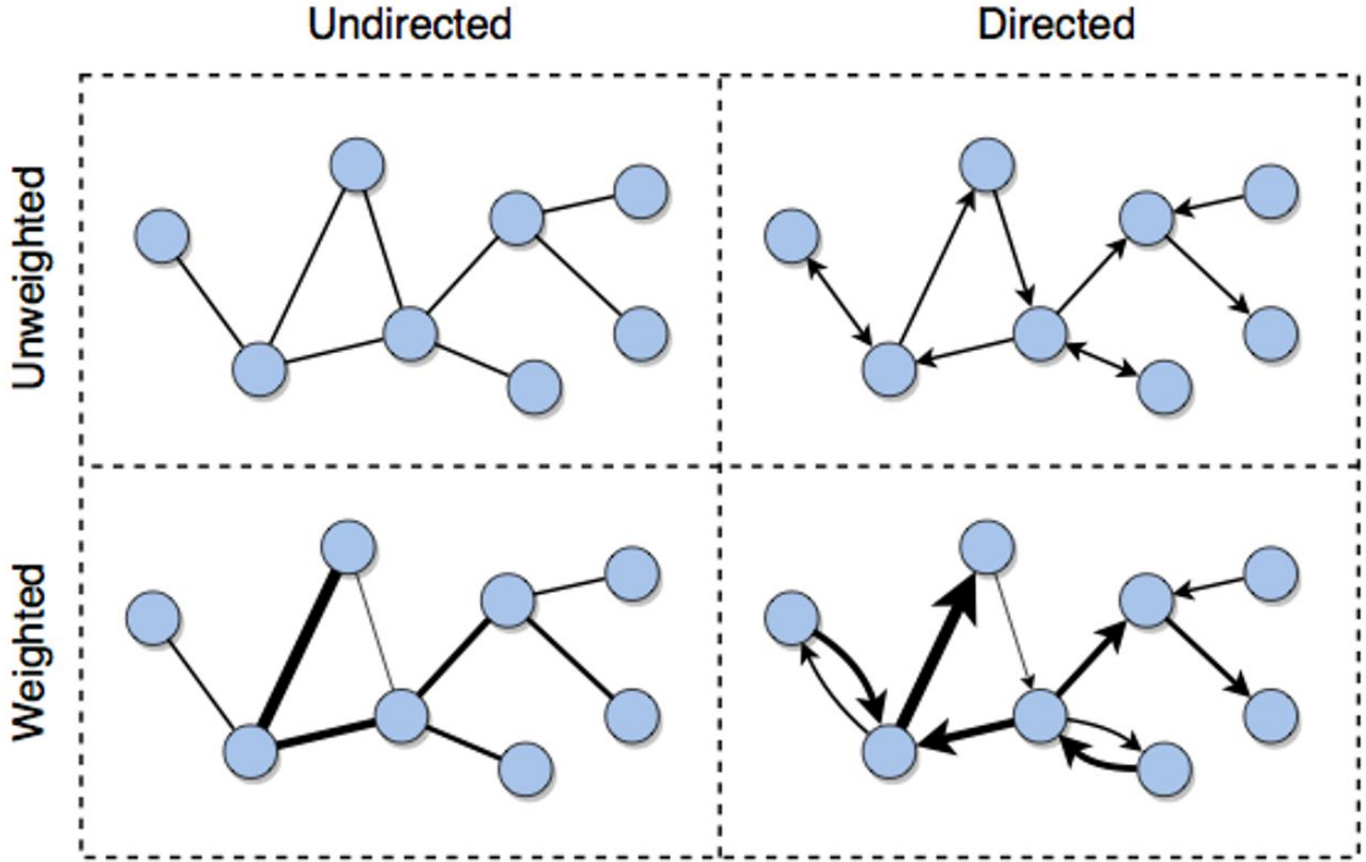


# Graphical representation

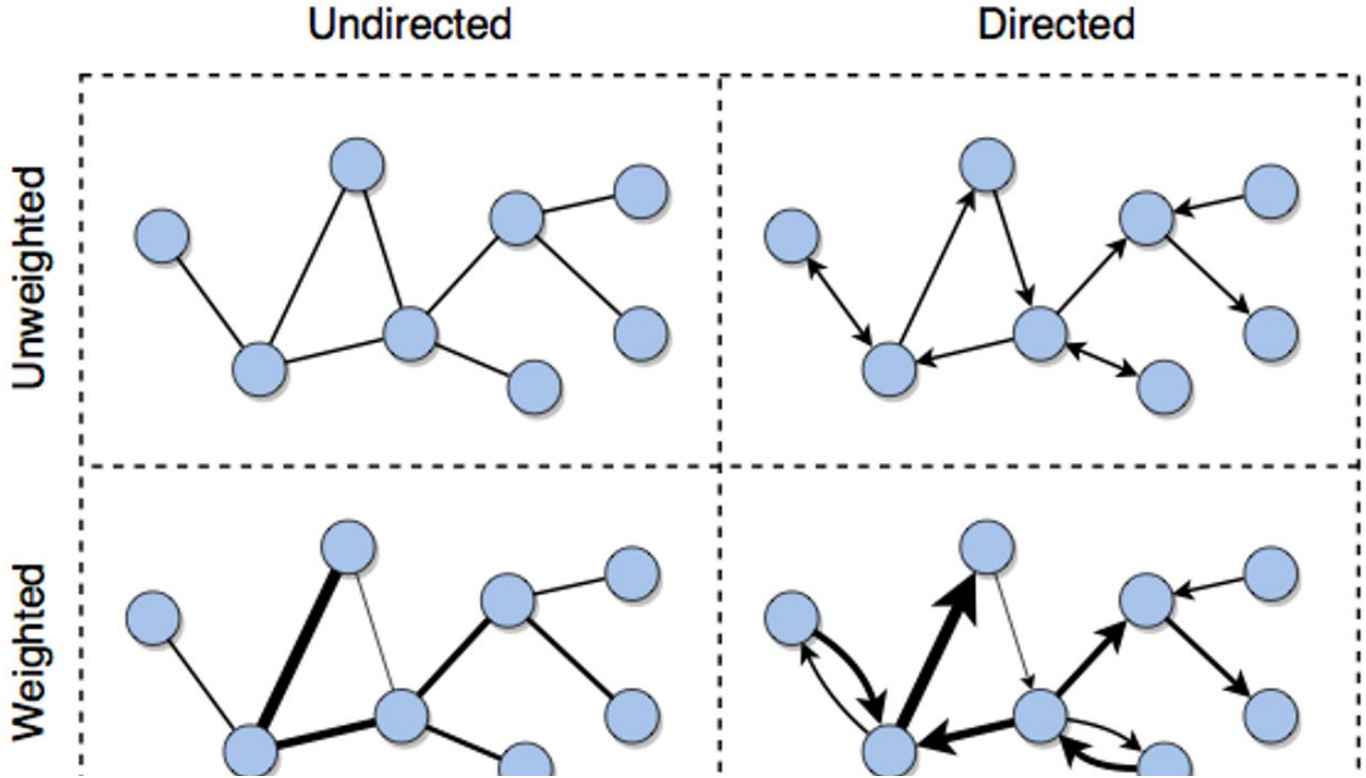



Note: The edge thickness is proportional to the weight

# Graphical representation



# Graphical representation

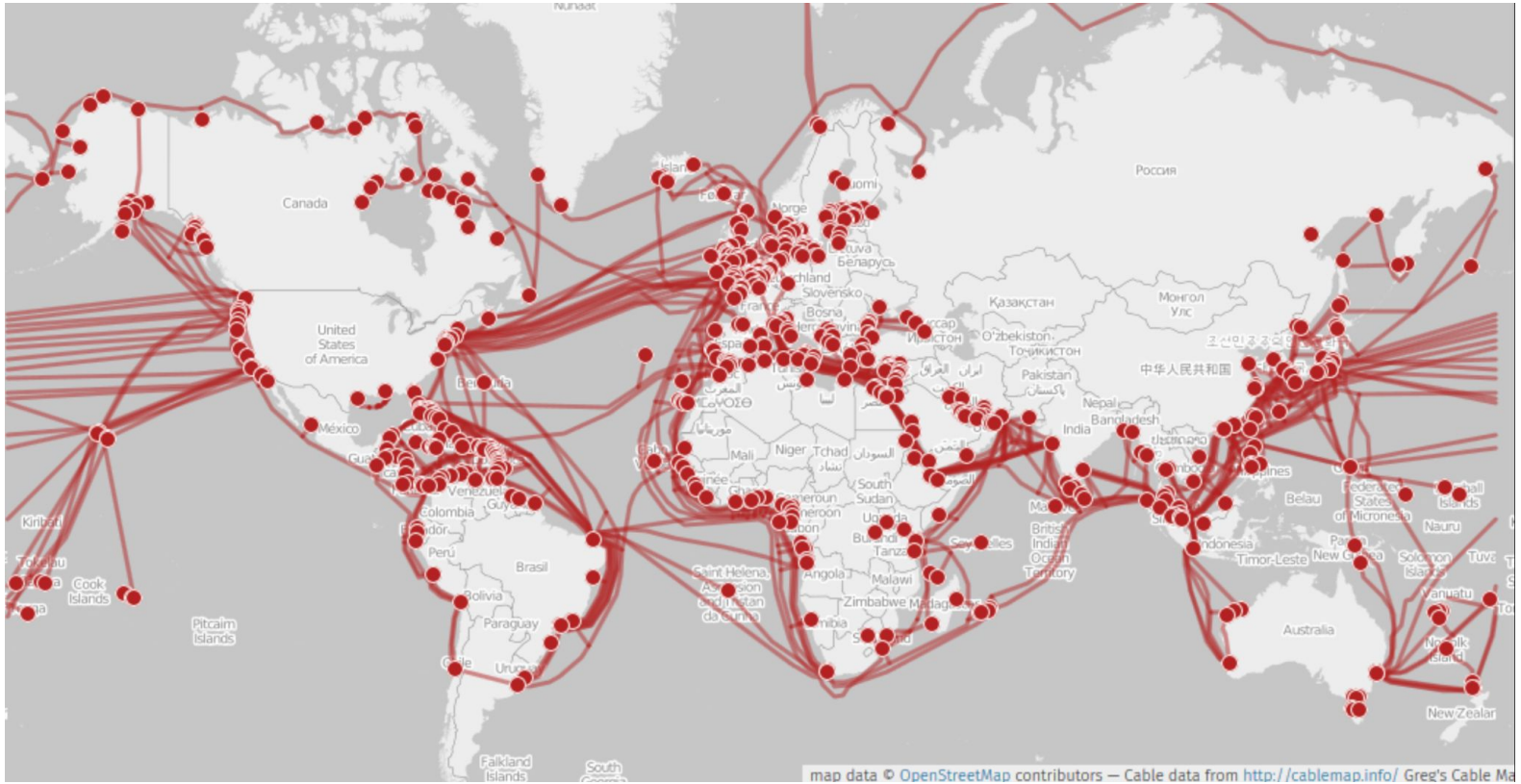



**Can you think of an example of a graph (Network)?**

# Transportation systems

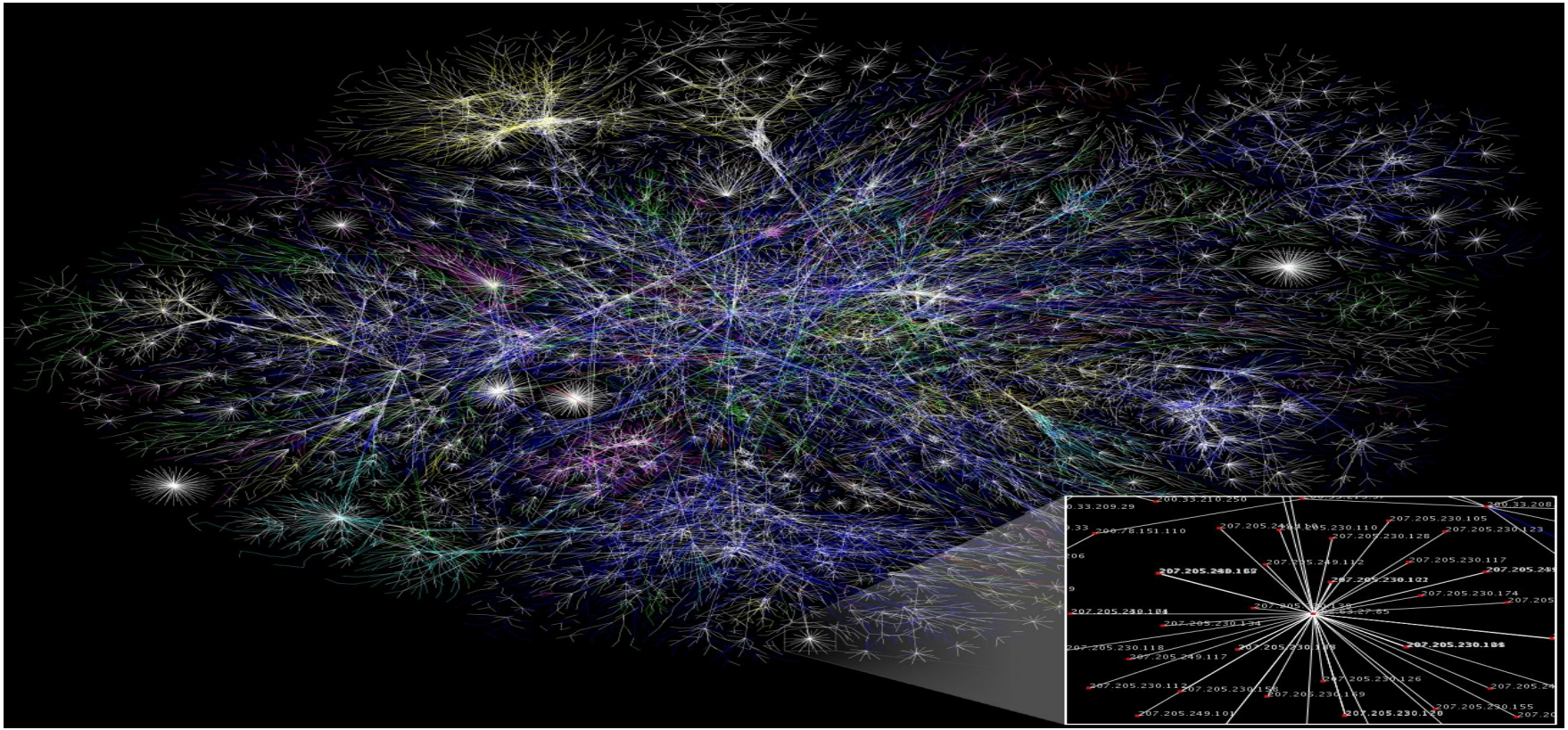
Fonte: https://infrastructuremagazine.com.au/2022/08/22/the-future-of-australias-transport-network/

# Telecommunication systems (Submarine cables)




Fonte: https://infrastructuremagazine.com.au/2022/08/22/ikdskadas

# Telecommunication systems (Internet routers)

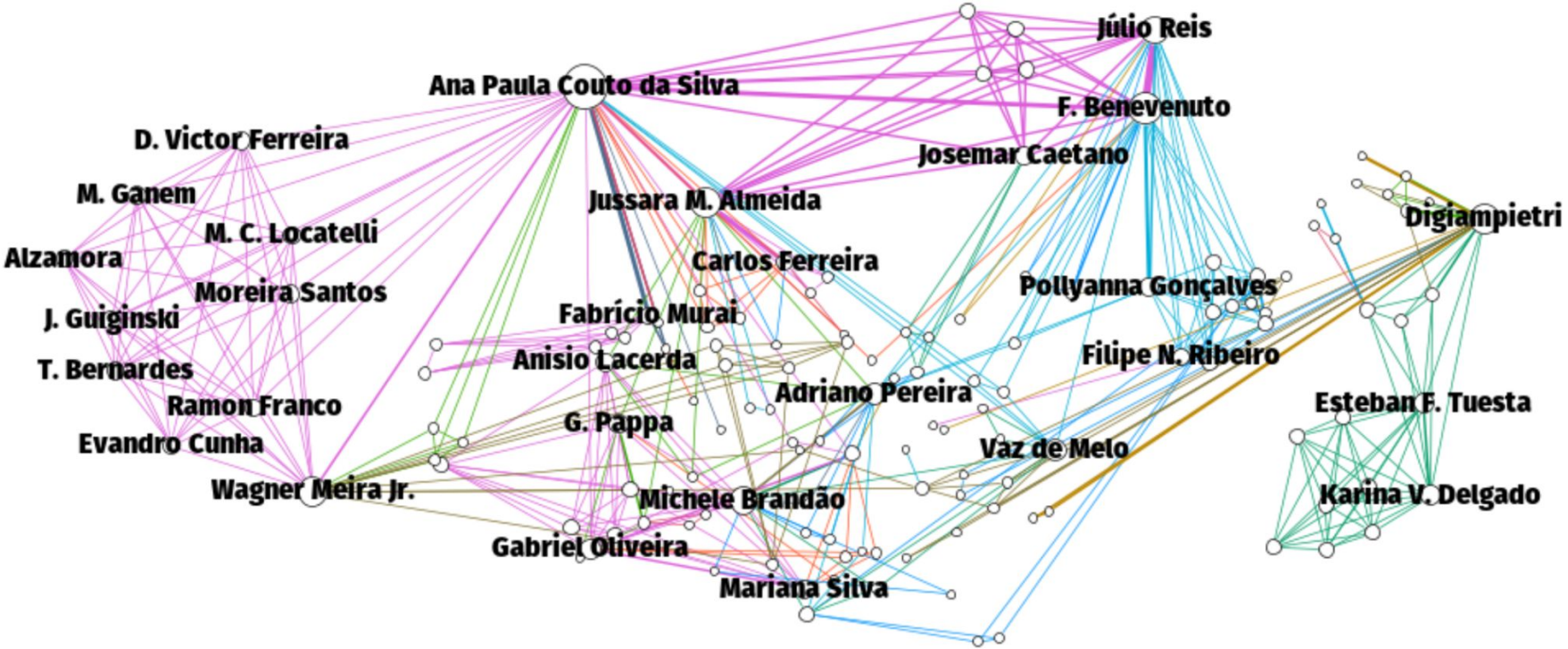



Fonte: https://infrastructuremagazine.com.au/2022/08/22/ikdskadas

# Online social networks

Fonte:https://medium.com/analytics-vidhya/social-network-analytics-f082f4e21b16

# Co-authorship networks




Fonte https://dl.acm.org/doi/10.1145/3617023.3617048

# Why do I need a graph?

Many real-world systems are inherently **networked**

**Graph theory** provides a broad set of theoretical and analytical tools for analyzing these systems

- It is used in the field of **network science** to offer insights that other models may not reveal

Simplify the representation of complex relationships, facilitating the identification of patterns of interest

# And why are they important to be studied?

These are systems that allow society to **interact**

- Just imagine a world without efficient social contacts, telecommunications networks, interconnected highways and many other networked systems

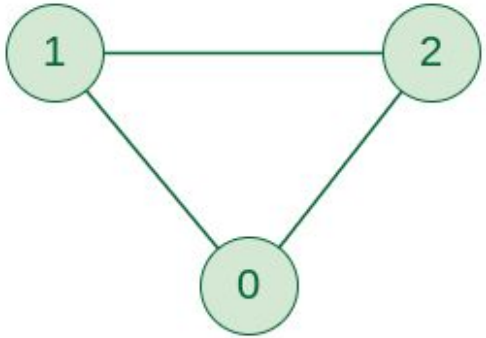
They can be characterized as **complex systems**

- They cannot be fully understood or explained by isolated analyses of the parts
- They exhibit **non-trivial characteristics** such as heterogeneity, dynamics, and collectivity, which can be modeled using graphs

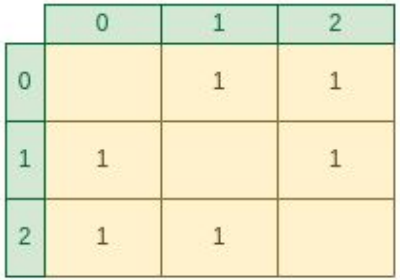
# How to represent a graph computationally?

Primarily be represented in two ways:

- **Adjacency Matrix:** A square matrix where each element indicates the presence of an edge between two vertices and, in weighted graphs, the weight of that edge




Undirected Graph



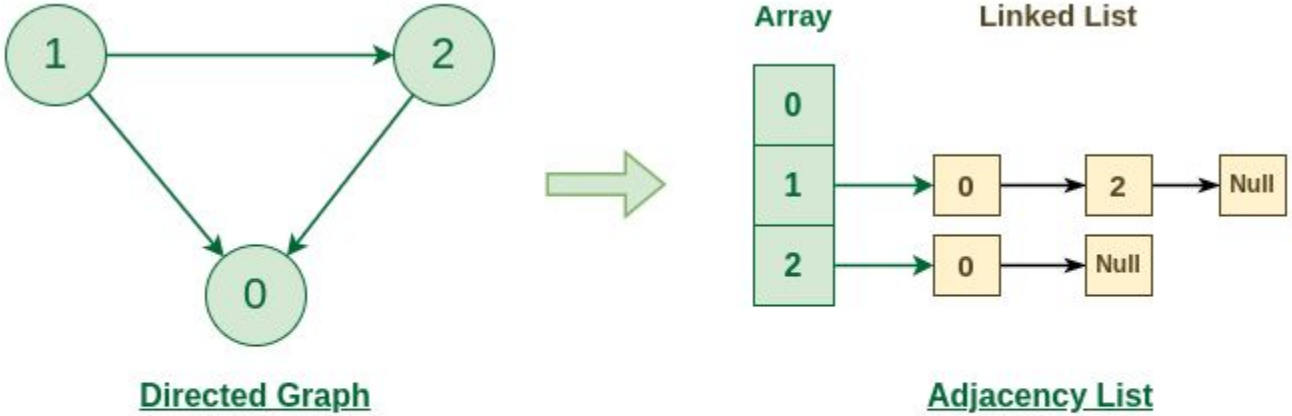
| | 0 | 1 | 2 |
|---|---|---|---|
| 0 | | 1 | 1 |
| 1 | 1 | | 1 |
| 2 | 1 | 1 | |

Adjacency Matrix

In a weighted graph, the weight of an edge is represented by the value stored in the cell corresponding to the connected vertices

# How to represent a graph computationally?

- **Adjacency List:** Represents a graph through lists that indicate the neighbors of each vertex




In weighted graphs, each entry in the adjacency list is a tuple containing the adjacent vertex and the weight of the edge

# How to represent a graph computationally?

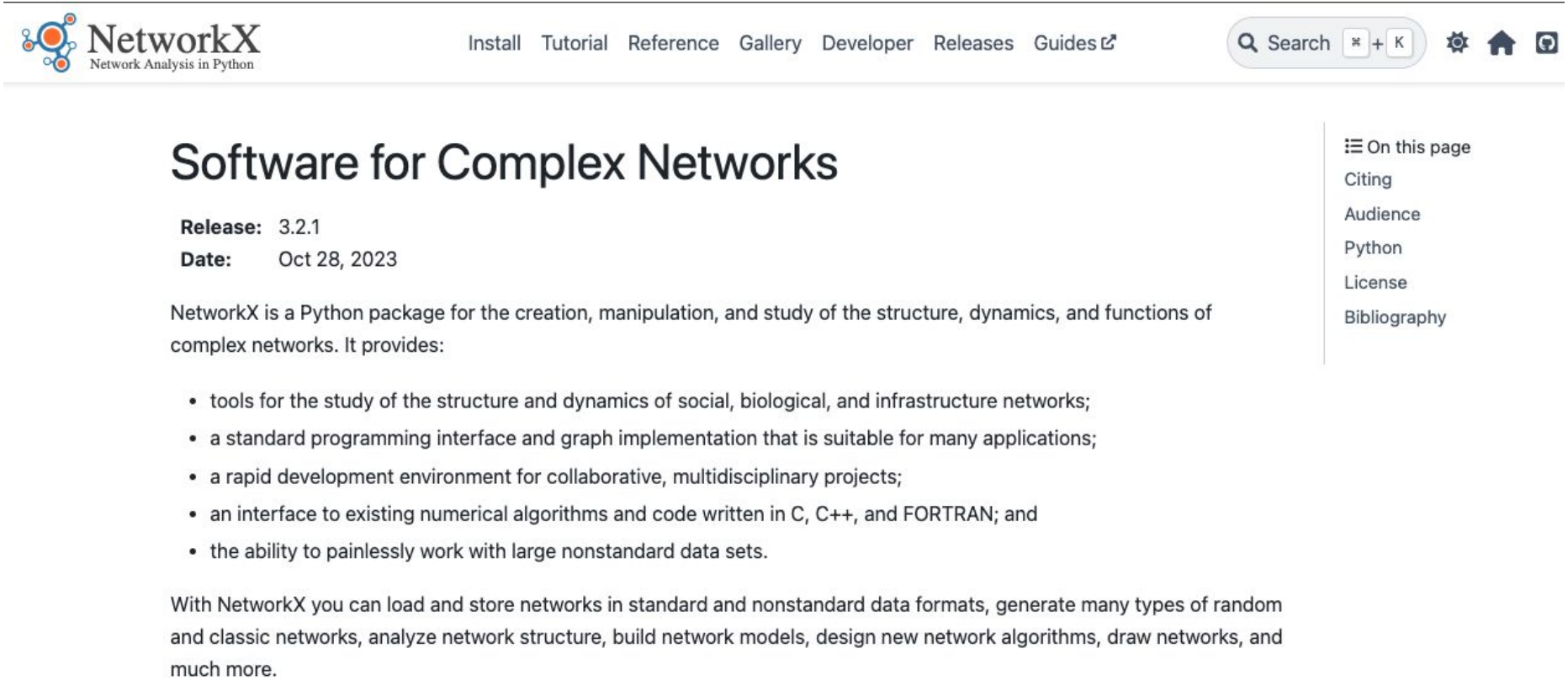
To manage, analyze, and visualize graphs with many vertices and links, the use of tools is essential

**Ex:** NetworkX (Link) and IGraph (Link) are Python packages dedicated to creating, manipulating, and analyzing the graphs structure

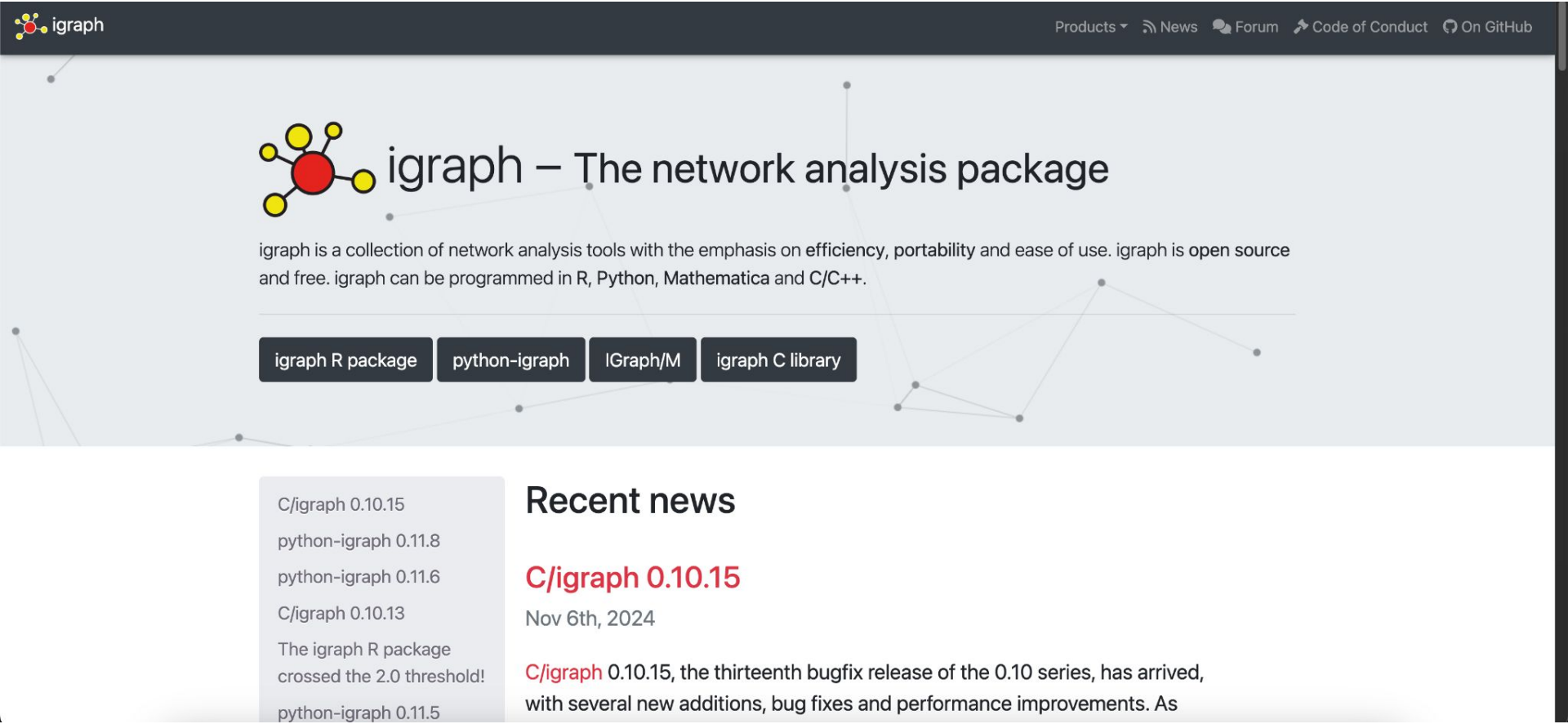
NetworkX has a gentler learning curve and offers algorithms, metrics, and network generators, while IGraph is faster and more robust

- For visualization, a good alternative is Gephi (Link)

# NetworkX


NetworkX
Network Analysis in Python
Install
Tutorial
Reference
Gallery
Developer
Releases
Guides
Search
⌘ + K
Software for Complex Networks
Release: 3.2.1
Date: Oct 28, 2023
NetworkX is a Python package for the creation, manipulation, and study of the structure, dynamics, and functions of complex networks. It provides:
• tools for the study of the structure and dynamics of social, biological, and infrastructure networks;
• a standard programming interface and graph implementation that is suitable for many applications;
• a rapid development environment for collaborative, multidisciplinary projects;
• an interface to existing numerical algorithms and code written in C, C++, and FORTRAN; and
• the ability to painlessly work with large nonstandard data sets.
With NetworkX you can load and store networks in standard and nonstandard data formats, generate many types of random and classic networks, analyze network structure, build network models, design new network algorithms, draw networks, and much more.
On this page
Citing
Audience
Python
License
Bibliography

# IGraph


igraph
Products
News
Forum
Code of Conduct
On GitHub
igraph – The network analysis package
igraph is a collection of network analysis tools with the emphasis on efficiency, portability and ease of use. igraph is open source and free. igraph can be programmed in R, Python, Mathematica and C/C++.
igraph R package
python-igraph
IGraph/M
igraph C library
C/igraph 0.10.15
python-igraph 0.11.8
python-igraph 0.11.6
C/igraph 0.10.13
The igraph R package crossed the 2.0 threshold!
python-igraph 0.11.5
Recent news
C/igraph 0.10.15
Nov 6th, 2024
C/igraph 0.10.15, the thirteenth bugfix release of the 0.10 series, has arrived, with several new additions, bug fixes and performance improvements. As

# Gephi

Gephi
makes graphs handy

Download | Blog | Wiki | Ask a question | Support | Bug tracker

Home | Features | Learn | Develop | Plugins

## The Open Graph Viz Platform

**Gephi is the leading visualization and exploration software for all kinds of graphs and networks. Gephi is open-source and free.**

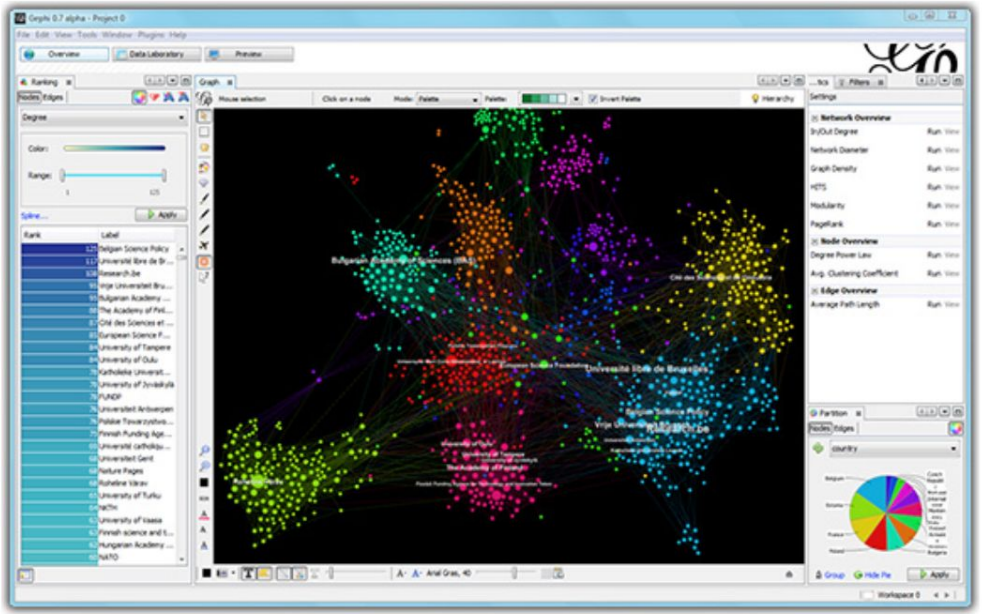
**Runs on Windows, Mac OS X and Linux.**

Learn More on Gephi Platform »

Download FREE
Gephi 0.10

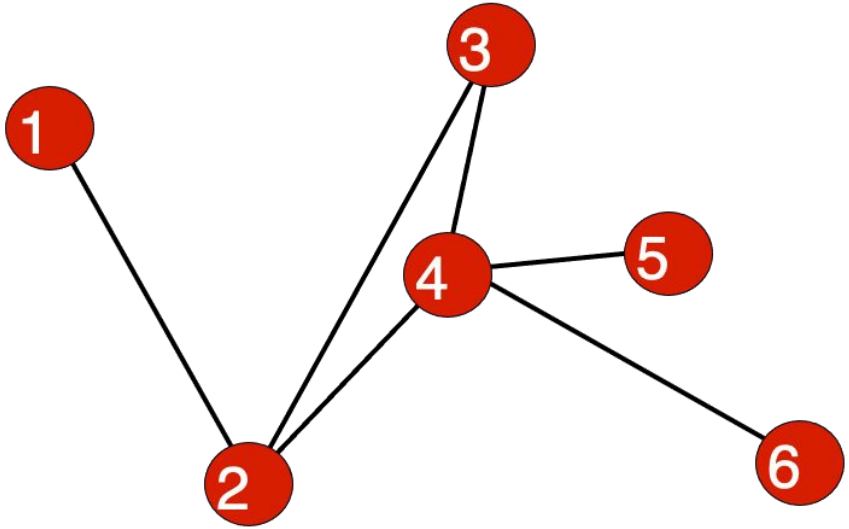
Release Notes | System Requirements

- Features
- Quick start
- Screenshots
- Videos



**Support us! We are non-profit. Help us to innovate and empower the community by donating only 8€:**

# How to create a graph with NetworkX?

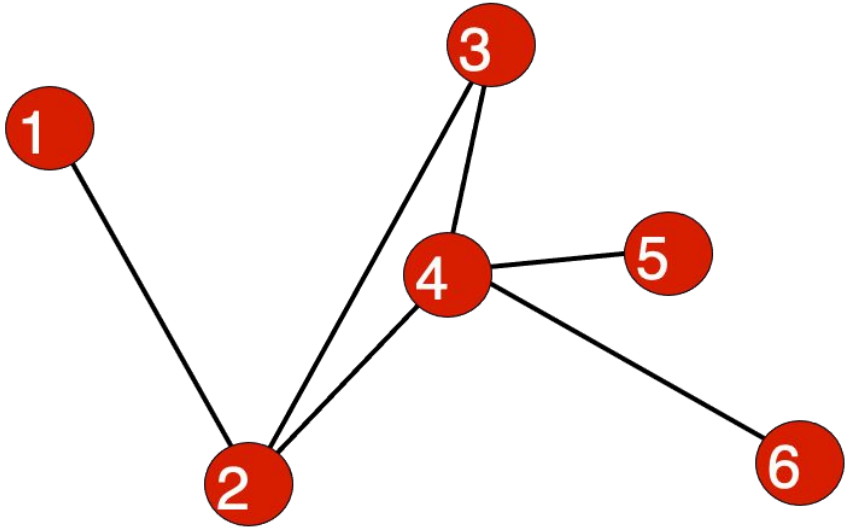



```
import networkx as nx # always!

G = nx.Graph()
G.add_node(1)
G.add_nodes_from([2,3,…])
…
G.add_edge(1,2)
G.add_edges_from([(2,3),(2,4),…])
…
G.nodes()
G.edges()
G.neighbors(4)

for n in G.nodes:
    print(n, G.neighbors(n))
for u,v in G.edges:
    print(u, v)
```

# How to create a graph with NetworkX?



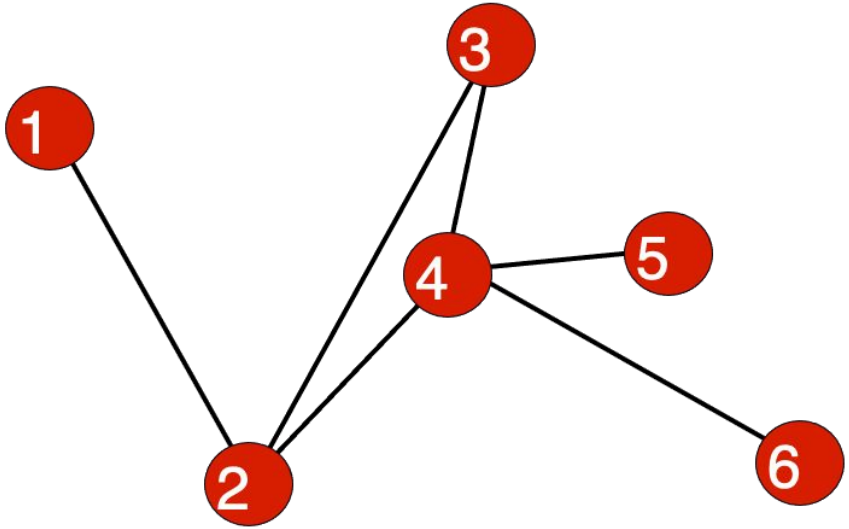

```
import networkx as nx # always!

G = nx.Graph()
G.add_node(1)
G.add_nodes_from([2,3,…])
…
G.add_edge(1,2)
G.add_edges_from([(2,3),(2,4),…])
…
G.nodes()
G.edges()
G.neighbors(4)

for n in G.nodes:
    print(n, G.neighbors(n))
for u,v in G.edges:
    print(u, v)
```

**Importing the NetworkX Module**

# How to create a graph with NetworkX?



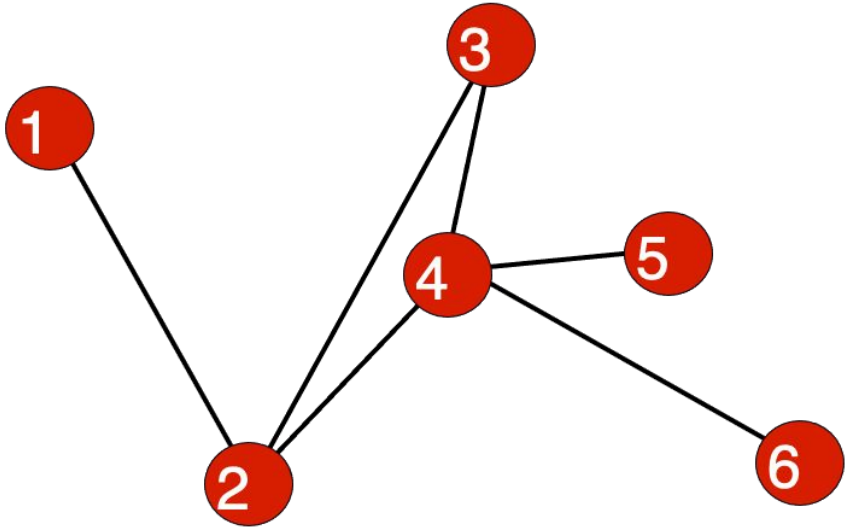

```
import networkx as nx # always!

G = nx.Graph()
G.add_node(1)
G.add_nodes_from([2,3,…])
…
G.add_edge(1,2)
G.add_edges_from([(2,3),(2,4),…])
…
G.nodes()
G.edges()
G.neighbors(4)

for n in G.nodes:
    print(n, G.neighbors(n))
for u,v in G.edges:
    print(u, v)
```

**Create an Undirected Graph. Object of type networkx.classes.graph.Graph**

# How to create a graph with NetworkX?



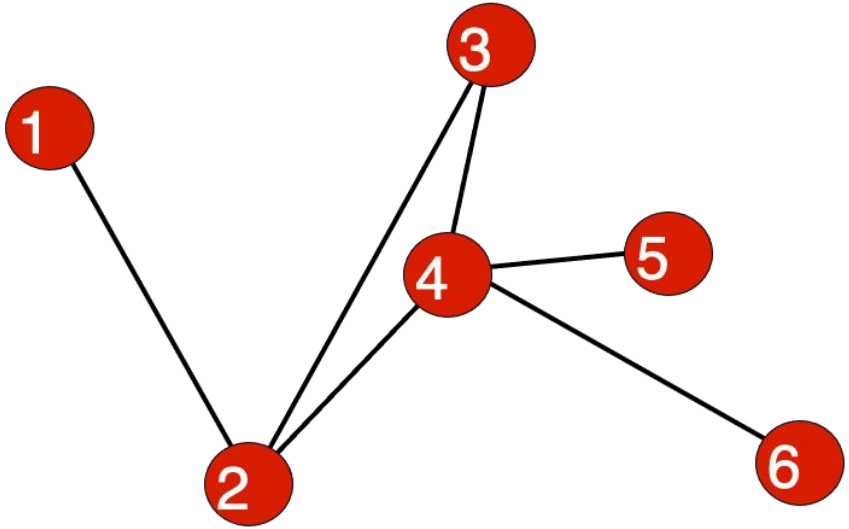

```
import networkx as nx # always!

G = nx.Graph()
G.add_node(1)
G.add_nodes_from([2,3,…])
…
G.add_edge(1,2)
G.add_edges_from([(2,3),(2,4),…])
…
G.nodes()
G.edges()
G.neighbors(4)

for n in G.nodes:
    print(n, G.neighbors(n))
for u,v in G.edges:
    print(u, v)
```

**Add a single node to the graph. The argument can be any hashable type (such as a number, string, tuple, etc.), used to uniquely identify the node**

# How to create a graph with NetworkX?



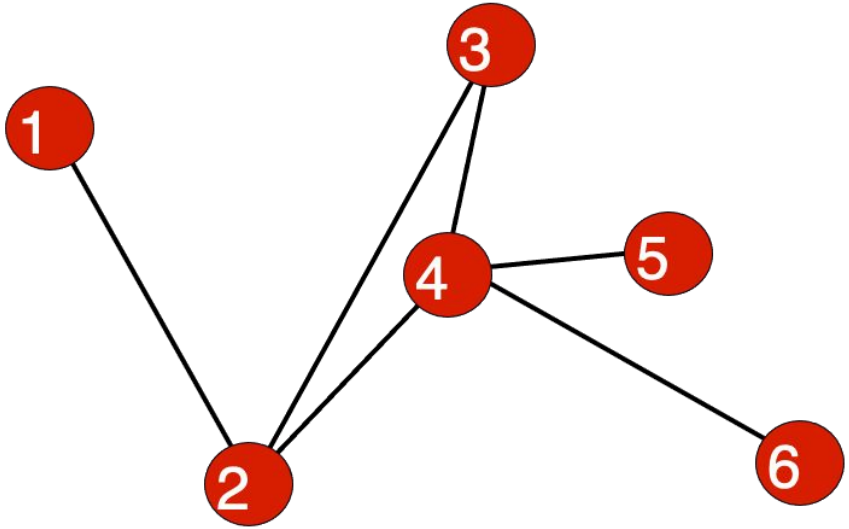

```
import networkx as nx # always!

G = nx.Graph()
G.add_node(1)
G.add_nodes_from([2,3,…])
…
G.add_edge(1,2)
G.add_edges_from([(2,3),(2,4),…])
…
G.nodes()
G.edges()
G.neighbors(4)

for n in G.nodes:
    print(n, G.neighbors(n))
for u,v in G.edges:
    print(u, v)
```

**Add multiple nodes from an iterable of IDs (list, set, or any iterable containing hashable elements)**

# How to create a graph with NetworkX?



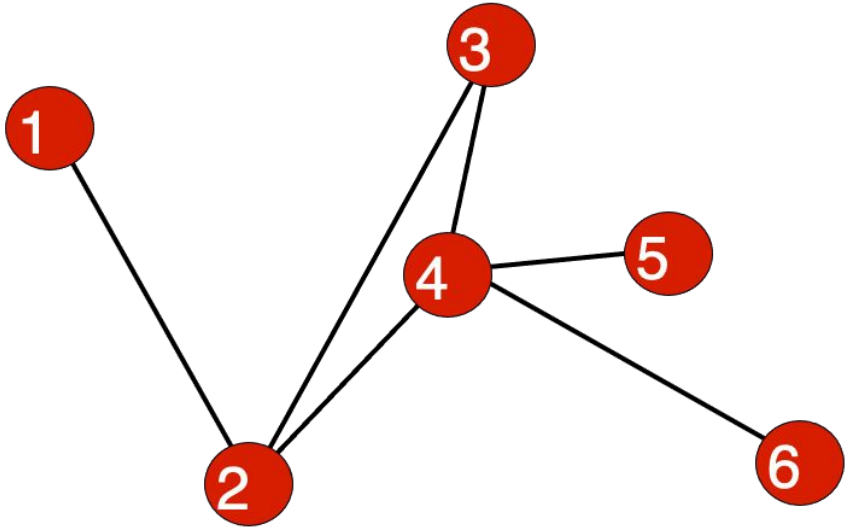

```
import networkx as nx # always!

G = nx.Graph()
G.add_node(1)
G.add_nodes_from([2,3,…])
…
G.add_edge(1,2)
G.add_edges_from([(2,3),(2,4),…])
…
G.nodes()
G.edges()
G.neighbors(4)

for n in G.nodes:
    print(n, G.neighbors(n))
for u,v in G.edges:
    print(u, v)
```

**Add an edge between nodes "u" and "v" in the graph. If "u" or "v" does not exist in the graph, they will be automatically added as nodes**

# How to create a graph with NetworkX?



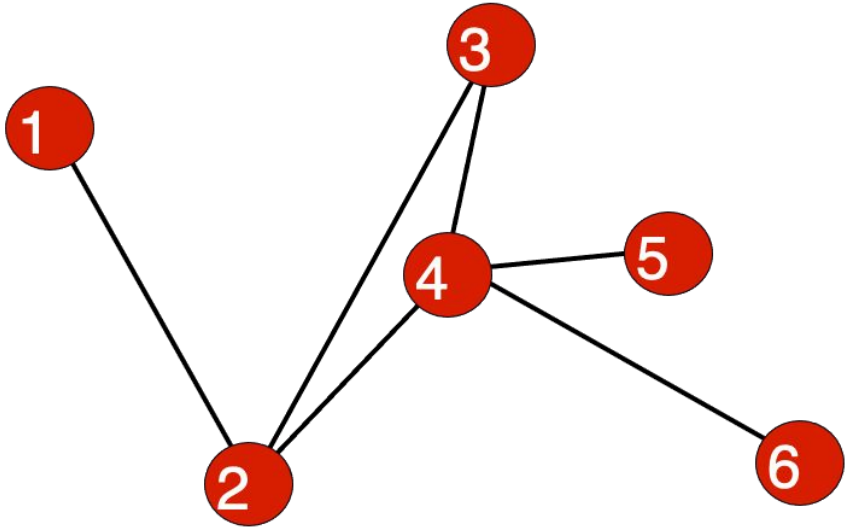

```
import networkx as nx # always!

G = nx.Graph()
G.add_node(1)
G.add_nodes_from([2,3,…])
…
G.add_edge(1,2)
G.add_edges_from([(2,3),(2,4),…])
…
G.nodes()
G.edges()
G.neighbors(4)

for n in G.nodes:
    print(n, G.neighbors(n))
for u,v in G.edges:
    print(u, v)
```

**Add multiple edges from an iterable of pairs "edges," where each pair is a tuple (u, v) representing an edge between nodes "u" and "v"**

# How to create a graph with NetworkX?



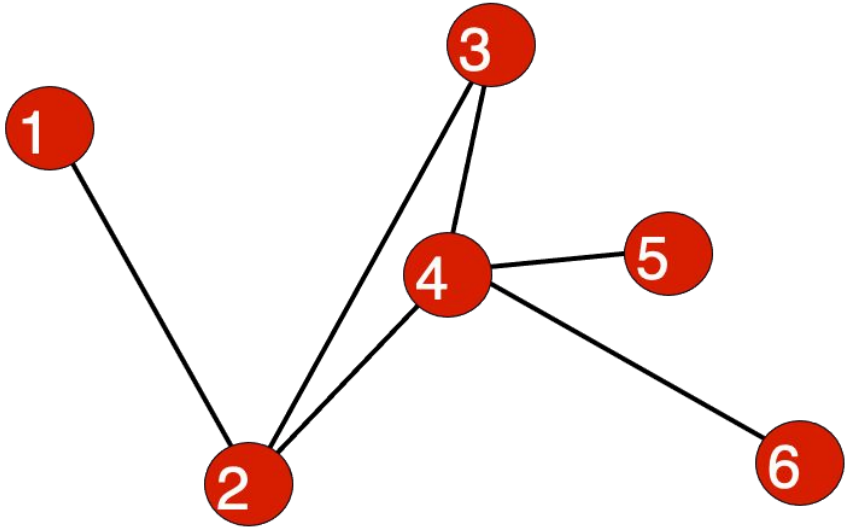

```
import networkx as nx # always!

G = nx.Graph()
G.add_node(1)
G.add_nodes_from([2,3,…])
…
G.add_edge(1,2)
G.add_edges_from([(2,3),(2,4),…])
…
G.nodes()
G.edges()
G.neighbors(4)

for n in G.nodes:
    print(n, G.neighbors(n))
for u,v in G.edges:
    print(u, v)
```

**Returns a NodeView object containing all nodes of graph G, which behaves similarly to a set, allowing iteration over the nodes**

# How to create a graph with NetworkX?



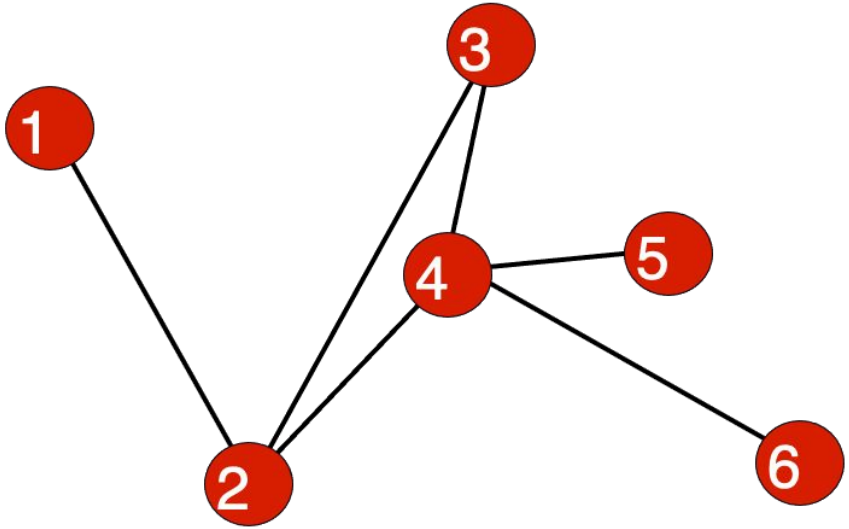

```
import networkx as nx # always!

G = nx.Graph()
G.add_node(1)
G.add_nodes_from([2,3,…])
…
G.add_edge(1,2)
G.add_edges_from([(2,3),(2,4),…])
…
G.nodes()
G.edges()
G.neighbors(4)

for n in G.nodes:
    print(n, G.neighbors(n))
for u,v in G.edges:
    print(u, v)
```

**Returns an EdgeView object with all edges of graph G. Each edge is represented as a tuple (u, v), where u and v are the nodes connected by the edge**

# How to create a graph with NetworkX?



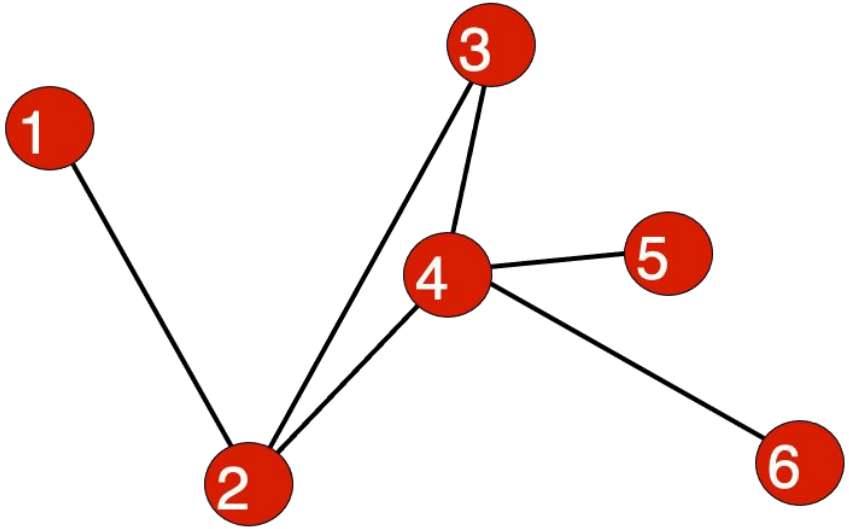

```
import networkx as nx # always!

G = nx.Graph()
G.add_node(1)
G.add_nodes_from([2,3,…])
…
G.add_edge(1,2)
G.add_edges_from([(2,3),(2,4),…])
…
G.nodes()
G.edges()
G.neighbors(4)

for n in G.nodes:
    print(n, G.neighbors(n))
for u,v in G.edges:
    print(u, v)
```

**Takes the ID of a node "n" and returns an iterator over all neighbors of node "n"**

# How to create a graph with NetworkX?



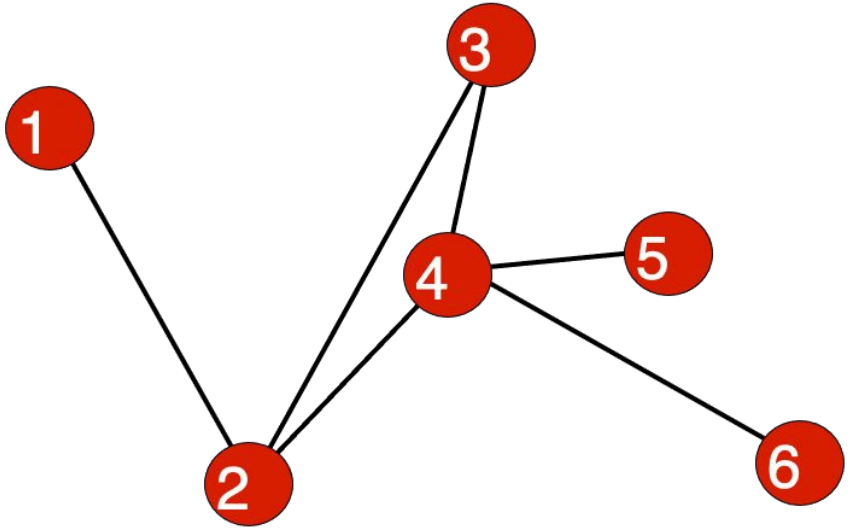

```
import networkx as nx # always!

G = nx.Graph()
G.add_node(1)
G.add_nodes_from([2,3,…])
…
G.add_edge(1,2)
G.add_edges_from([(2,3),(2,4),…])
…
G.nodes()
G.edges()
G.neighbors(4)

for n in G.nodes:
    print(n, G.neighbors(n))
for u,v in G.edges:
    print(u, v)
```

**Iterating over the nodes and their neighbors**

# How to create a graph with NetworkX?



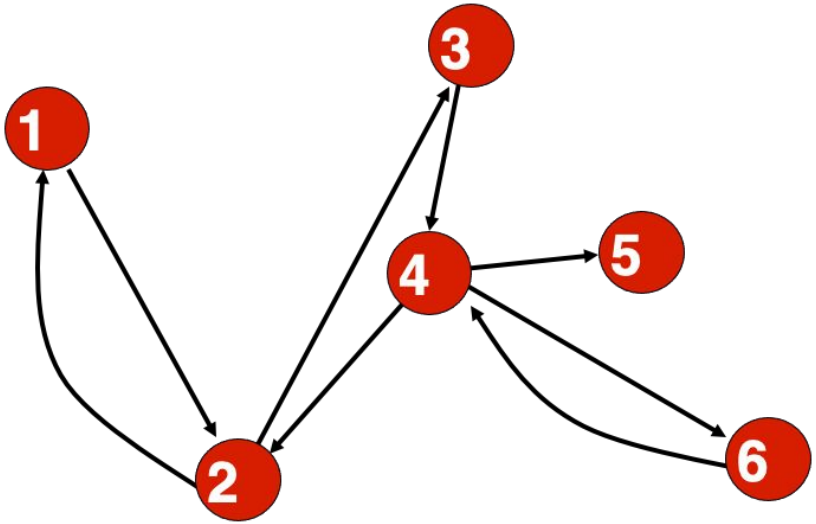

```
import networkx as nx # always!

G = nx.Graph()
G.add_node(1)
G.add_nodes_from([2,3,…])
…
G.add_edge(1,2)
G.add_edges_from([(2,3),(2,4),…])
…
G.nodes()
G.edges()
G.neighbors(4)

for n in G.nodes:
    print(n, G.neighbors(n))
for u,v in G.edges:
    print(u, v)
```

**Iterating over the edges of the graph**

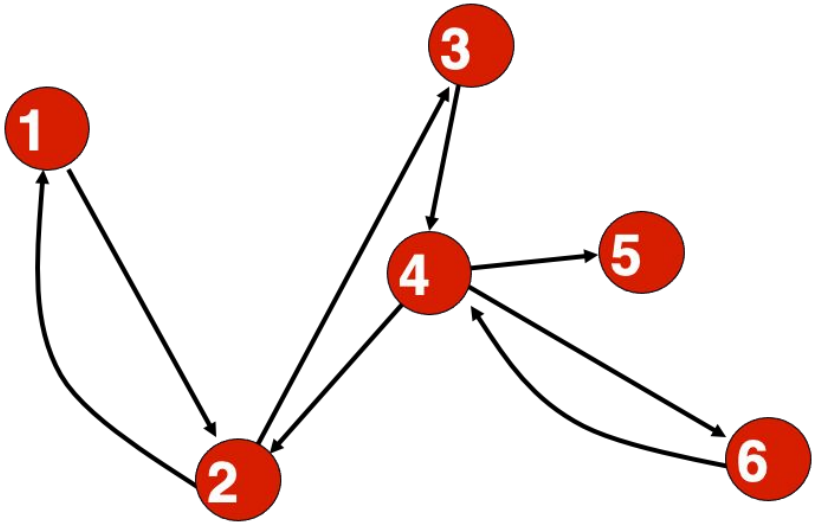
# How to create a graph with NetworkX?




```
import networkx as nx # don't forget!

D = nx.DiGraph()
D.add_edge(1,2)
D.add_edge(2,1)
D.add_edges_from([(2,3),(3,4),…])
…
D.number_of_nodes()
D.number_of_edges()
D.edges()
D.successors(2)
D.predecessors(2)
D.neighbors(2)
```

# How to create a graph with NetworkX?

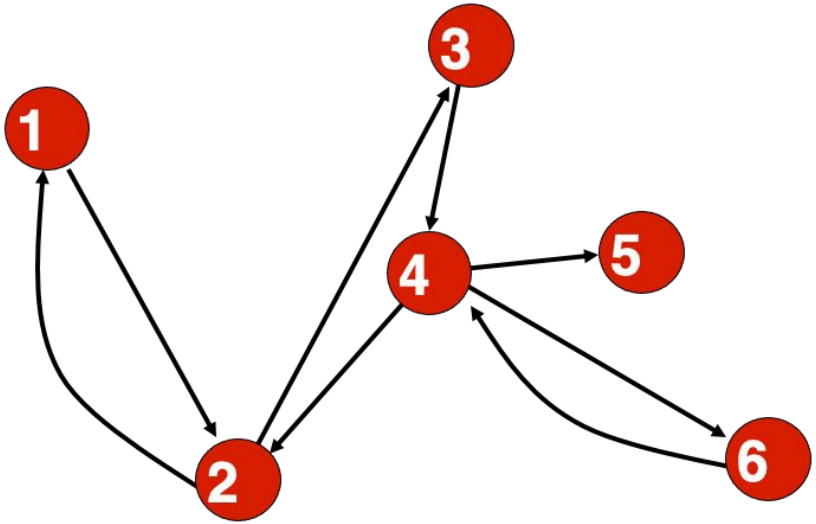



```
import networkx as nx # don't forget!

D = nx.DiGraph()
D.add_edge(1,2)
D.add_edge(2,1)
D.add_edges_from([(2,3),(3,4),…])
…
D.number_of_nodes()
D.number_of_edges()
D.edges()
D.successors(2)
D.predecessors(2)
D.neighbors(2)
```

**Creates a directed graph with an object type of networkx.classes.digraph.DiGraph**

# How to create a graph with NetworkX?

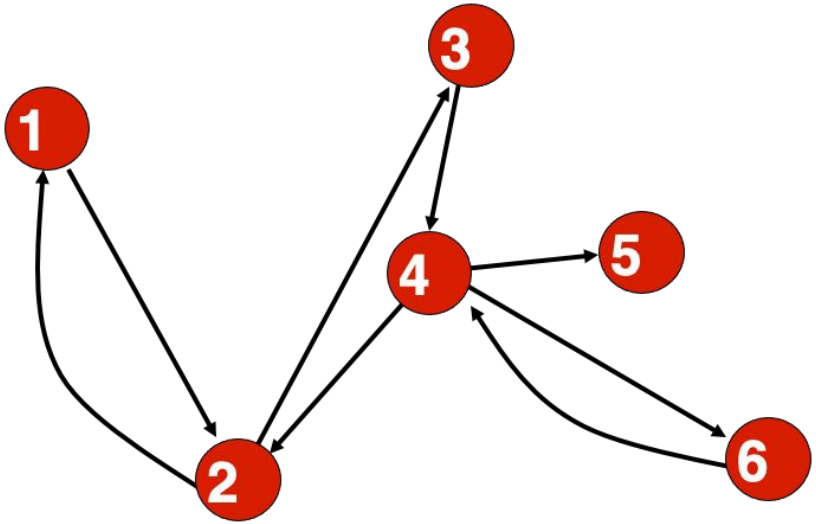



```
import networkx as nx # don't forget!

D = nx.DiGraph()
D.add_edge(1,2)
D.add_edge(2,1)
D.add_edges_from([(2,3),(3,4),…])
…
D.number_of_nodes()
D.number_of_edges()
D.edges()
D.successors(2)
D.predecessors(2)
D.neighbors(2)
```

**Addition of edges as before**

# How to create a graph with NetworkX?

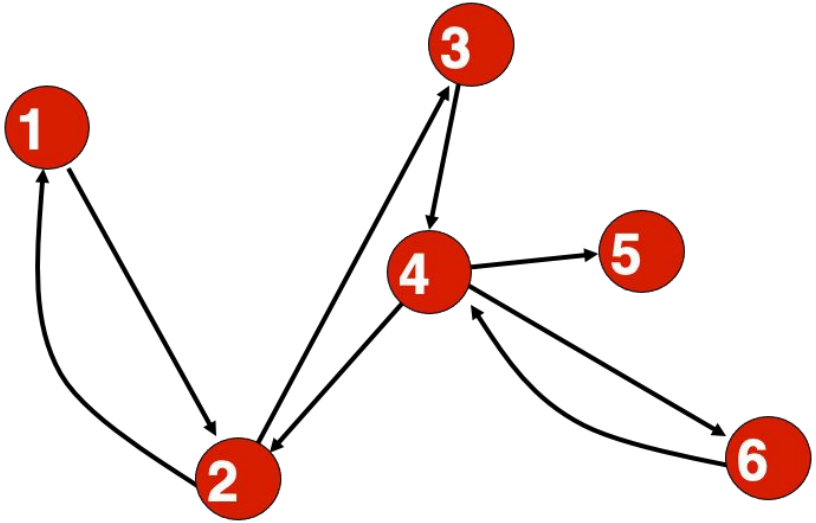



```
import networkx as nx # don't forget!

D = nx.DiGraph()
D.add_edge(1,2)
D.add_edge(2,1)
D.add_edges_from([(2,3),(3,4),…])
…
D.number_of_nodes()
D.number_of_edges()
D.edges()
D.successors(2)
D.predecessors(2)
D.neighbors(2)
```

**Methods that allow obtaining the number of edges and nodes in the graph**

# How to create a graph with NetworkX?

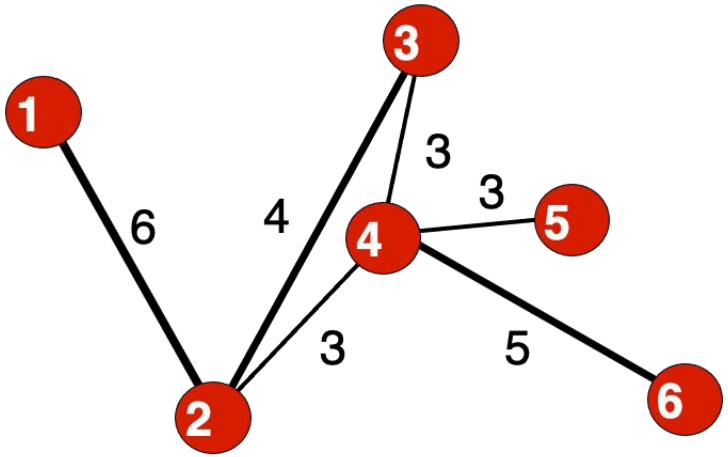



```
import networkx as nx # don't forget!

D = nx.DiGraph()
D.add_edge(1,2)
D.add_edge(2,1)
D.add_edges_from([(2,3),(3,4),…])
…
D.number_of_nodes()
D.number_of_edges()
D.edges()
D.successors(2)
D.predecessors(2)
D.neighbors(2)
```

**Methods that allow iterating over the neighbors of a node "n" with and without order**

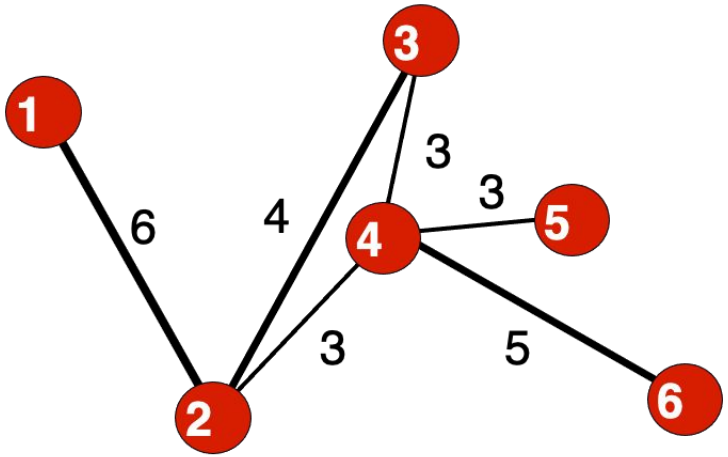
# How to create a graph with NetworkX?




```
W = nx.Graph()
W.add_edge(1,2,weight=6)
…
W.add_weighted_edges_from([(4,5,3),(4,6,5),…])
…
W.edges()
W.edges(data='weight')
for (u,v,d) in W.edges(data='weight'):
    if d>3:
        print('(%d, %d, %d)'%(u,v,d))
```

**Creation of an undirected and weighted edge.**
**The attribute "weight" is used to identify the weights**

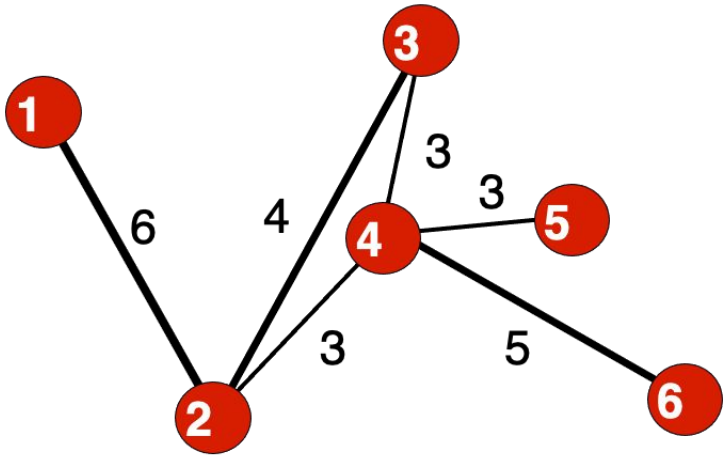
# How to create a graph with NetworkX?




```
W = nx.Graph()
W.add_edge(1,2,weight=6)
…
W.add_weighted_edges_from([(4,5,3),(4,6,5),…])
…
W.edges()
W.edges(data='weight')
for (u,v,d) in W.edges(data='weight'):
    if d>3:
        print('(%d, %d, %d)'%(u,v,d))
```

**Other attributes can be added to an edge. For example, "relation = colleagues." But be mindful of memory consumption!**

# How to create a graph with NetworkX?

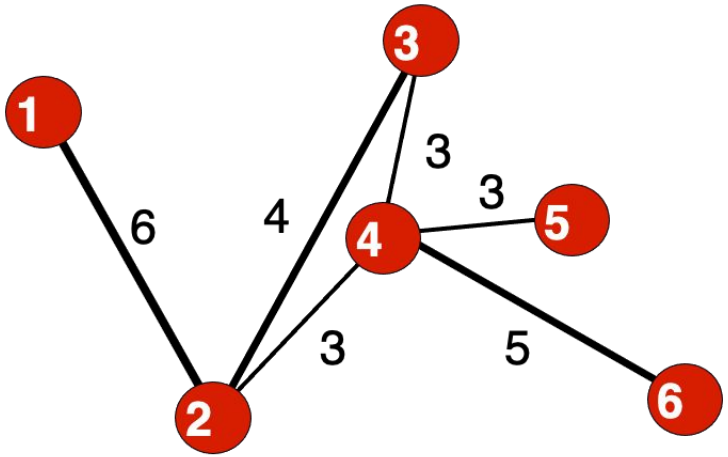



```
W = nx.Graph()
W.add_edge(1,2,weight=6)

W.add_weighted_edges_from([(4,5,3),(4,6,5),…])
…
W.edges()
W.edges(data='weight')
for (u,v,d) in W.edges(data='weight'):
    if d>3:
        print('(%d, %d, %d)'%(u,v,d))
```

**Addition of multiple weighted edges at once**

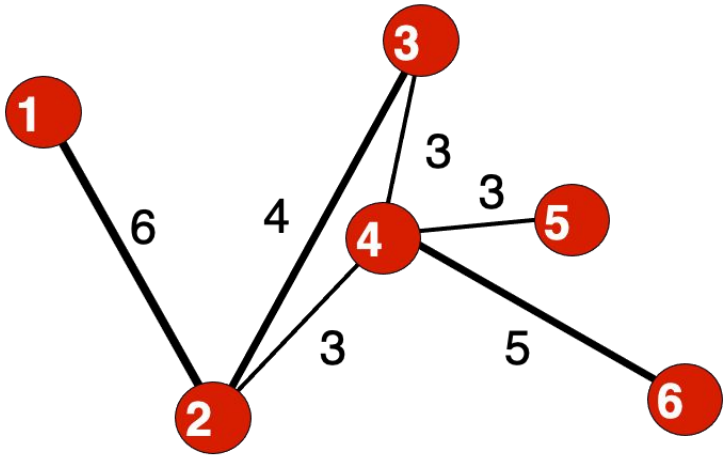
# How to create a graph with NetworkX?




```
W = nx.Graph()
W.add_edge(1,2,weight=6)
…
W.add_weighted_edges_from([(4,5,3),(4,6,5),…])
…
W.edges()
W.edges(data='weight')
for (u,v,d) in W.edges(data='weight'):
    if d>3:
        print('(%d, %d, %d)'%(u,v,d))
```

**It is possible to obtain an iterator of the edges with and without attributes**

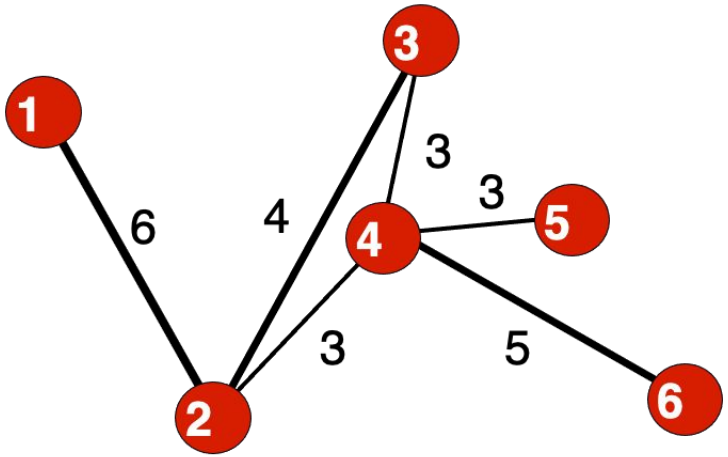
# How to create a graph with NetworkX?




```
W = nx.Graph()
W.add_edge(1,2,weight=6)
…
W.add_weighted_edges_from([(4,5,3),(4,6,5),…])
…
W.edges()
W.edges(data='weight')
for (u,v,d) in W.edges(data='weight'):
    if d>3:
        print('(%d, %d, %d)'%(u,v,d))
```

**Returns an "EdgeDataView" object that behaves similarly to a list of tuples**

# How to create a graph with NetworkX?

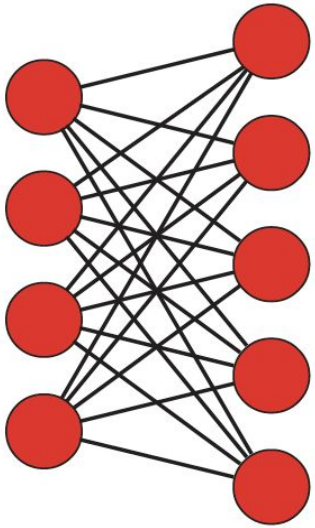



```
W = nx.Graph()
W.add_edge(1,2,weight=6)
…
W.add_weighted_edges_from([(4,5,3),(4,6,5),…])
…
W.edges()
W.edges(data='weight')
for (u,v,d) in W.edges(data='weight'):
    if d>3:
        print('(%d, %d, %d)'%(u,v,d))
```

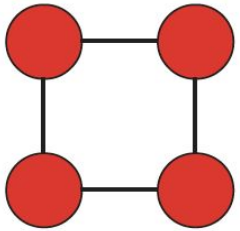
**Use data=True to fetch more than one attribute for each edge. This will return a list of tuples in the format (u, v, attr_dict), where "attr_dict" is a dictionary containing all the edge attributes**

# Other types of graphs

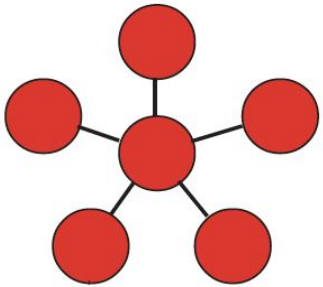
B = nx.complete_bipartite_graph(4,5)



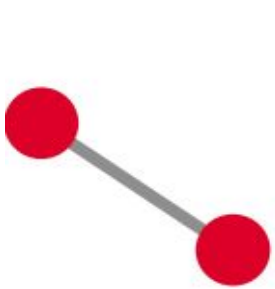
C = nx.cycle_graph(4)



S = nx.star_graph(6)

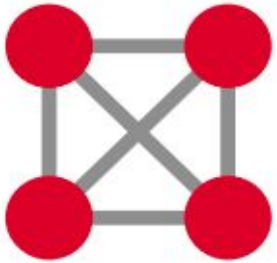


P = nx.path_graph(5)

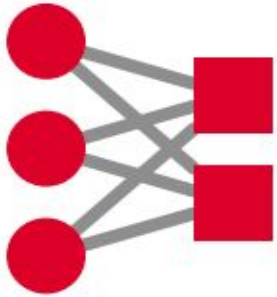


(a) 2-clique.

(b) 3-clique.



(c) 4-clique.



(d) 3,2-clique.

# Roadmap

Graph Modeling Fundamentals

**Structural and Centrality Analysis**

Community Detection

Final Considerations

# Structural Measures: Density and Sparsity

**Network density** is the ratio of actual links to possible links, representing the proportion of connected pairs of nodes

A **complete network** has a maximum **density of one**, indicating that all pairs of nodes are connected

In most systems, the number of links is far below the maximum, as most node pairs are not directly connected

- This condition characterizes the network as being **sparse**

# Structural Measures: Density and Sparsity

The maximum number of links in **an undirected network** with "N" nodes is given by the number of distinct pairs of nodes

$$L_{max} = \binom{N}{2} = \frac{N(N-1)}{2}$$

For a **directed network**:

$$L_{max} = N(N-1)$$

# Structural Measures: Density and Sparsity

Thus, the density "*d*" of a network is calculated by:

$$d = \frac{L}{L_{max}}$$

**Undirected**

$$d = \frac{L}{L_{max}}$$

**Directed**

```
G.number_of_nodes()
G.number_of_edges()
nx.density(G)
nx.density(D)

CG = nx.complete_graph(8471)
print(nx.density(CG)) # what does this print?
```

# Structural Measures: Density and Sparsity

## Example: The U.S. Congressional Network

**Complete network**

# Structural Measures: Density and Sparsity

## Example: The U.S. Congressional Network


**Complete network**

# Structural Measures: Density and Sparsity

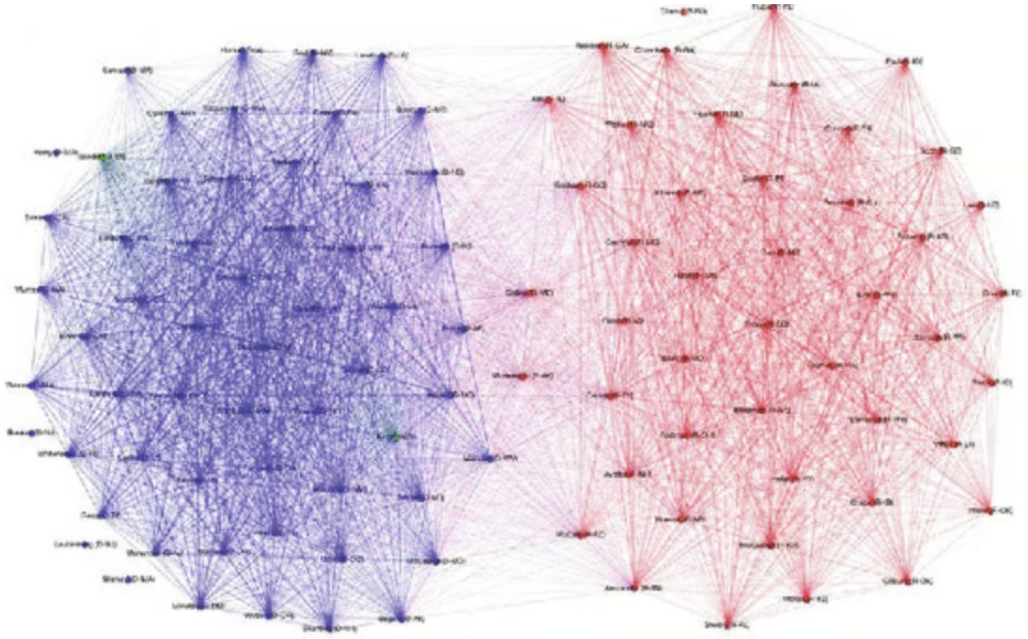
## Example: The U.S. Congressional Network

Edges link two congress members based on the number of voting sessions in which they agreed

**Complete network**

# Structural Measures: Density and Sparsity

## Example: The U.S. Congressional Network



**Complete network**

**Sparse network**

# Structural Measures: Degree

The degree of a node refers to its number of links or neighbors

It is typically denoted as $k_i$ to represent the degree of node *i*

A node with no neighbors is called **isolated** (*k=0*)

```
G.degree(2) # returns the degree of node 2
G.degree()  # dict with the degree of all nodes of G
```

# Structural Measures: Degree

In a **directed network**, the following degree concepts are defined:

- **In degree:** # links entering a node from another node $k^{out}_i$
- **Out degree:** # links leaving a node toward another node $k^{out}_i$

In NetworkX, it is possible to use:

```
D.in_degree(4)
D.out_degree(4)
D.degree(4)
```

# Structural Measures: Weighted Degree (Strength)

In **weighted networks**, there is also the concept of weighted degree (also known as **"strength"**)

$$s_i = \sum_j w_{ij}$$

**Directed**

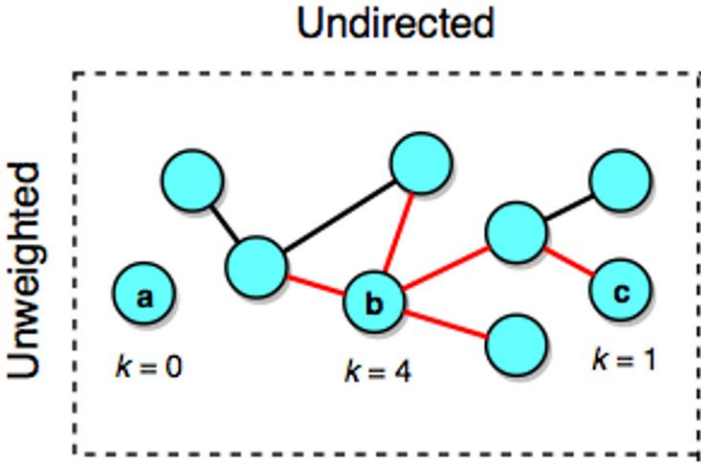
$$s_i^{in} = \sum_j w_{ji} \qquad s_i^{out} = \sum_j w_{ij}$$

**Undirected**

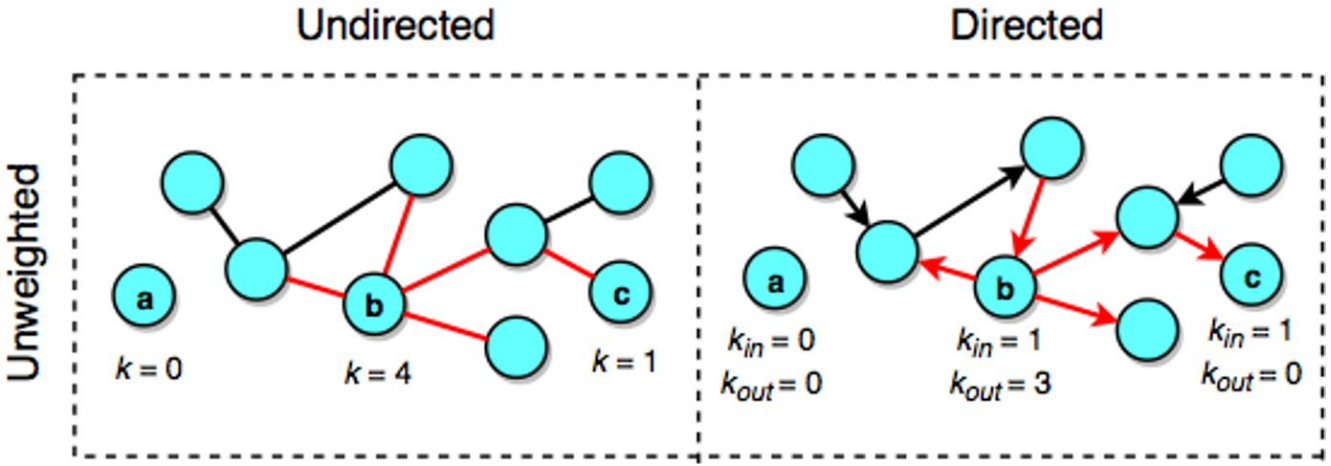
It represents the sum of the weights of all edges of the neighboring nodes $j$ incident to the node in question (node $i$)

```
G.degree(4) # degree
G.degree(4, weight='weight') # strength
```

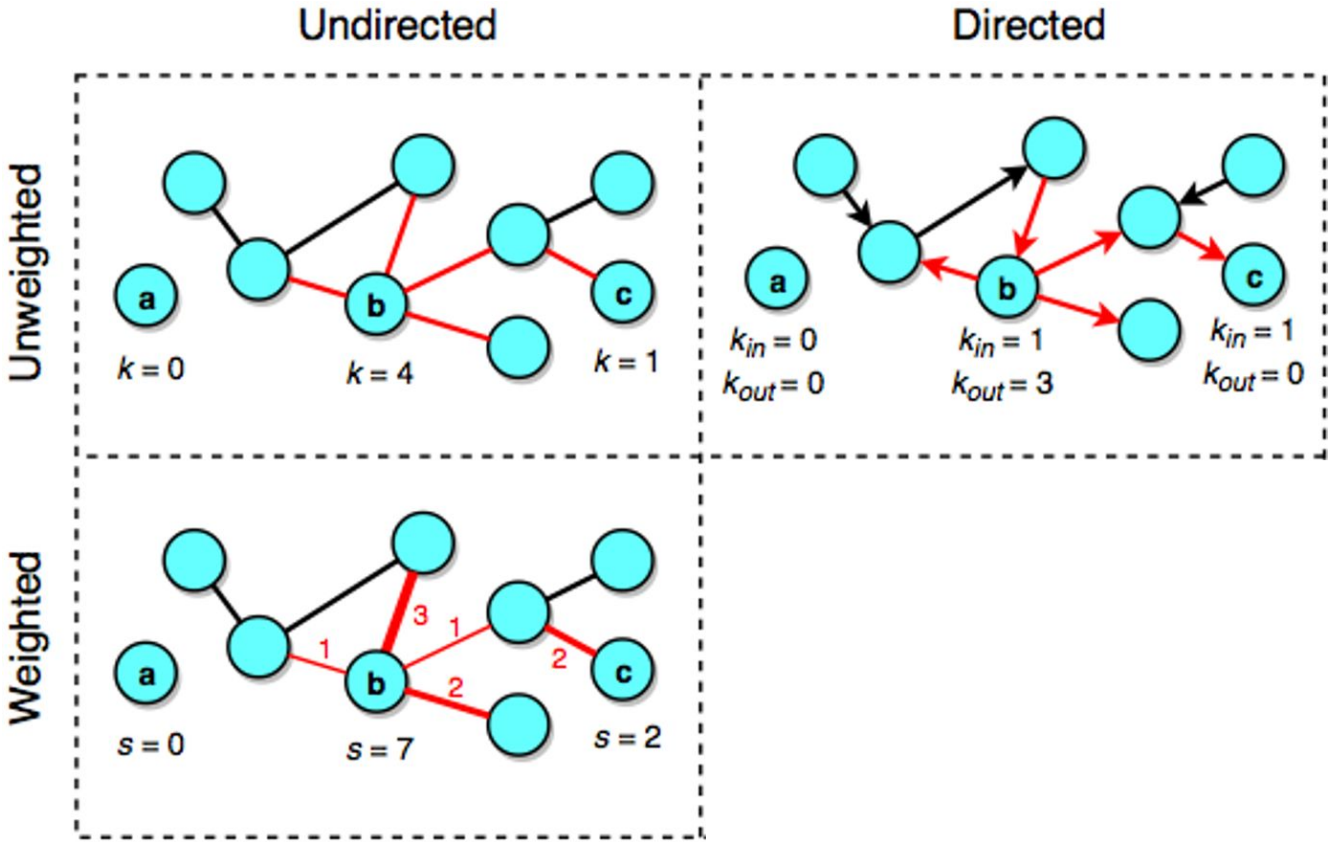
# Structural Measures: Weighted Degree (Strength)



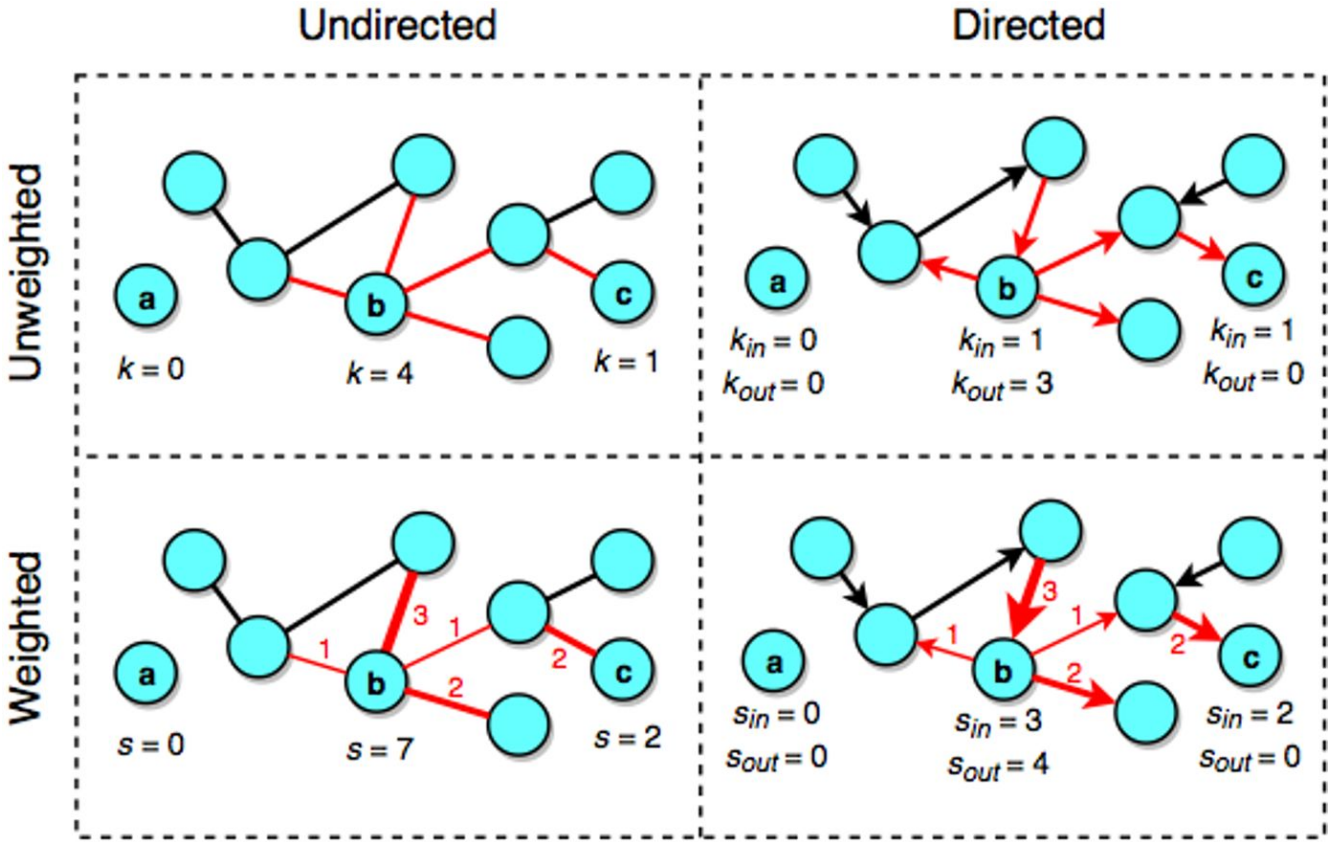
# Structural Measures: Weighted Degree (Strength)



# Structural Measures: Weighted Degree (Strength)



# Structural Measures: Weighted Degree (Strength)



# Structural Measures: Average Degree

The **average degree** of a network is the **average number of connections per node**

$$\langle k \rangle = \frac{\sum_i k_i}{N}$$

The same applies to the **average inbound degree** and **average outbound degree** in directed networks

The measures of degree and density are closely related

Function in NetworkX: “nx.average_degree_connectivity”

# How to identify important nodes in the network?

Several criteria are used to define the importance of a node in the network. Many of them are related to the definition of **centrality**

# How to identify important nodes in the network?

Several criteria are used to define the importance of a node in the network. Many of them are related to the definition of **centrality**

For example, which nodes in a network allow:

- **Reach more nodes immediately:** Nodes with the highest number of immediate connections (**Degree Centrality**)
- **Quickly access all nodes:** Nodes in the network with the fewest steps to all other nodes (**Closeness Centrality**)
- **Maintain bridges to keep the network connected:** Crucial nodes for ensuring that different parts of the network remain connected (**Betweenness Centrality**)

# How to identify important nodes in the network?

All these measures of centrality are based on the structure and connections of the network

They can be classified as:

- **Local:** Based on neighborhood and have **low computational cost**
  - **Metrics:** Degree, In Degree and Out Degree
- **Global:** Based on the entire structure of the network and have **high computational cost**
  - **Medidas:** Closeness, Betweenness, PageRank, among others!

**Which is the best?** It depends on the context!

# Degree Centrality

The simplest and most intuitive measure of centrality is degree centrality

**Degree of a node:** the number of neighbors of the node: $k_i$

Average degree of the network: $\langle k \rangle = \frac{\sum_i k_i}{N} = \frac{2L}{N}$

- Recall: It is also available in an undirected version

Nodes with a high degree in the network are called ***Hubs***

# Degree Centrality

Very simple and low-cost metric

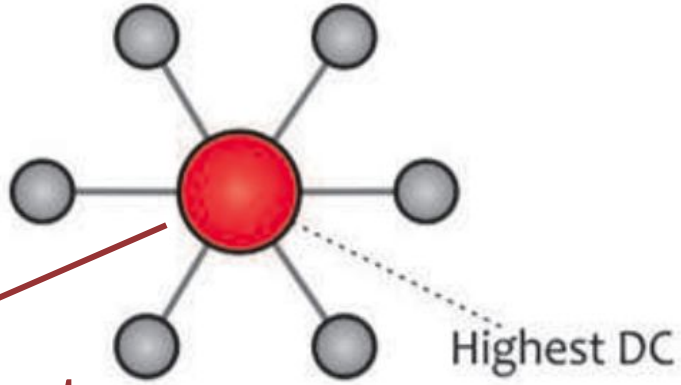
When does the degree of a node matter?

- It can indicate the ability to communicate quickly with many nodes
- It reflects the probability that information originating from anywhere in the network reaches and departs from a given node more quickly

In NetworkX:

```
G.degree(2) # returns the degree of node 2
G.degree()  # dict with the degree of all nodes of G
```

# Degree Centrality




**Node with the highest degree centrality in the network**

Fonte: https://www.frontiersin.org/journals/neuroscience/articles/10.3389/fnins.2019.00585/full

# Closeness Centrality

A node is more central the closer it is to all other nodes in the network

- I.e., measures how "close" a node is to the other nodes

If, **on average**, the distances to other nodes are short, their sum will be low, indicating that the node has high centrality

Closeness is defined as the inverse of the sum of the shortest distances from a node to all other nodes

# Closeness Centrality

$$g_i = \frac{1}{\sum_{j \neq i} \ell_{ij}}$$

Given a node *i*

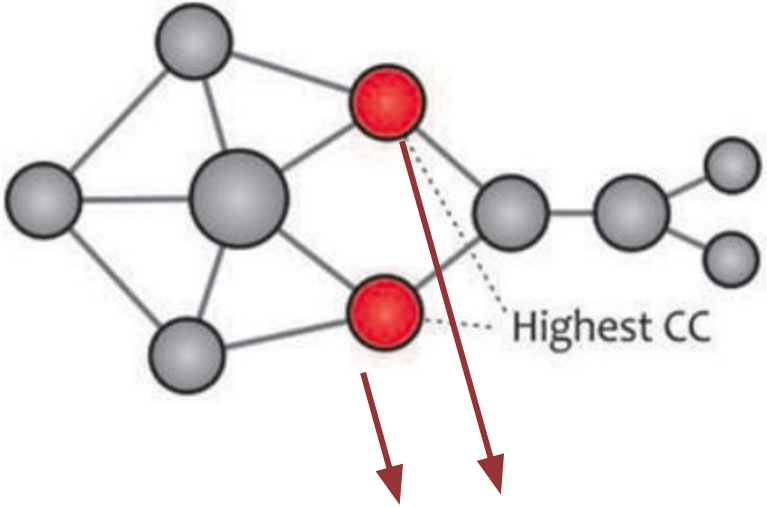
Sum of the minimum distances between the node *i* and all the other nodes in the network

No Networkx:

```
nx.closeness_centrality(G, node) # closeness centrality
                                 # of node
```

Fonte: https://www.frontiersin.org/journals/neuroscience/articles/10.3389/fnins.2019.00585/full

# Closeness Centrality




**These are the nodes that are, on average, closest to the other nodes in the network**

Fonte: https://www.frontiersin.org/journals/neuroscience/articles/10.3389/fnins.2019.00585/full

# *Betweenness* Centrality

It highlights the importance of a node as an **intermediary** in interactions within the network

It quantifies the number of times a node acts as a **bridge** along the shortest paths between two given nodes in the network

Nodes with higher betweenness have significant control over the flow of information/relationships in the network

- It can influence the speed and efficiency with which information propagates

# *Betweenness* Centrality

A node becomes more central according to the frequency with which it is used as a path in the shortest paths

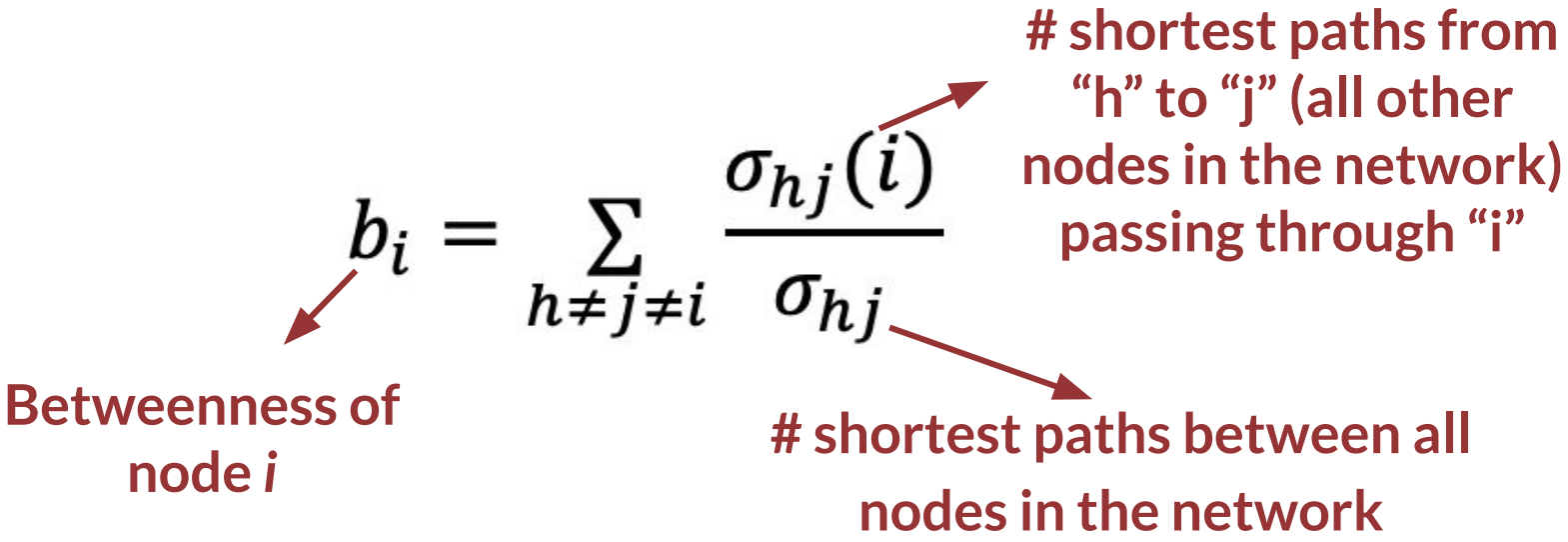
$$b_i = \sum_{h \neq j \neq i} \frac{\sigma_{hj}(i)}{\sigma_{hj}}$$

**Betweenness of node *i***

Fonte: https://www.frontiersin.org/journals/neuroscience/articles/10.3389/fnins.2019.00585/full

# *Betweenness* Centrality

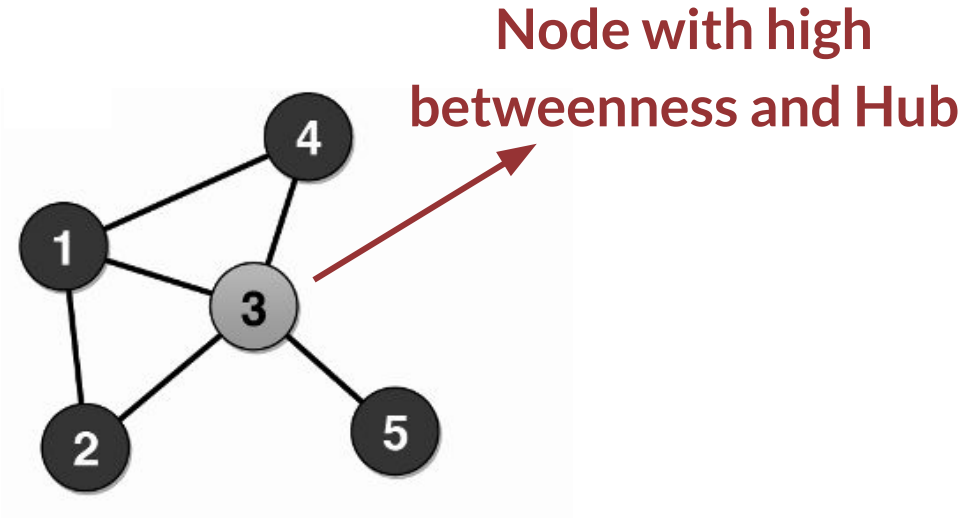
A node becomes more central according to the frequency with which it is used as a path in the shortest paths




Fonte: https://www.frontiersin.org/journals/neuroscience/articles/10.3389/fnins.2019.00585/full

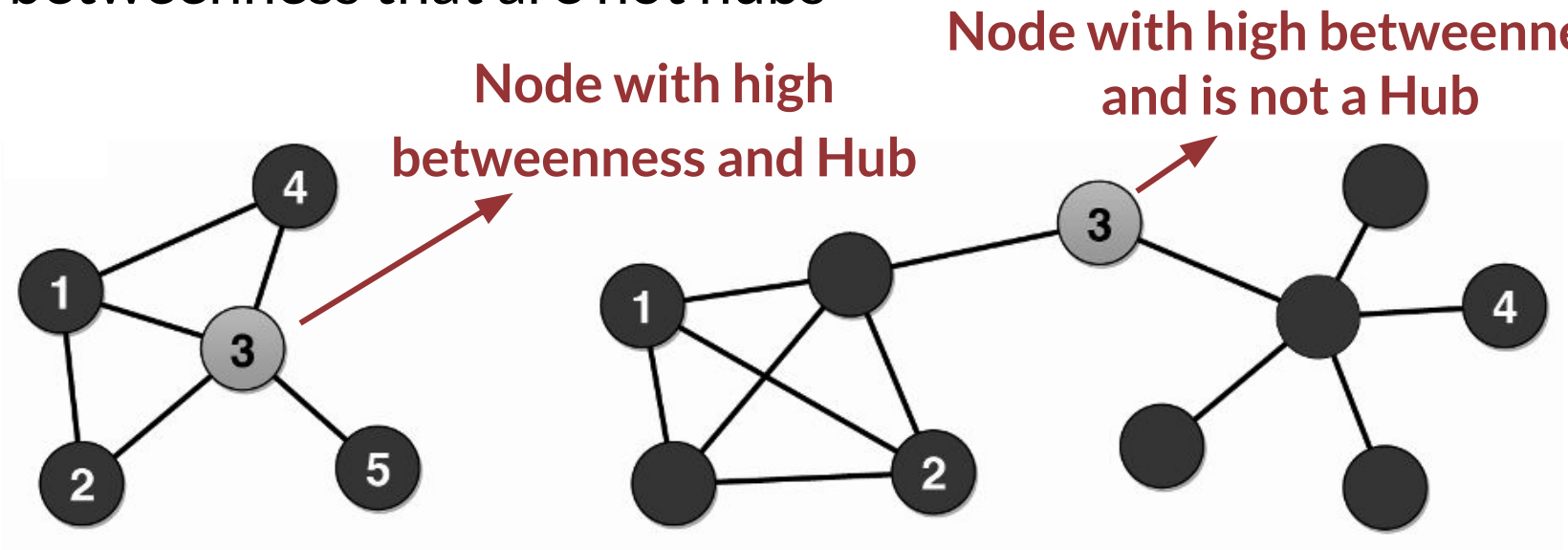
# *Betweenness* Centrality

Hubs generally have high betweenness, but there can be nodes with high betweenness that are not hubs



# *Betweenness* Centrality

Hubs generally have high betweenness, but there can be nodes with high betweenness that are not hubs



# *Betweenness* Centrality

This metric can be easily extended to edges with the same idea

- **Edge betweenness:** the fraction of shortest paths between all possible pairs of nodes that pass through the link

In Networkx:

```
nx.betweenness_centrality(G)        # dict nodes ->
                                    # betweenness centrality
nx.edge_betweenness_centrality(G) # dict links ->
                                    # betweenness centrality
```

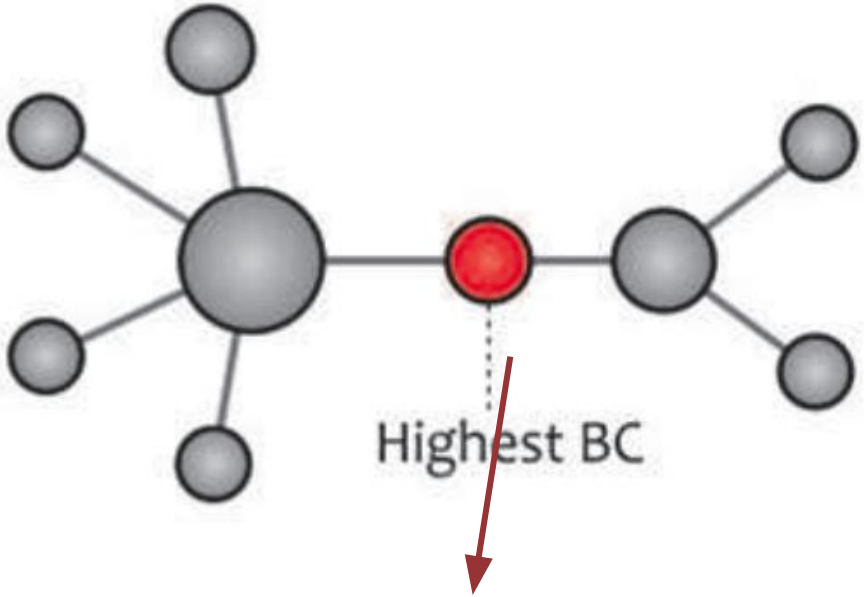
# *Betweenness* Centrality

Some important parameters:

- ***Normalized*:** Ensures that centrality values are between 0 and 1, facilitating comparison between nodes of graphs of different sizes

- ***Weights*:** Weights are used to calculate weighted shortest paths, and thus are interpreted as distances
  - If "None," all edge weights will be considered equal
  - Otherwise, it contains the name of the "weight" attribute used as the weight

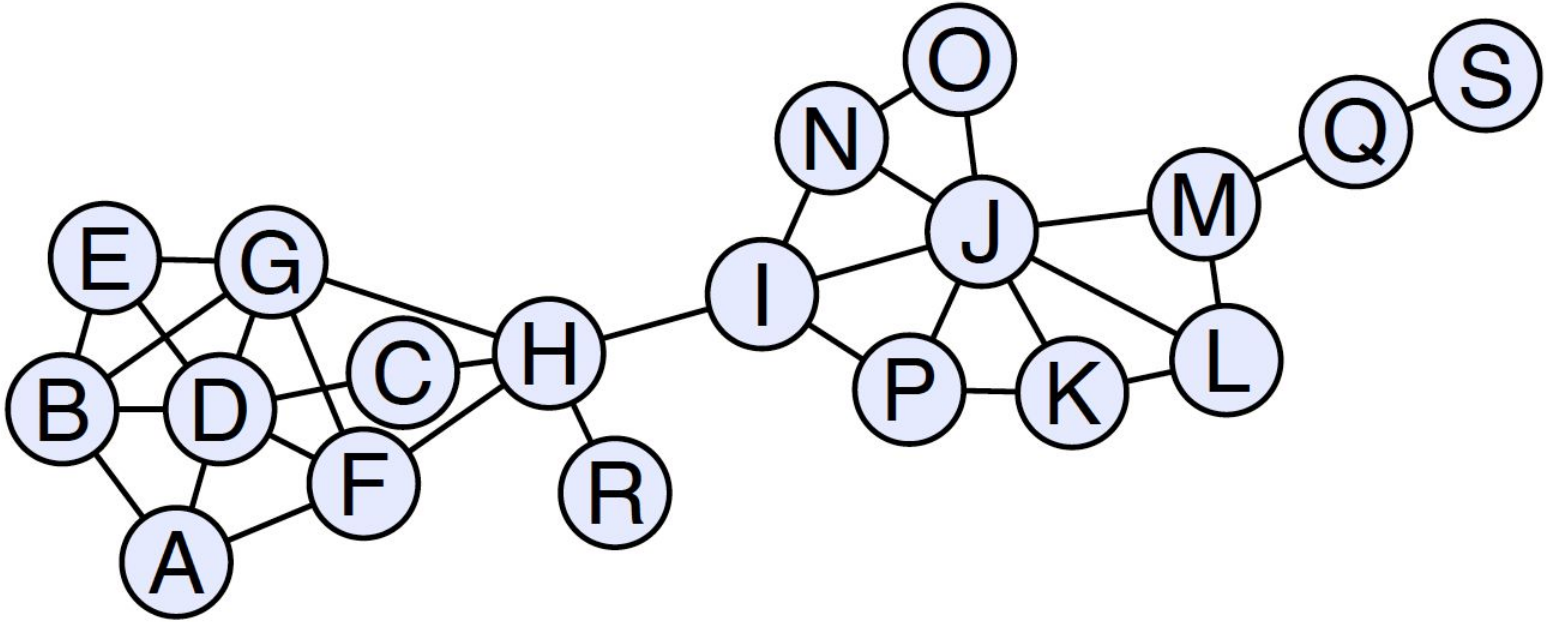
# *Betweenness* Centrality



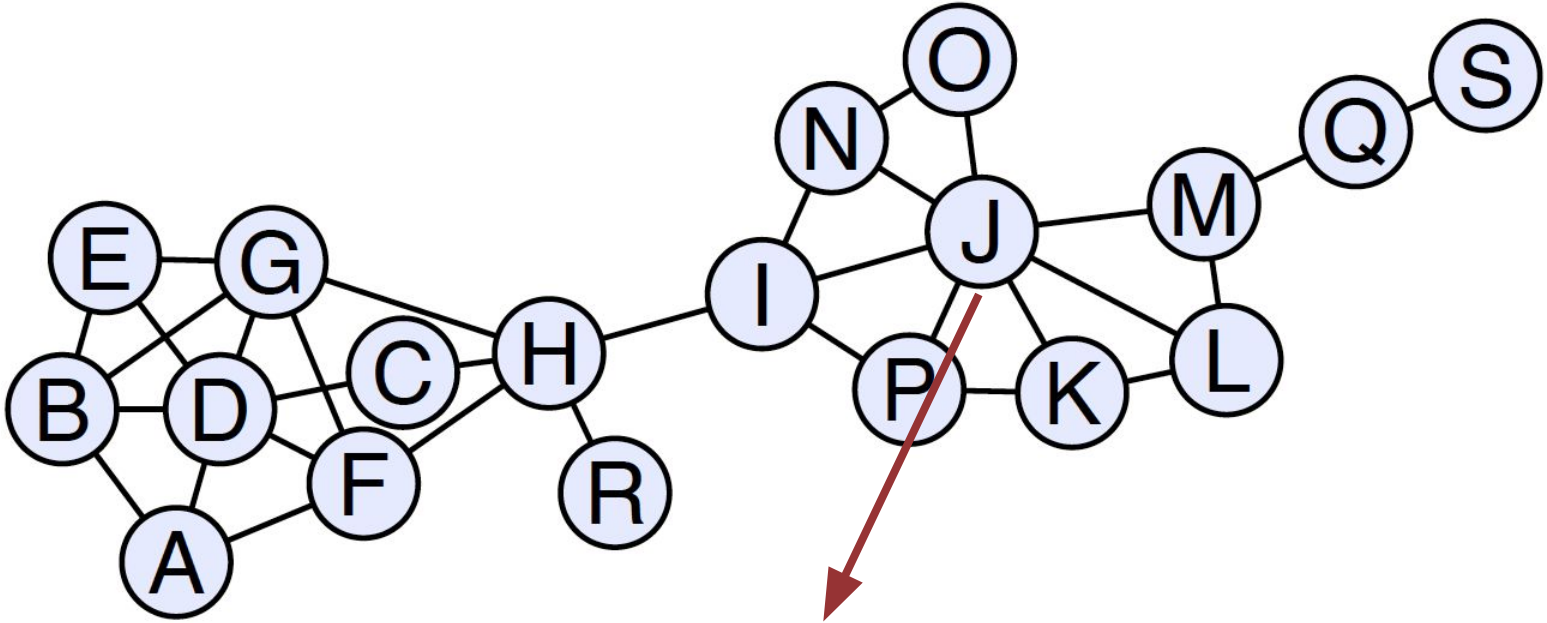

**Node with the highest Betweenness**

Fonte: https://www.frontiersin.org/journals/neuroscience/articles/10.3389/fnins.2019.00585/full

# Network Centrality: Each Measure a Perspective!

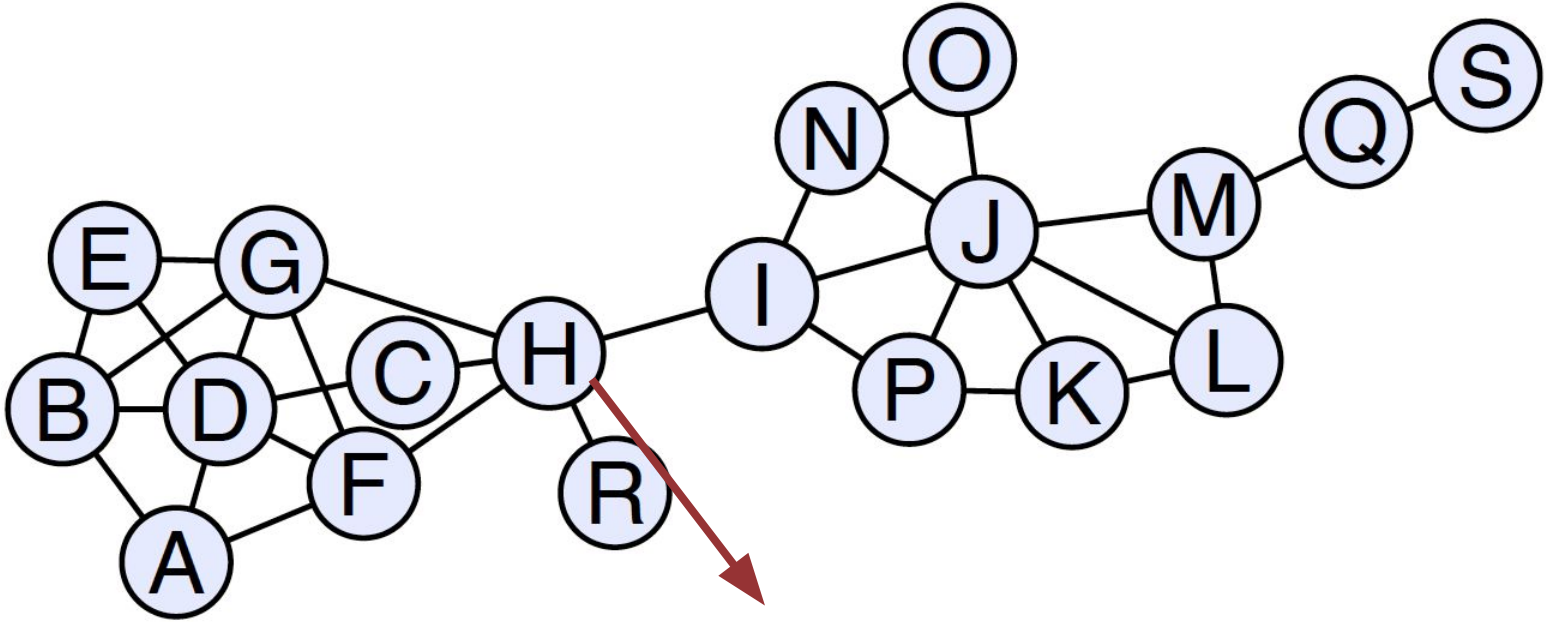


# Network Centrality: Each Measure a Perspective!



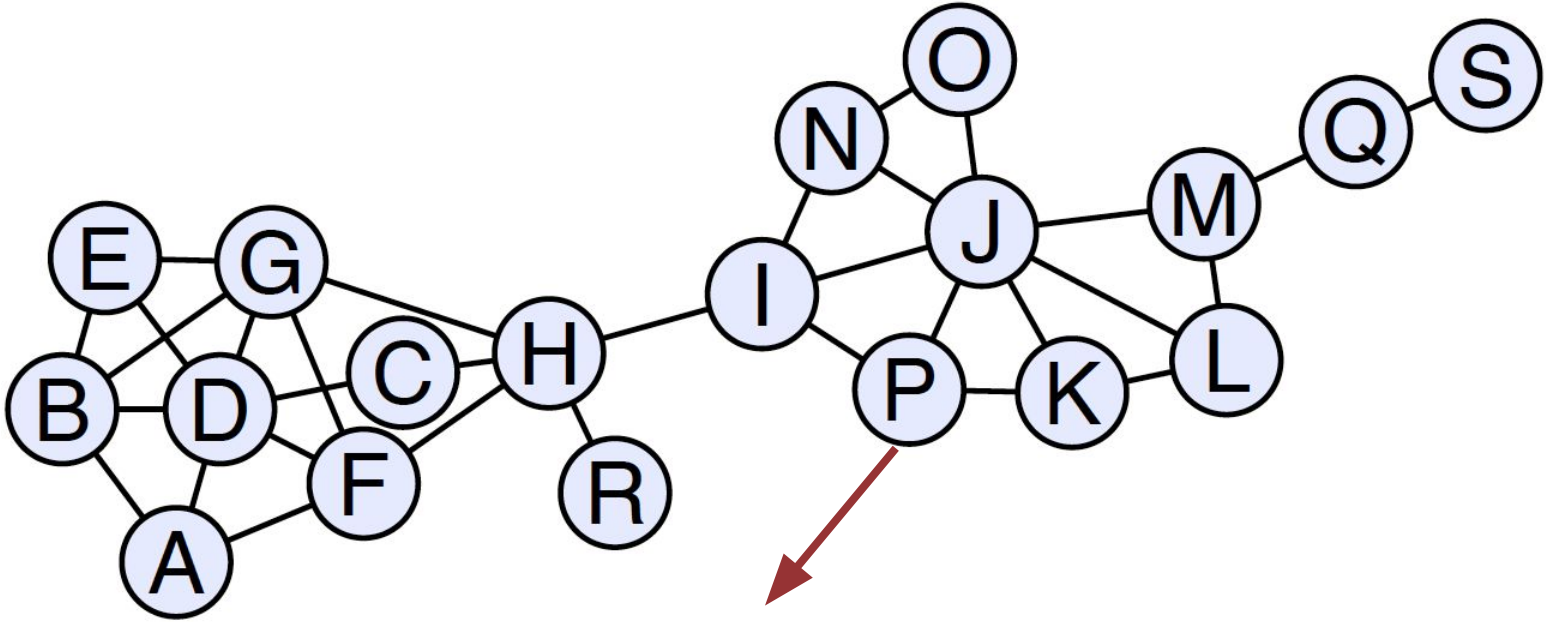

**Node with the highest degree centrality**

# Network Centrality: Each Measure a Perspective!



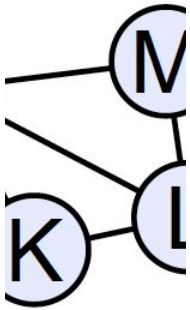

**Node with the highest betweenness!**

# Network Centrality: Each Measure a Perspective!




**Node with the highest closeness centrality**

# Network Centrality: Each Measure a Perspective!


Degree
Eigenvector
Closeness
Current Flow Closeness
(Shortest Path) Betweenness
Current Flow Betweenness
Communicability Betweenness
Group Centrality
Load
Subgraph
Harmonic Centrality
Dispersion
Reaching
Percolation
Second Order Centrality
Trophic
VoteRank
Laplacian

See here!

# Social Distance

The natural distance between two nodes in a network is the smallest number of edges to be crossed, known as the **shortest path**

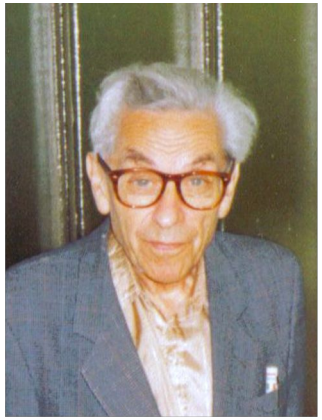
The length of the shortest path acts as a measure of distance, enabling the calculation of aggregate metrics for the network:

- **Average shortest path length:** Calculated between all pairs of nodes
- **Diameter:** The longest of all shortest paths, representing the maximum length among the shortest paths between all pairs of nodes

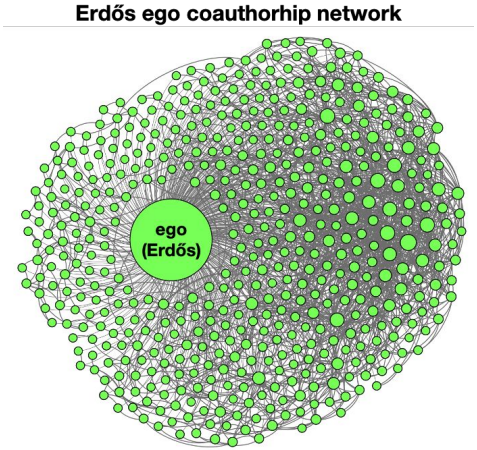
# Social Distance

**Example:** The co-authorship network from Paul Erdős, one of the greatest mathematicians in the world

- Considered the father of graph theory alongside Alfréd Rényi.
- Collaborated with over 500 co-authors: a hub in the co-authorship network!







# Social Distance

The **Erdős number** of an author is the length of the shortest path between them and Erdős in the co-authorship network

Many mathematicians take pride in having a small Erdős number

Some tools to calculate the Erdős number:

- https://mathscinet.ams.org/mathscinet/collaborationDistance.html
- https://www.csauthors.net/distance/

**Tip: If you want to look up your own, use your advisor's name and add 1**

# Examples: My academic social distance

Find the path between two authors:

Carlos Henrique Gomes | Paul Erdős

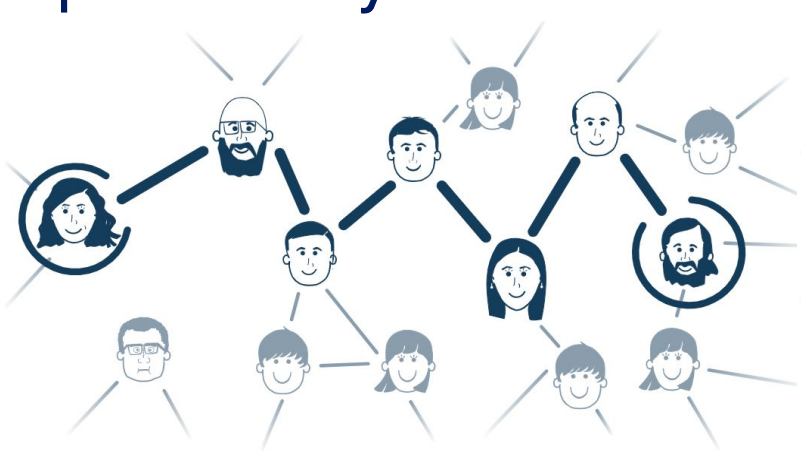
**Carlos Henrique Gomes Ferreira**
co-authored 11 papers with
Jussara M. Almeida
co-authored 2 papers with
Marc J. van Kreveld
co-authored 1 paper with
János Pach
co-authored 13 papers with
**Paul Erdős**
**distance = 4**

Find the path between two authors:

Carlos Henrique Gome | Tim Berners-Lee

**Carlos Henrique Gomes Ferreira**
co-authored 1 paper with
Charith Perera
co-authored 1 paper with
m. c. schraefel
co-authored 4 papers with
**Tim Berners-Lee**
**distance = 3**

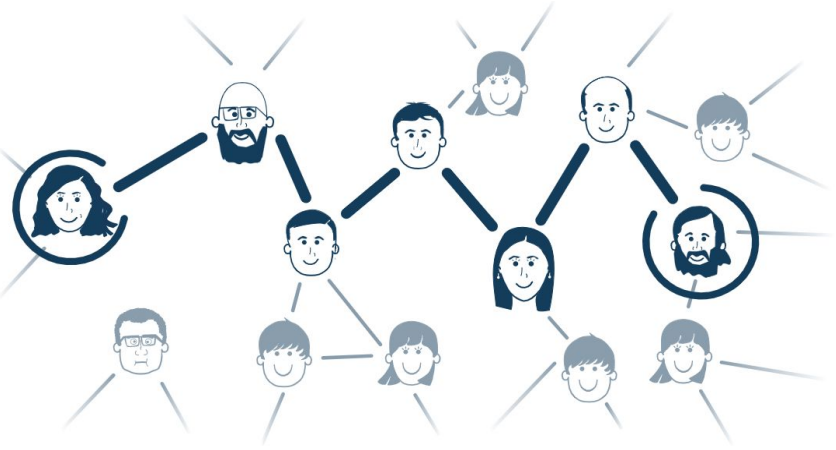
# Examples: My academic social distance



Find the path between two authors:

Carlos Henrique Gome | Sylvio Barbon Junior

**Carlos Henrique Gomes Ferreira**
co-authored 1 paper with
Marcos André Gonçalves
co-authored 30 papers with
Ricardo da Silva Torres
co-authored 1 paper with
**Sylvio Barbon Junior**
**distance = 3**



Find the path between two authors:

Carlos Henrique Gome | Eric Medvet

**Carlos Henrique Gomes Ferreira**
co-authored 13 papers with
Jussara M. Almeida
co-authored 8 papers with
Gisele L. Pappa
co-authored 1 paper with
**Eric Medvet**
**distance = 3**

# Six Degrees of Separation

**Six degrees of separation:** The idea that any two people are, on average, six steps apart from each other in the social network

The first idea emerged in the story "Chains" by Hungarian writer Frigyes Karinthy in 1929

- Psychologist Stanley Milgram provided the first evidence in 1967 through a famous experiment
- The goal was to measure the social distance between any two people in the U.S.

# Milgram's Experiment

**Instructions:** Send to a personal acquaintance who is most likely to know the target

- 160 letters were sent to people in Omaha (NE), and Wichita (KS)
- 2 initial targets: The wife of a student in Sharon and a stockbroker in Boston
- 42 letters were returned (only 26%)

**Result:** Average of 6.5 steps (range: 3-12 steps)

**Take away message:** Much lower than what most people expected!

# More "Small World" Experiments

**In 2003, conducted by Yahoo Research using email:**

- 18 targets in 13 countries
- 384 completed paths out of more than 24,000 initiated
- **Average path found = 4**

**In 2011, researchers from Facebook and the University of Milan:**

- 721 million active Facebook users
- 69 billion friendships
- **Average path found = 4.74 steps**

**Take away:** The "Small World" effect still surprises in networks

# Small World: Very common in various networks

A very common property in various networks:

| Network | Nodes ($N$) | Links ($L$) | Average path length ($\langle \ell \rangle$) | Clustering coefficient ($C$) |
|---|---|---|---|---|
| Facebook Northwestern Univ. | 10,567 | 488,337 | 2.7 | 0.24 |
| IMDB movies and stars | 563,443 | 921,160 | 12.1 | 0 |
| IMDB co-stars | 252,999 | 1,015,187 | 6.8 | 0.67 |
| Twitter US politics | 18,470 | 48,365 | 5.6 | 0.03 |
| Enron Email | 87,273 | 321,918 | 3.6 | 0.12 |
| Wikipedia math | 15,220 | 194,103 | 3.9 | 0.31 |
| Internet routers | 190,914 | 607,610 | 7.0 | 0.16 |
| US air transportation | 546 | 2,781 | 3.2 | 0.49 |
| World air transportation | 3,179 | 18,617 | 4.0 | 0.49 |
| Yeast protein interactions | 1,870 | 2,277 | 6.8 | 0.07 |
| C. elegans brain | 297 | 2,345 | 4.0 | 0.29 |
| Everglades ecological food web | 69 | 916 | 2.2 | 0.55 |

# Robustness

**Definition:** A network is robust if the removal of some of its nodes does not affect its function

- **Example**: The Internet, which would be unable to transmit signals (e.g., emails) between routers in different locations if it were not connected

**How to quantify it?:** Remove nodes and/or connections to observe the impact on the network's structure

- Key Point: Connectivity

# Robustness

Evaluated by its ability to remain connected and functional after failures or attacks

- Helps identify vulnerabilities and critical points that may compromise the network

**Connected Component:** A group of nodes directly connected, allowing communication within the group but not with external nodes

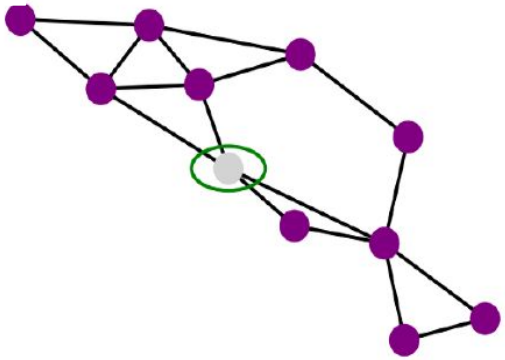
- A network can have multiple components. The **largest** is called the **giant component**

# Robustness

**Robustness Test:** Evaluates the impact on **connectivity** of the largest component as nodes are removed

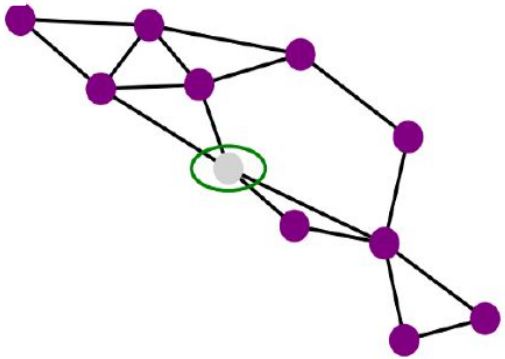
1. Plot the **relative size** $S$ of **the largest connected component** as a function of the fraction of nodes removed
    - Assume the network starts fully connected: there is only one component (S = 1)
2. As more nodes (and their links) are removed, the network progressively splits into components, and $S$ decreases

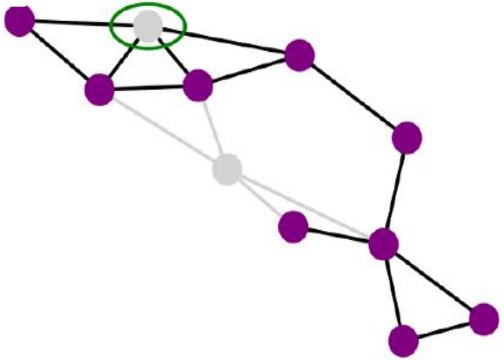
# Robustness: Example



**Largest Connected Component (C.C.) and a Node is Selected for Removal**

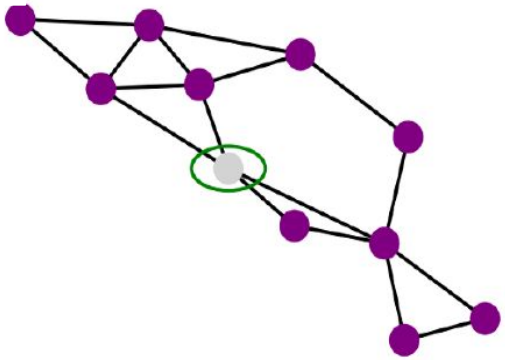
# Robustness: Example

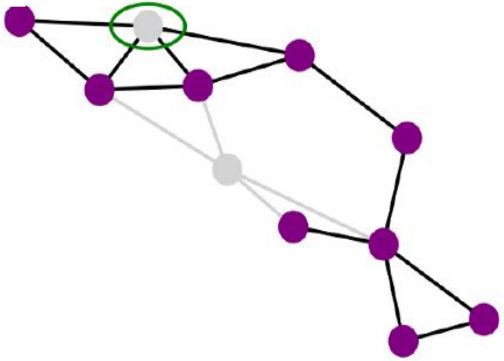


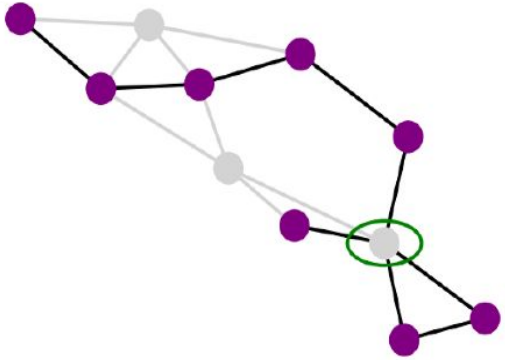


**It remains connected.**
**Therefore, ( S = 1)**

# Robustness: Example

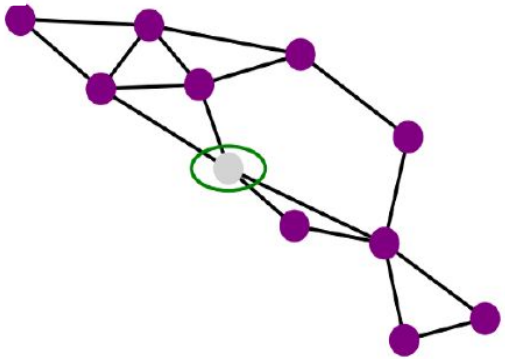


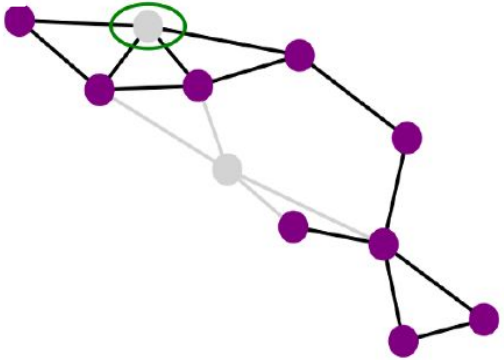


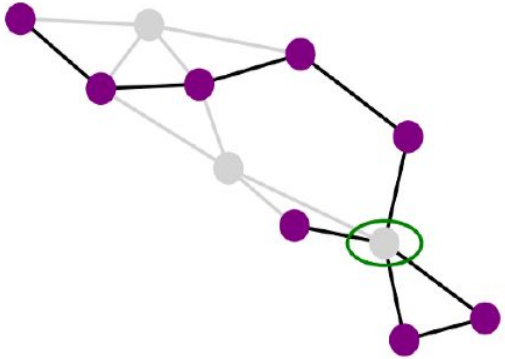


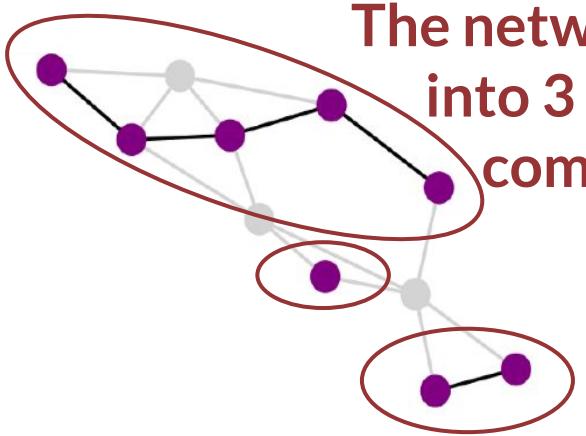
**It remains connected**

# Robustness: Example









**The network is broken into 3 connected components**

# Robustness

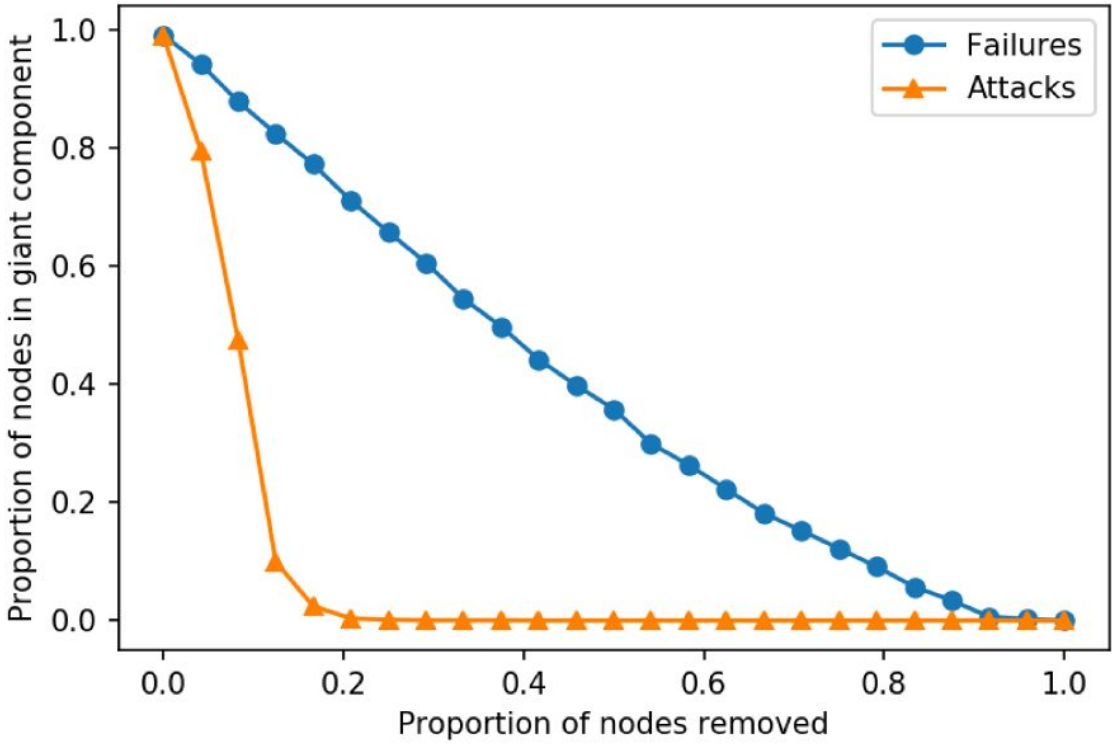
## Strategies for robustness analysis:

- **Random Failures:** Nodes are removed randomly with equal probability (uniformly)
  - Remove a fraction $f$ of nodes chosen at random

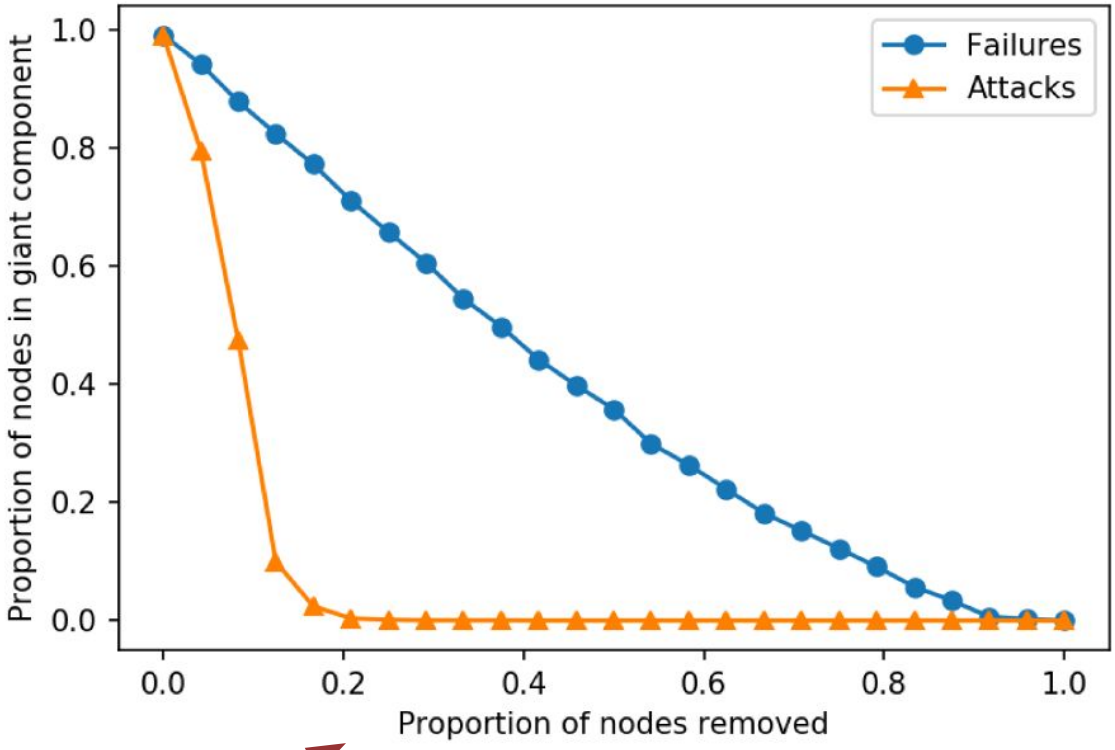
- **Attacks:** Hubs are deliberate targets — the higher the degree, the greater the likelihood of node removal
  - Remove the fraction $f$ of nodes with the highest degree, starting from the highest degree downwards

Other strategies could be employed, for example, based on other centrality measures or contextual information about the nodes

# Robustness

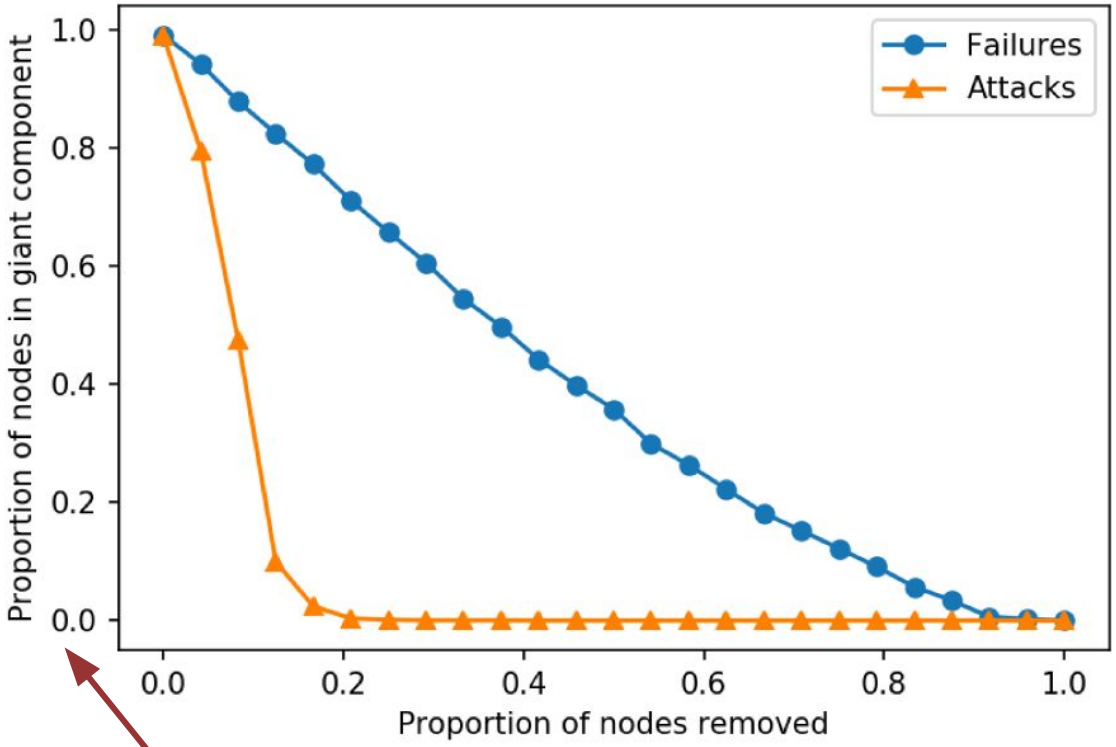

Failures
Attacks
Proportion of nodes in giant component
1.0
0.8
0.6
0.4
0.2
0.0
0.0
0.2
0.4
0.6
0.8
1.0
Proportion of nodes removed

# Robustness



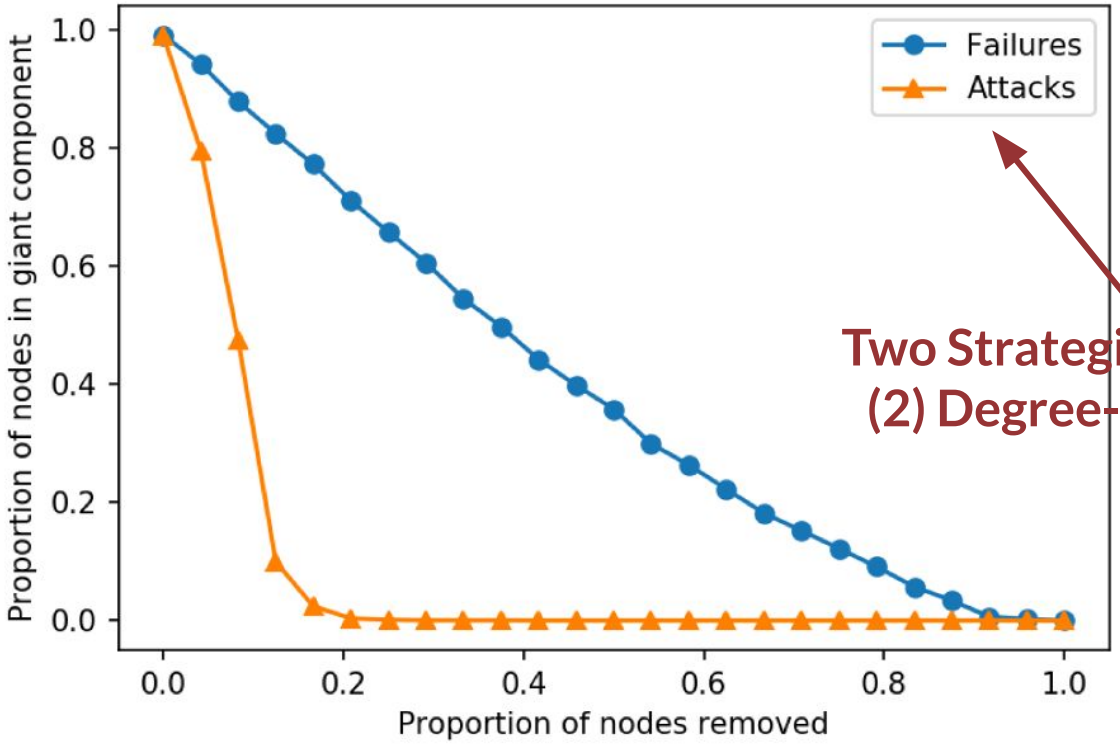

**Proportion of nodes removed from the network**

# Robustness



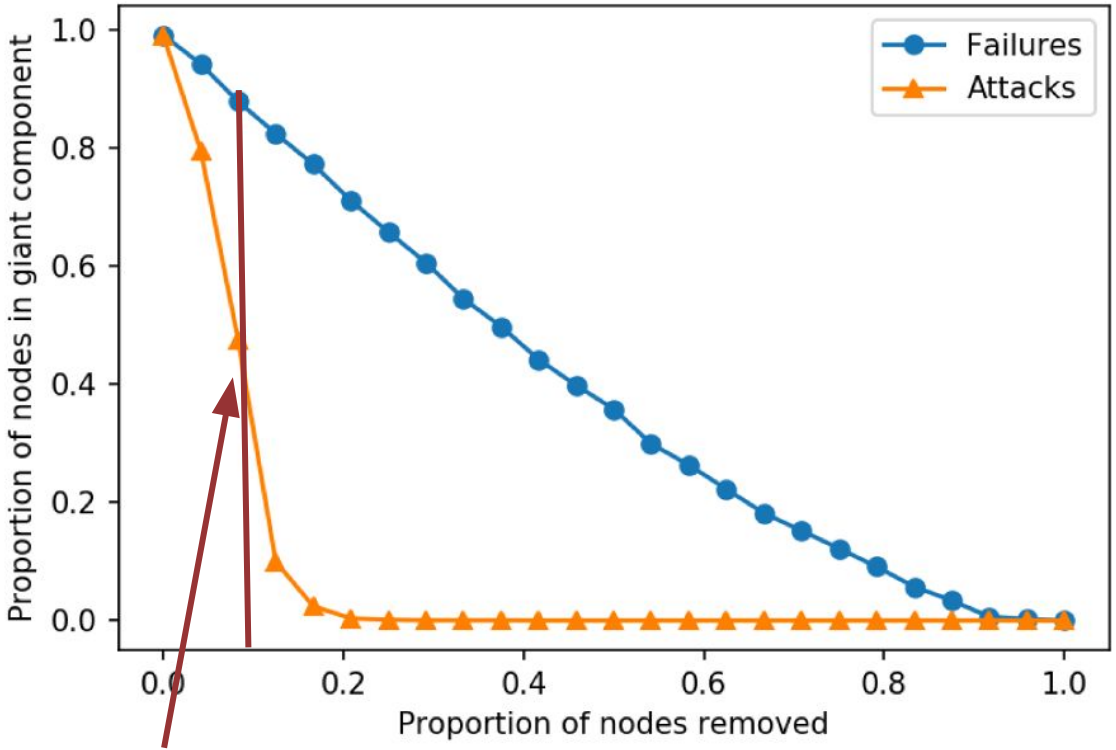

**Proportion of nodes in the Largest Connected Component (Giant Component)**

# Robustness



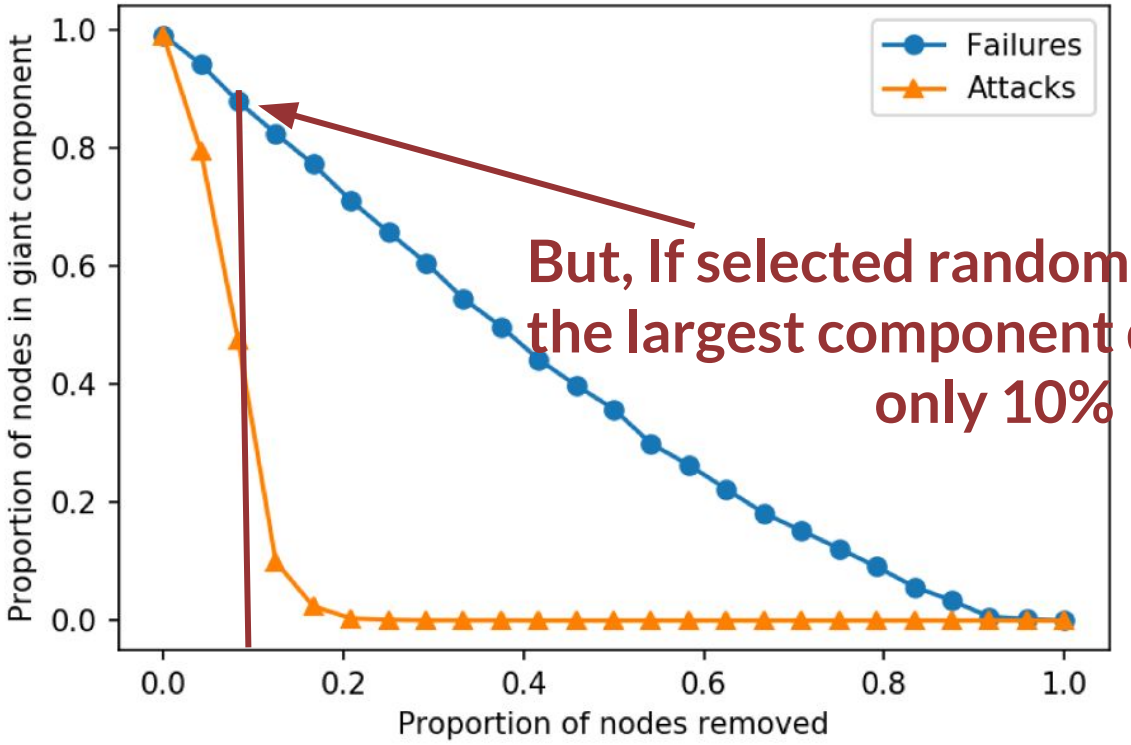

**Two Strategies: (1) Random. (2) Degree-Based Attacks**

# Robustness



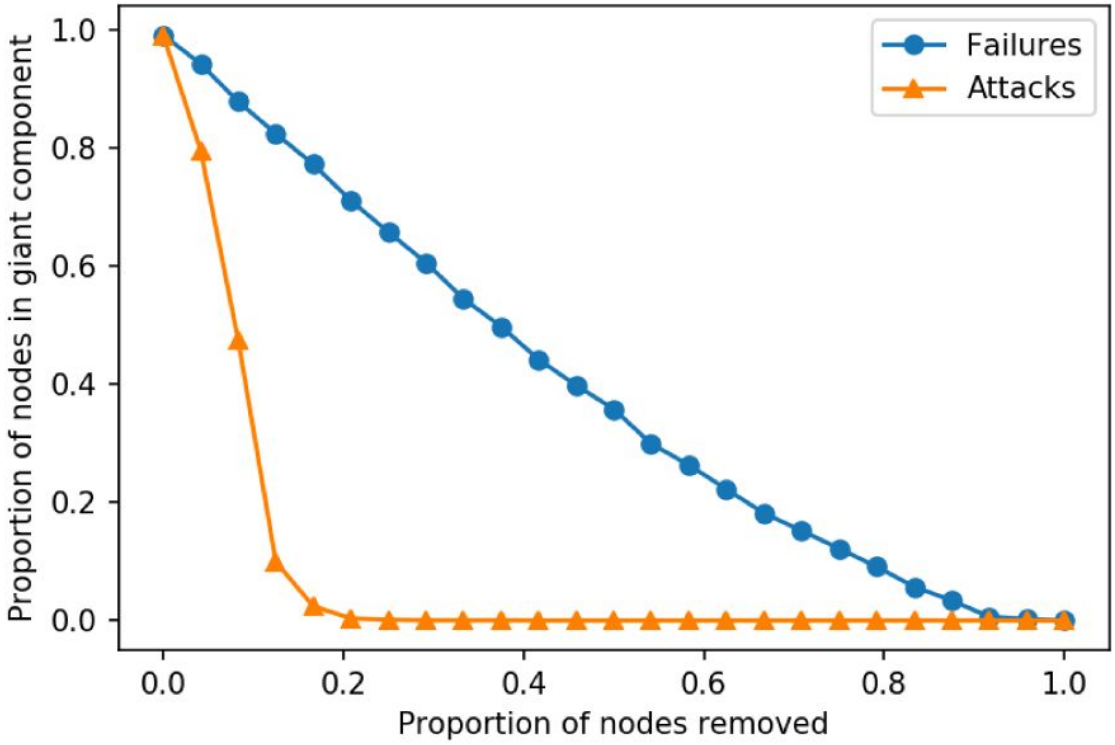

**With 10% of the nodes attacked, the largest component of the network is reduced by 50%**

# Robustness




**But, If selected randomly, the size of the largest component decreases by only 10%**

# Robustness




**Real networks tend to be robust against random failures but fragile against targeted attacks!**

# Roadmap

Graph Modeling Fundamentals

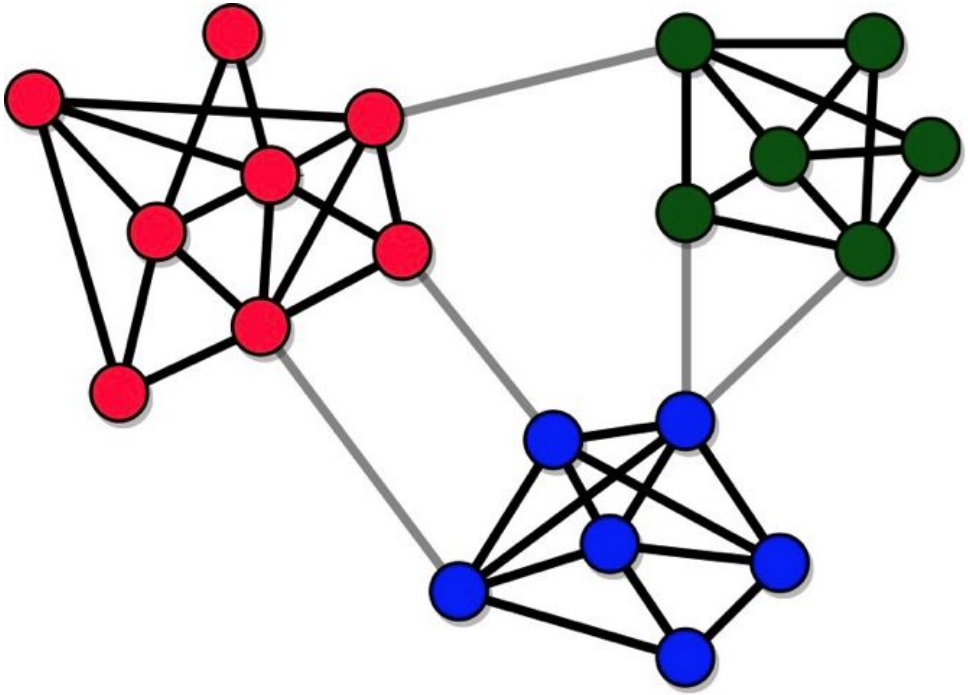
Structural and Centrality Analysis

**Community Detection**

Final Considerations

# Community Detection

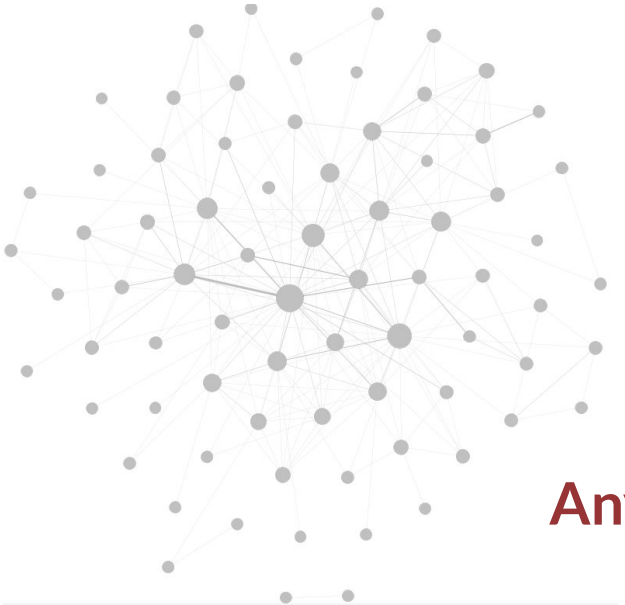
In a network, nodes tend to cluster into **communities**



# Community Detection

Communities can be visually evident or require algorithms

- In most cases, algorithms are essential for identification and visualization as we manage large networks

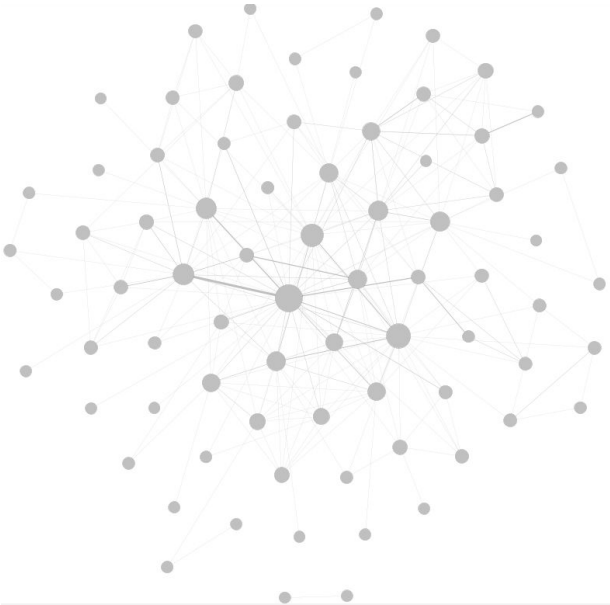


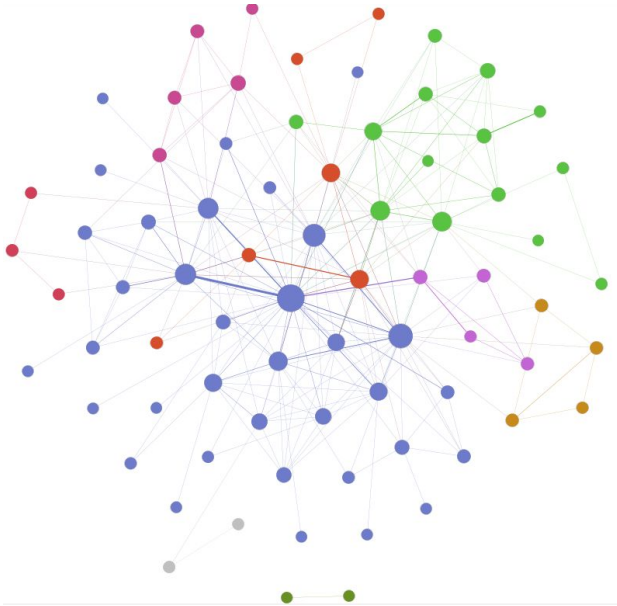
**Any community?**

# Community Detection

Communities can be visually evident or require algorithms

- In most cases, algorithms are essential for identification and visualization as we manage large networks





**And now?**

# Community Detection

It is an interdisciplinary area, also known as community discovery or clustering

Grouping nodes into communities is an **unsupervised task**, lacking a unique definition of communities

**Natural intuition:** Communities have more internal connections than external ones

- A criterion that can be formalized in various ways

# Community Detection Algorithms

There are various algorithms in the literature for community detection designed for different characteristics

- Static Networks
- Dynamic Networks
- Hard partitions (each node belongs to only one community)
- Overlapping partitions (a node can belong to multiple communities)
- Bipartite Networks
- …

# Community Detection Algorithms

There are various algorithms in the literature for community detection designed for different characteristics

- **Static Networks**
- Dynamic Networks
- Hard partitions (each node belongs to only one community)
- Overlapping partitions (a node can belong to multiple communities)
- Bipartite Networks
- …

# But… How can we evaluate communities?

It is not expected to find communities in a **random network** because the nodes are connected to each other randomly

- Therefore, there are no groups of nodes that prefer to connect with each other rather than with nodes outside the network, even with high density

This is a well-established principle of community detection: **Random networks do not have communities!**

**Modularity** is a function that captures this idea and measures the quality of a community partition in a network

# Modularity Function

**Principle:** Evaluate communities with respect to a random baseline

- A random version of the original network is generated, preserving its degree sequence

Use the difference between  the number of internal links in the community and the expected value of this number in a random network

If the number of links within communities exceeds what is expected in the random network, modularity will be high

# Modularity Function

Modularity *Q* ranges from -0.5 to 1, reflecting the community structure in the network

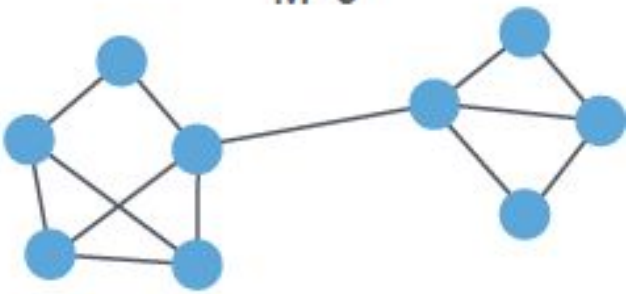
- *Q<0* suggests the absence of communities
- *Q=0* indicates that the network is a single community
- *Q* between 0.3 and 0.7 are expected to occur in practice and indicate well-defined communities

```
# returns the modularity of the input partition
modularity = nx.community.quality.modularity(G,partition)
```

# Modularity Function: Example

Negative Modularity
M=0.12

Single Community
M=0



Suboptimal Partition
M=0.22

Optimal Partition
M=0.41

# Louvain Algorithm

An efficient heuristic for optimizing modularity in networks aiming to identify community divisions

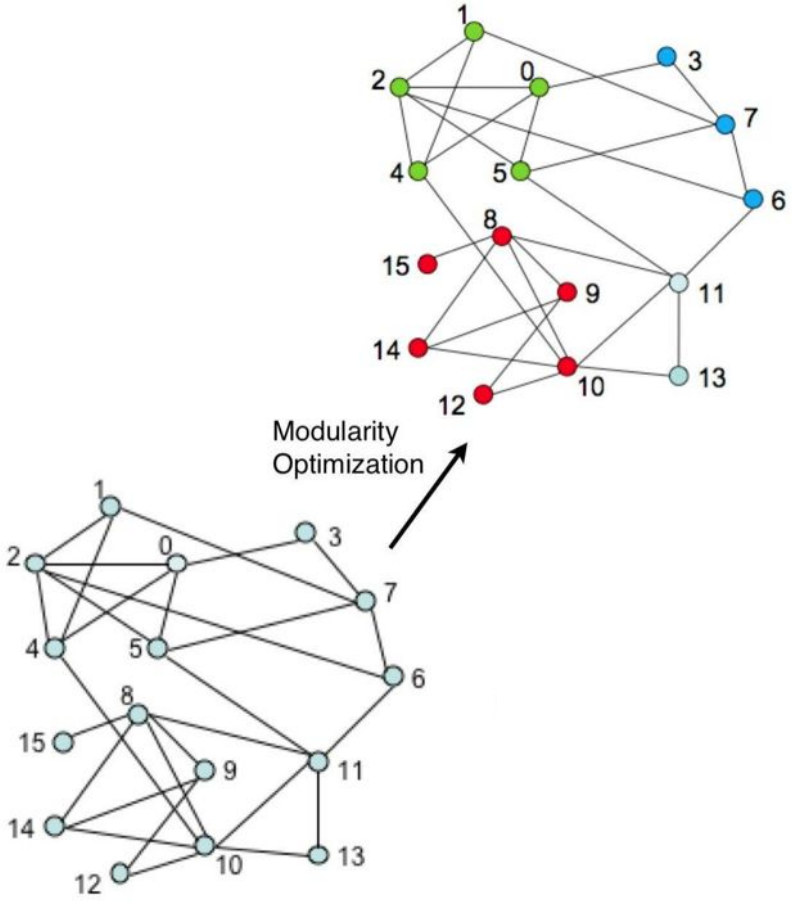
Although highly effective and efficient for community detection in large networks, it does not guarantee finding the optimal solution

- Performs iterative local improvements on the community structure

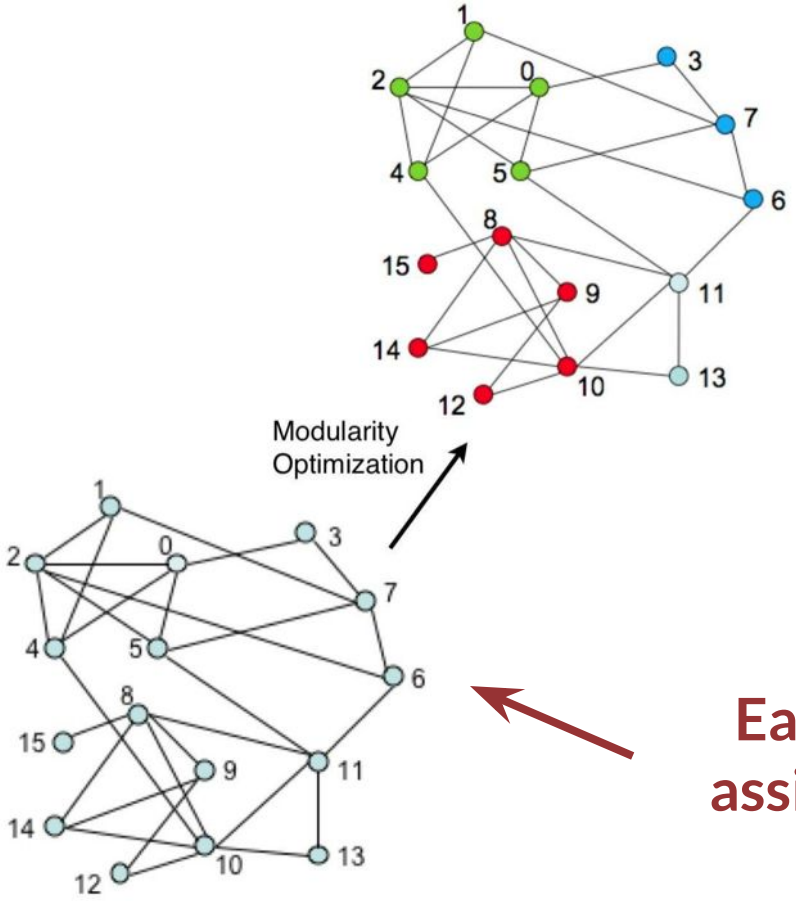
Finds good quality solutions in a feasible computational time

- But without the guarantee that the global maximum modularity solution will be achieved for all networks

# Louvain Algorithm

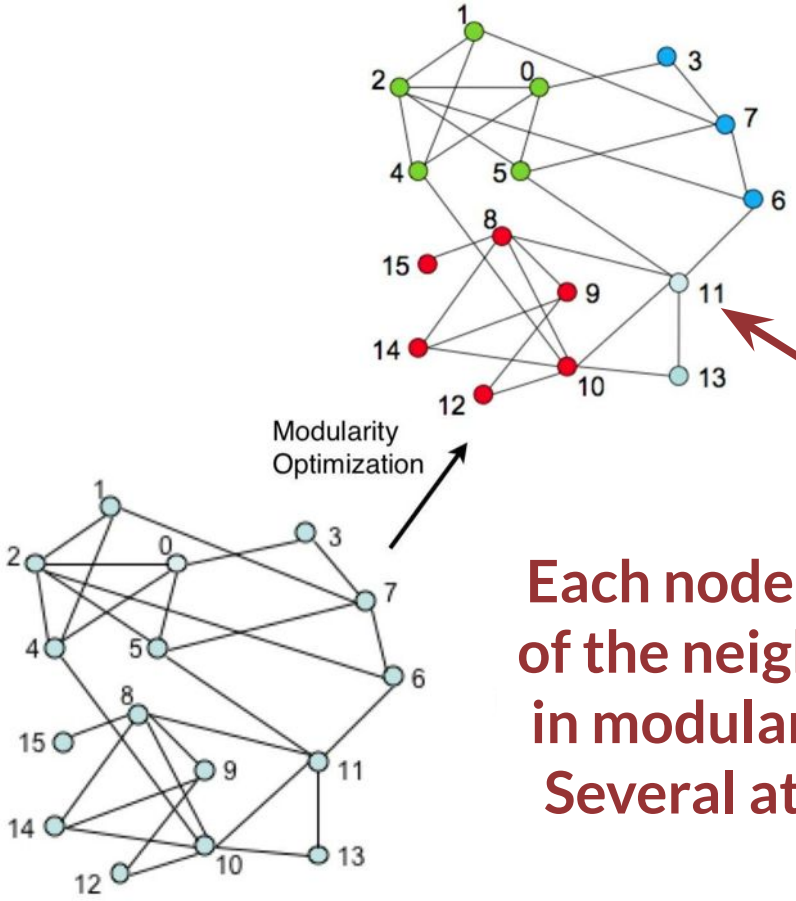


# Louvain Algorithm



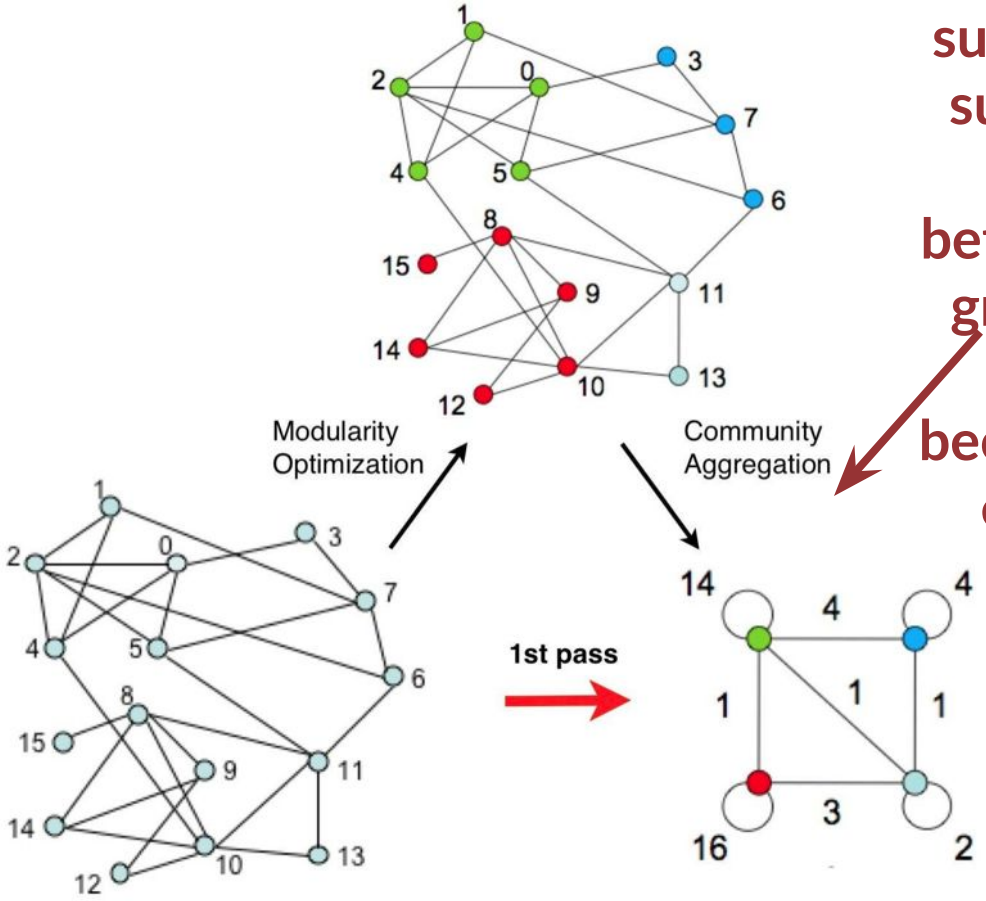

**Each node is initially assigned to a different community**

# Louvain Algorithm




**Each node is reassigned to the community of the neighbor that maximizes the change in modularity ΔQ of the current partition. Several attempts are made until the best solution is reached!**

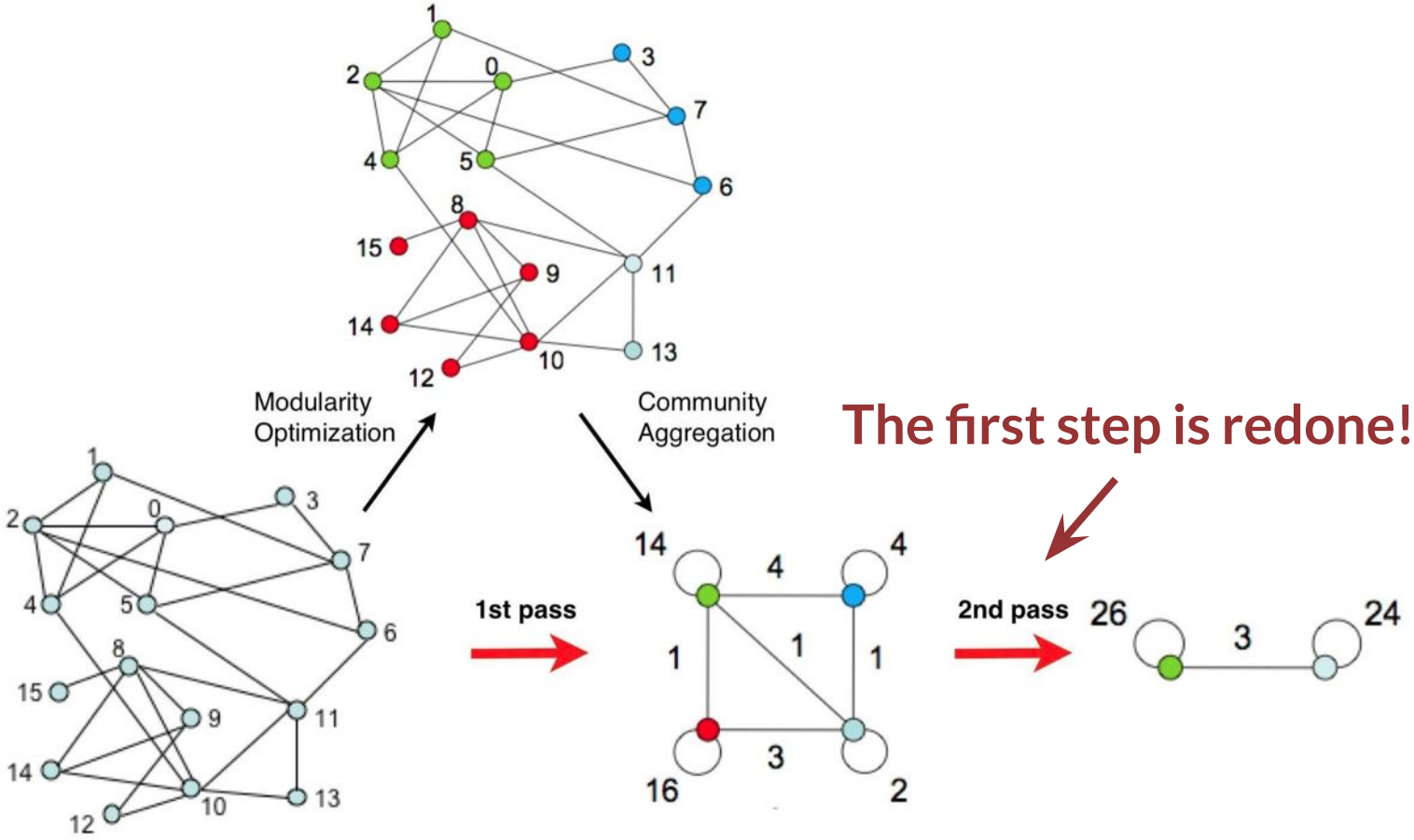
# Louvain Algorithm



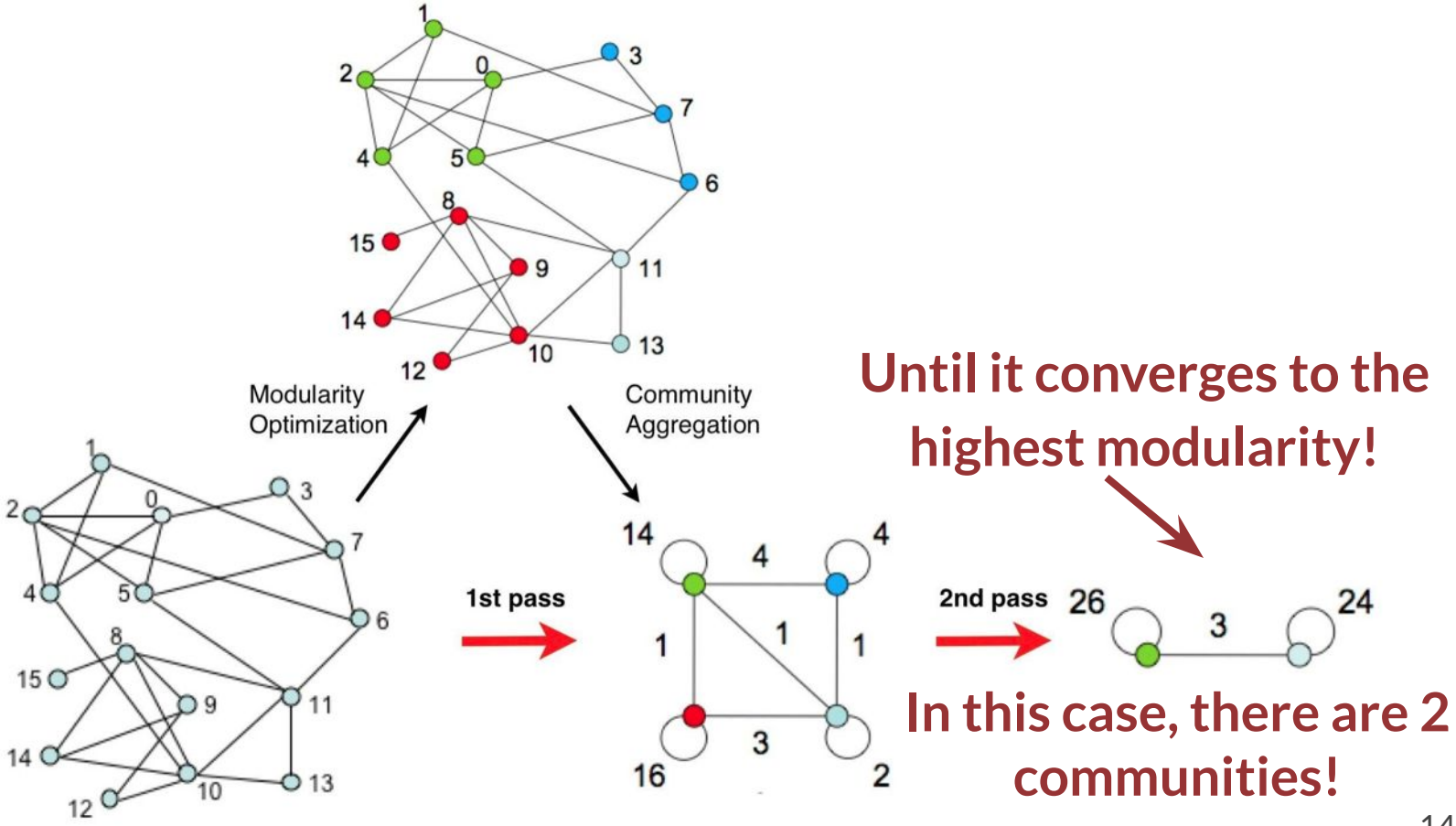

**Communities become supernodes, links between supernodes are weighted by the number of links between the corresponding groups, and internal links within a community become loops with a weight equal to the number of internal links.**

**Note: Weights are not shown**

# Louvain Algorithm



# Louvain Algorithm



# Louvain Algorithm

Example of how to use the Louvain Algorithm in NetworkX:

```
import networkx as nx

# returns the partition with largest modularity
partition = nx.community.louvain_communities(G, weight='weight')
modularity = nx.community.modularity(G, partition, weight='weight')
```

# Louvain Algorithm: Limitations

The maximum modularity tends to be higher in larger networks, preventing quality comparisons of partitions between networks

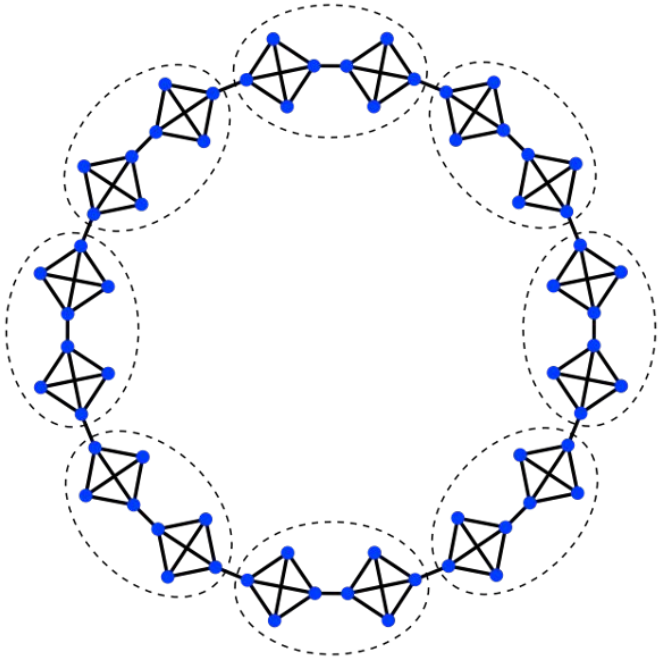
The maximum modularity does not necessarily correspond to the best partition

- May fail to detect smaller communities due to the resolution limit

A new parameter was subsequently added to the modularity function:

- *Resolution*. Typically defined as 1 (Maximum resolution in the network)

# Louvain Algorithm: Limitations



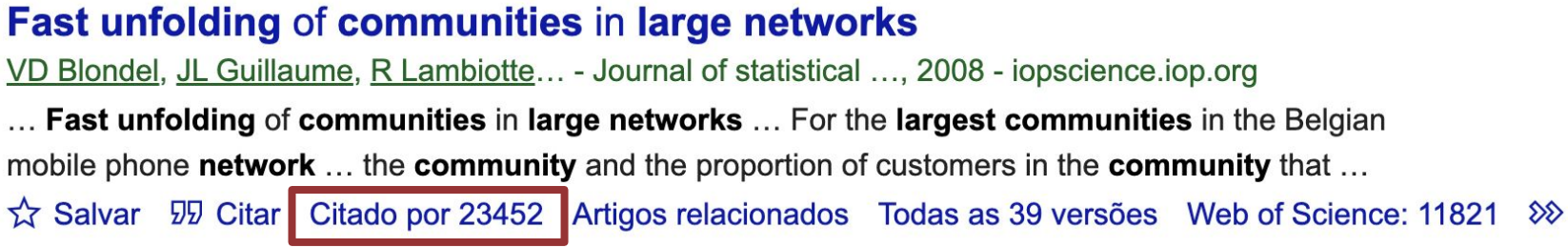
**The natural partition is the one where the communities are the cliques but modularity is larger for the partition where pairs of cliques are merged**

**Example of the Resolution Problem:**

Modularity = 0.79, even though the network is not well partitioned!

# Leiden Algorithm

Louvain has dominated for over a decade as one of the most widely used algorithms



# Leiden Algorithm

Louvain has dominated for over a decade as one of the most widely used algorithms

**Fast unfolding** of **communities** in **large networks**

VD Blondel, JL Guillaume, R Lambiotte… - Journal of statistical …, 2008 - iopscience.iop.org

… **Fast unfolding** of **communities** in **large networks** … For the **largest communities** in the Belgian mobile phone **network** … the **community** and the proportion of customers in the **community** that …

☆ Salvar 99 Citar Citado por 23452 Artigos relacionados Todas as 39 versões Web of Science: 11821 »

More recently (2019), a new solution exploring this and other weaknesses was proposed

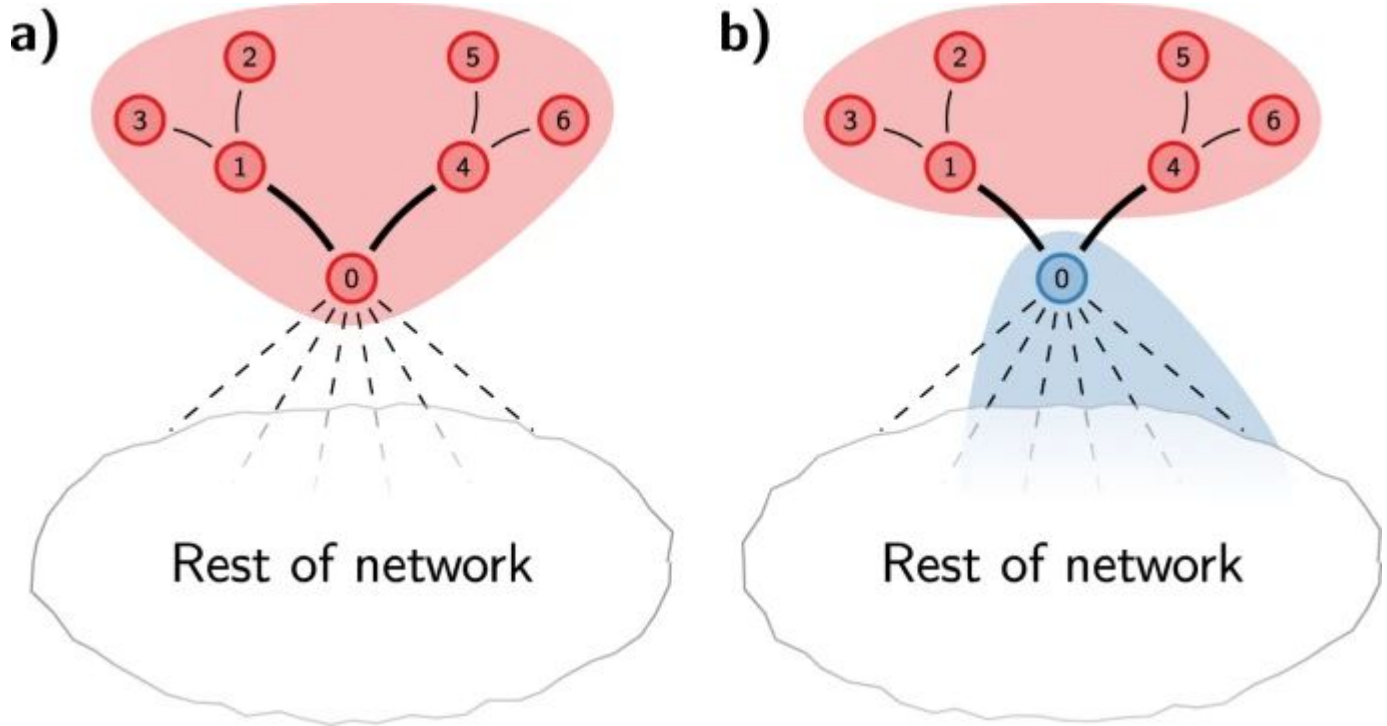
[HTML] From **Louvain** to **Leiden**: guaranteeing well-connected communities

VA Traag, L Waltman, NJ Van Eck - Scientific reports, 2019 - nature.com

… , the **Leiden** algorithm runs faster than the **Louvain** algorithm. We demonstrate the performance of the **Leiden** … We find that the **Leiden** algorithm is faster than the **Louvain** algorithm and …

☆ Salvar 99 Citar Citado por 3780 Artigos relacionados Todas as 14 versões Web of Science: 1867 »

# Leiden Algorithm




The Louvain algorithm may in a very specific case produce disconnected communities!

https://www.nature.com/articles/s41598-019-41695-z

# Leiden Algorithm

The **Leiden algorithm** introduces an additional refinement step to address the limitations of the Louvain algorithm

This extra step allows the algorithm to improve the community partition even after the aggregation phase

It can split communities that may have become disconnected

- Ensures greater accuracy in community detection and an improvement in the quality of the network's modularity
- It is **significantly faster** than Louvain

# Summarizing

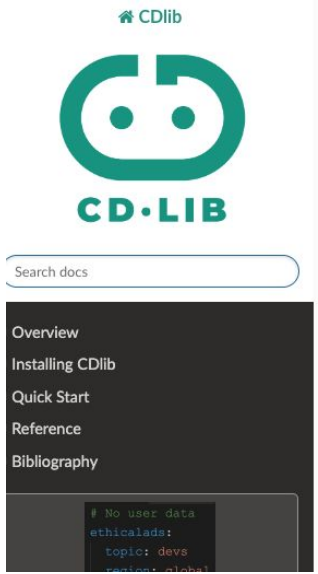
Communities are one of the areas of greatest interest in the study of networks

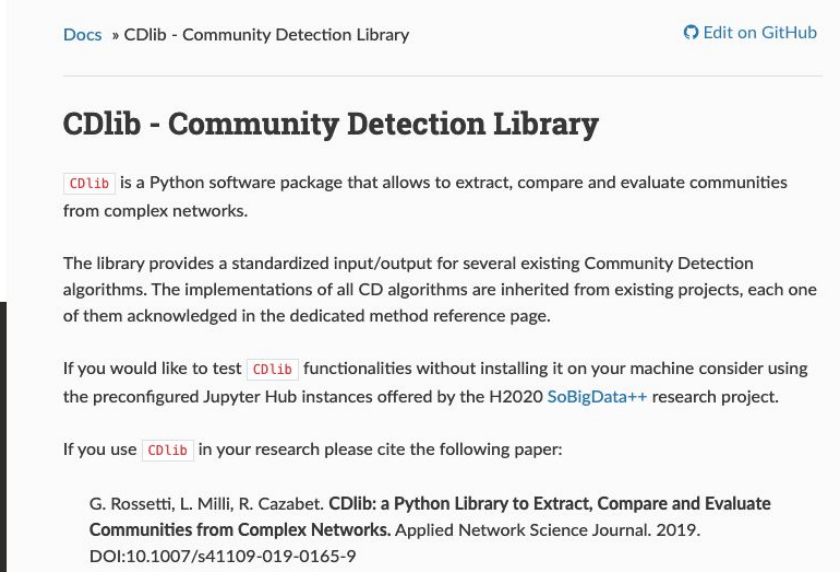
- Allow for the subsequent characterization of representative groups of nodes related to the problem under study

Several algorithms/metrics:

**CD-Lib:** A library with dozens of community detection algorithms and different evaluation measures/functions

https://cdlib.readthedocs.io/



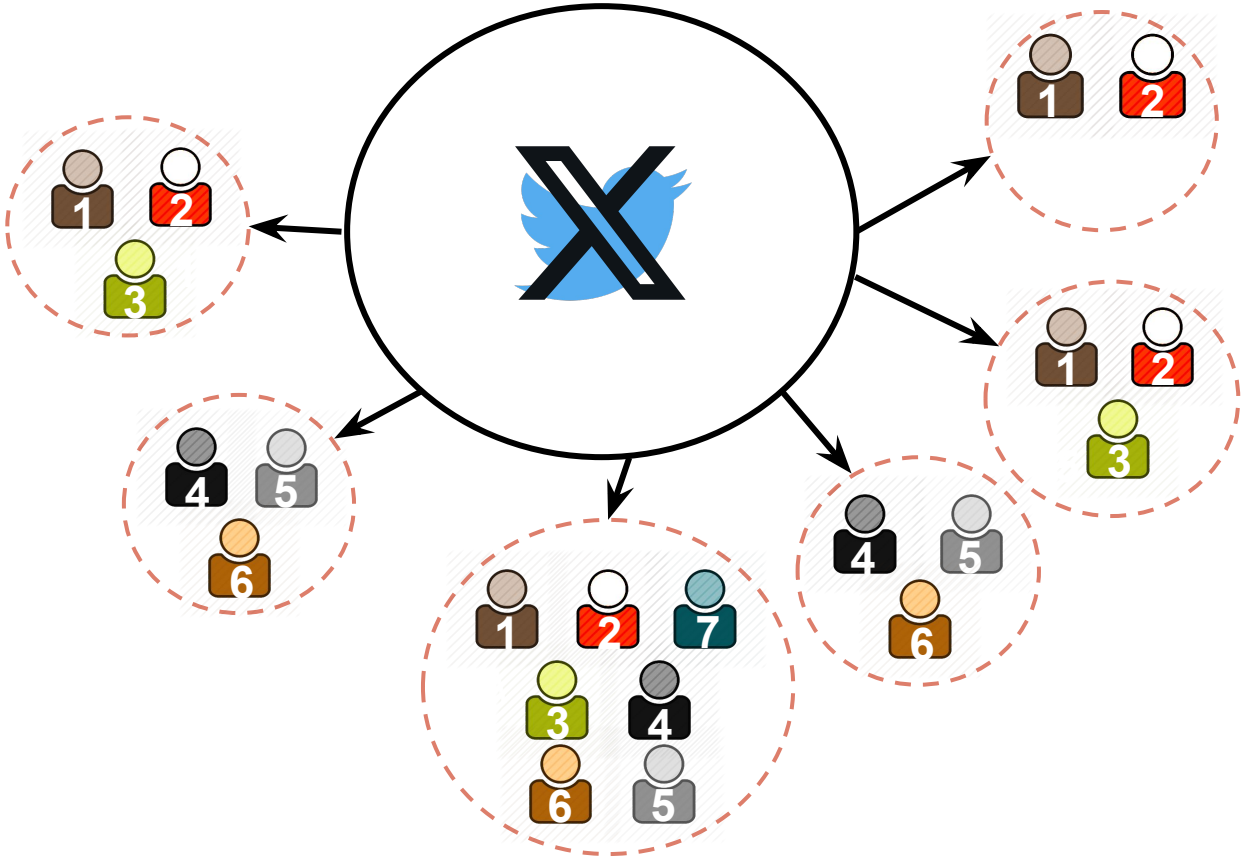
# Community Detection: Challenges

Community detection algorithms rely purely on the structure of the network

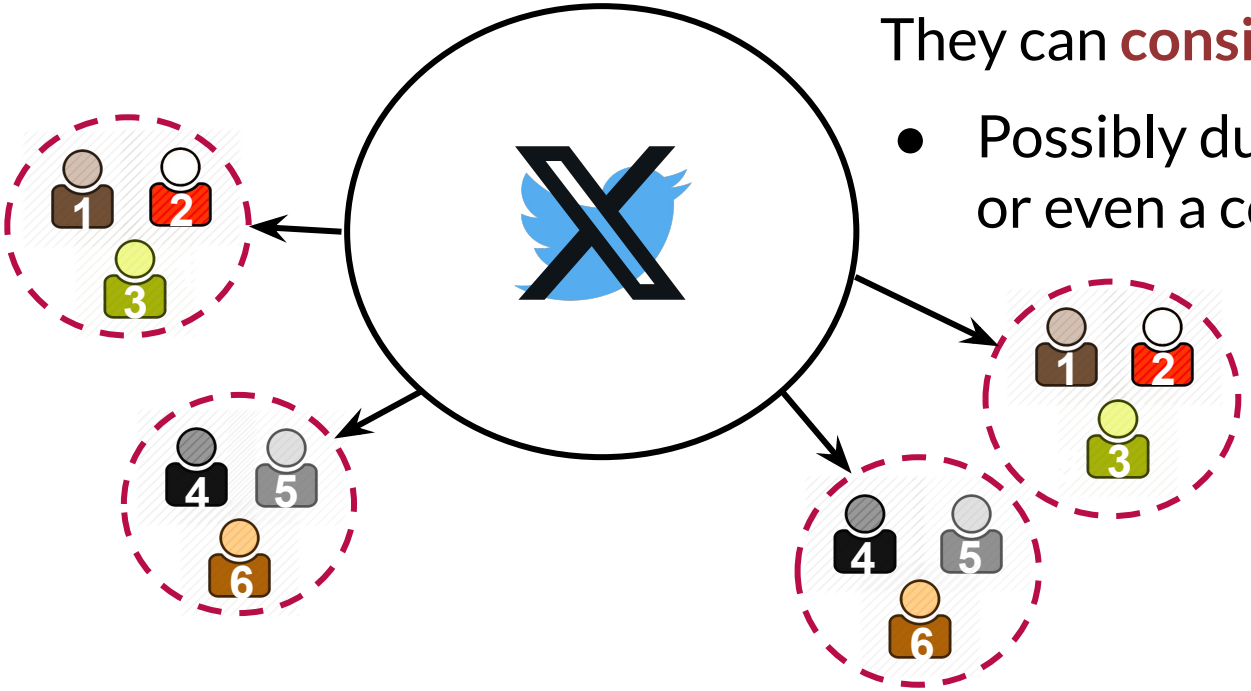
Typically, this structure is noisy and may obscure the phenomenon of interest

There are techniques for extracting the backbone that allow for more accurate modeling and analysis of the network

# Community Detection: Challenges



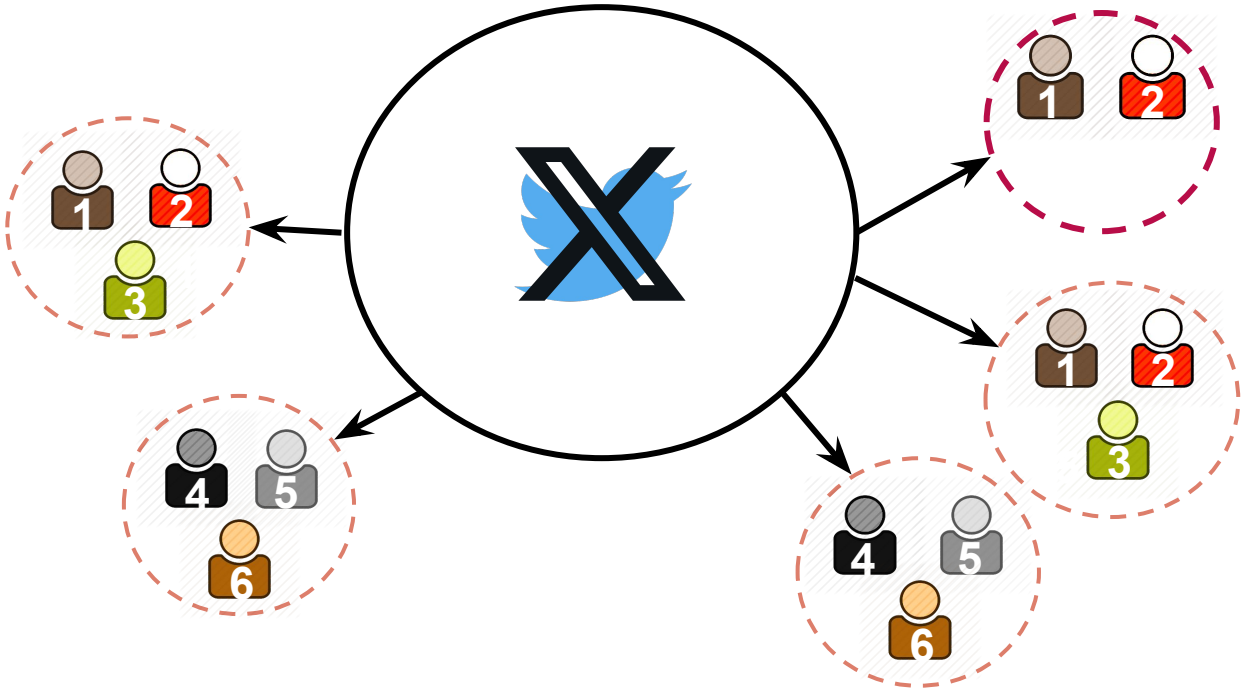
# Community Detection: Challenges




They can **consistently repeat**

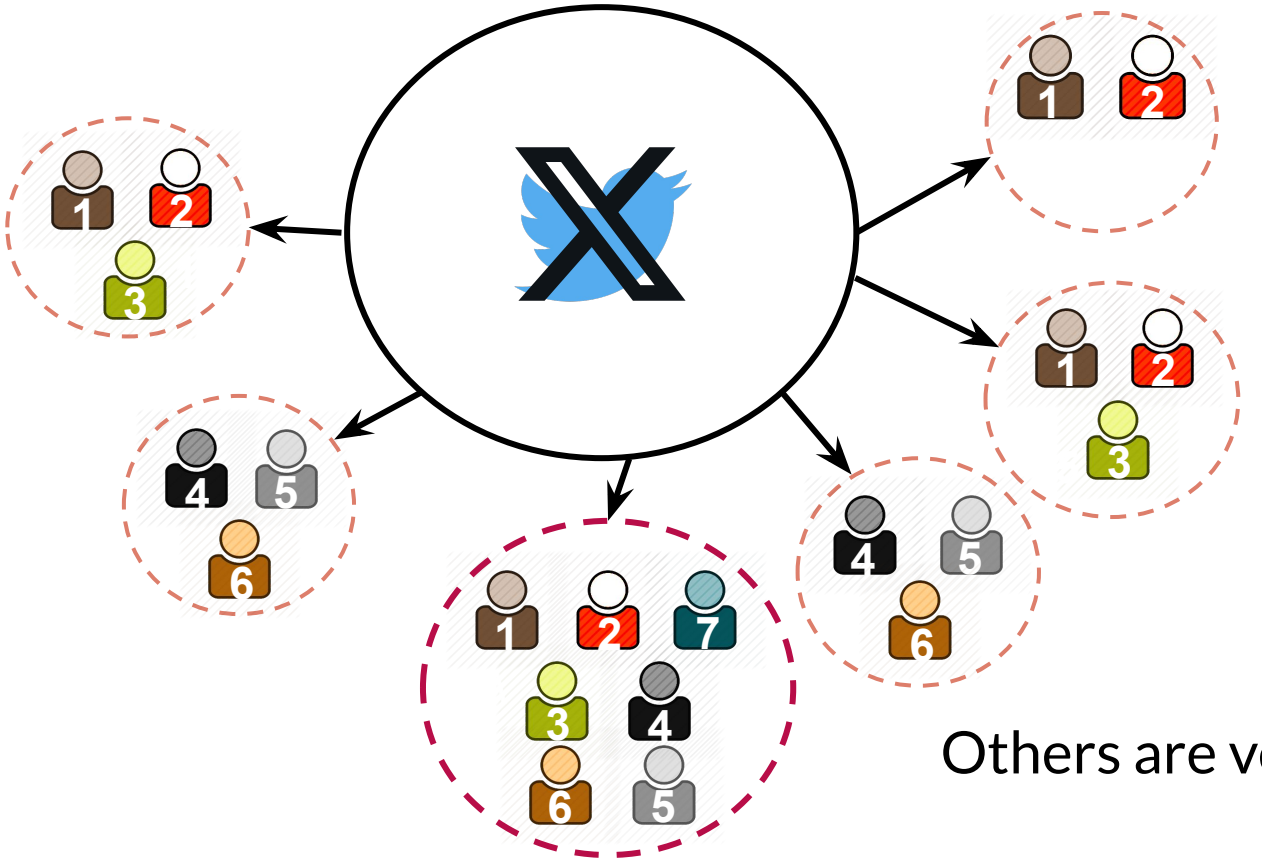
- Possibly due to interest in the topic or even a coordinated action

# Community Detection: Challenges



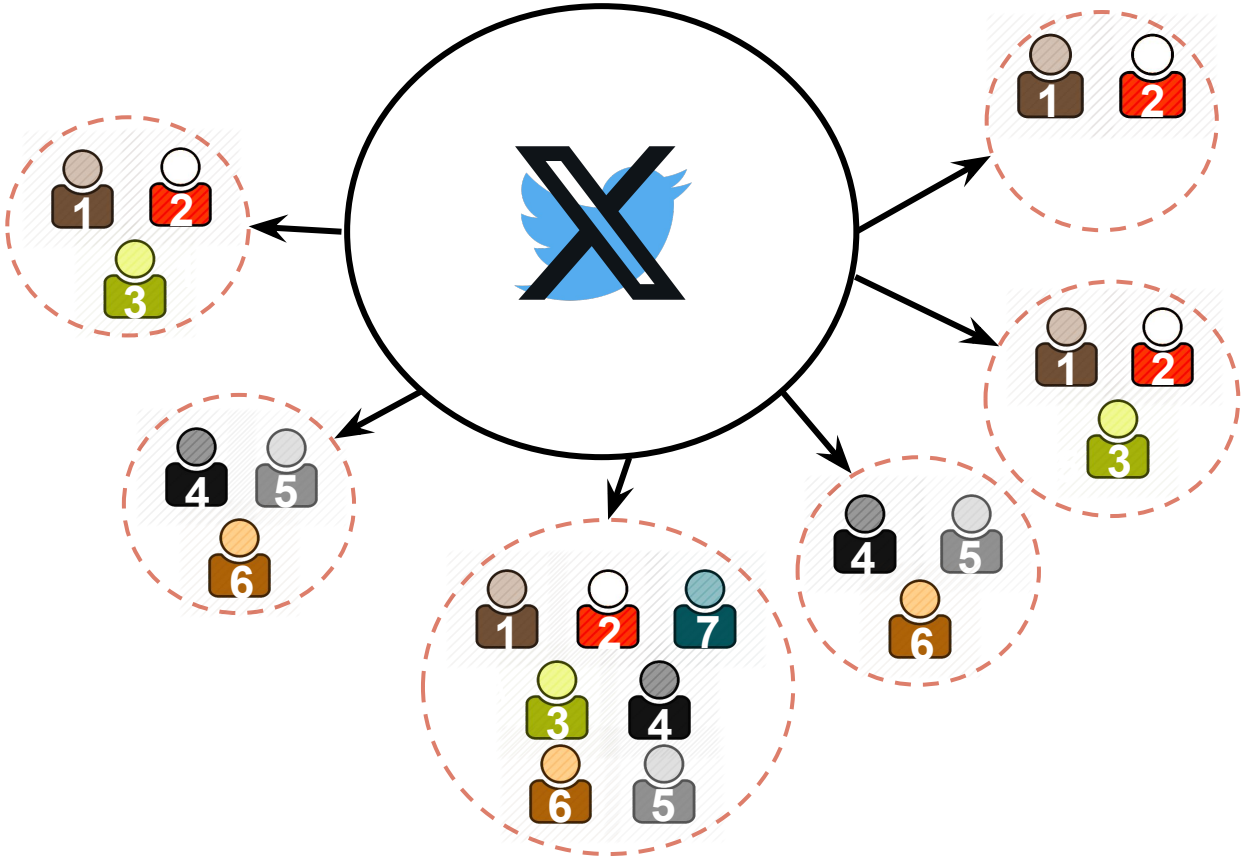

Others may arise from partial **copies** or **combinations** of the previous ones

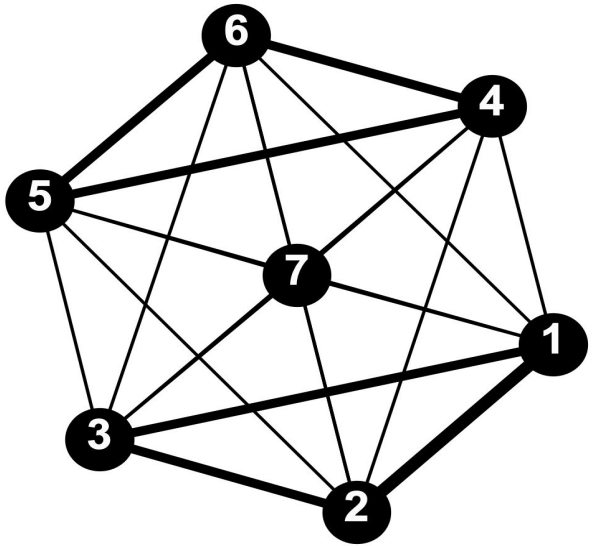
# Community Detection: Challenges



Others are very **random** or **sporadic**

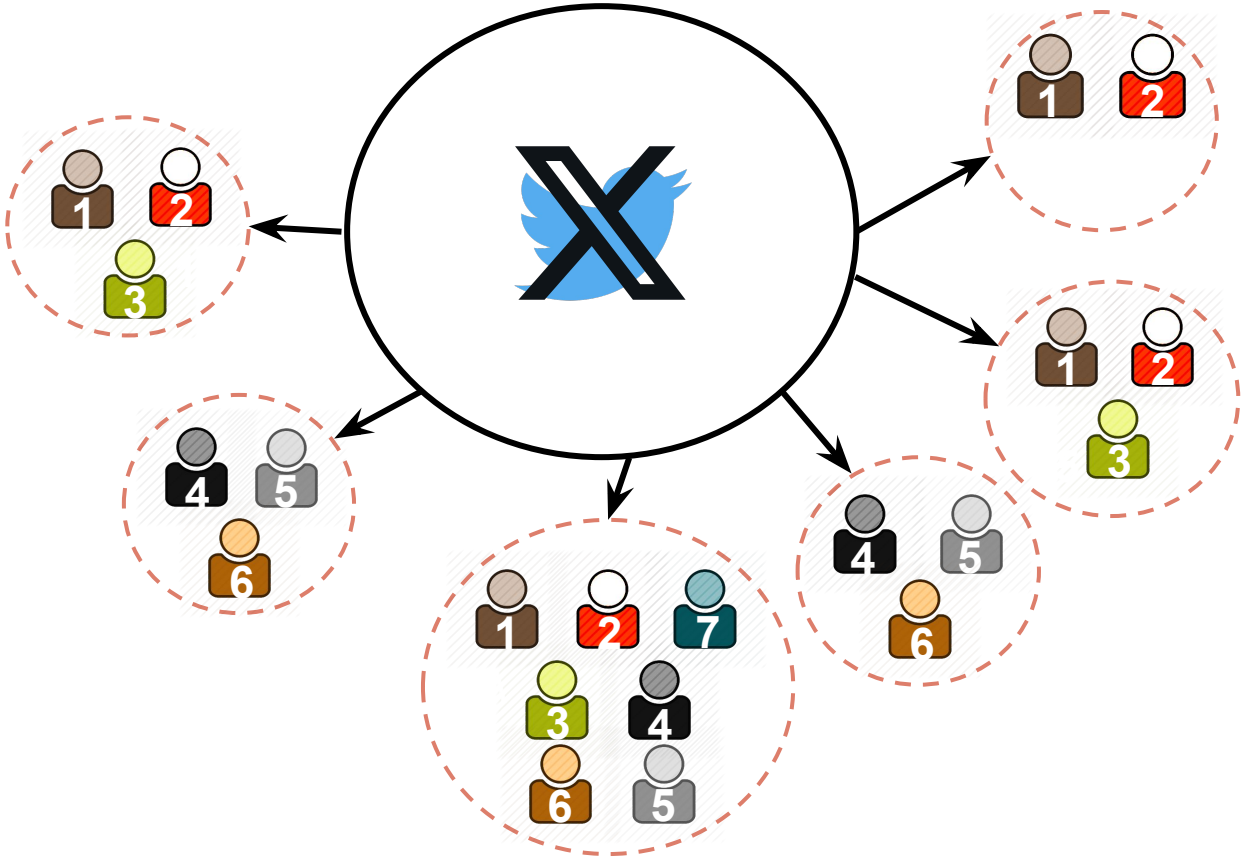
# Community Detection: Challenges

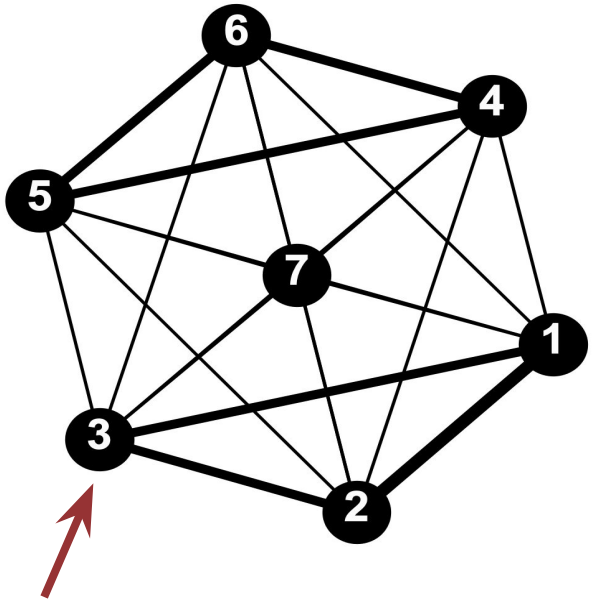




Network Modeled

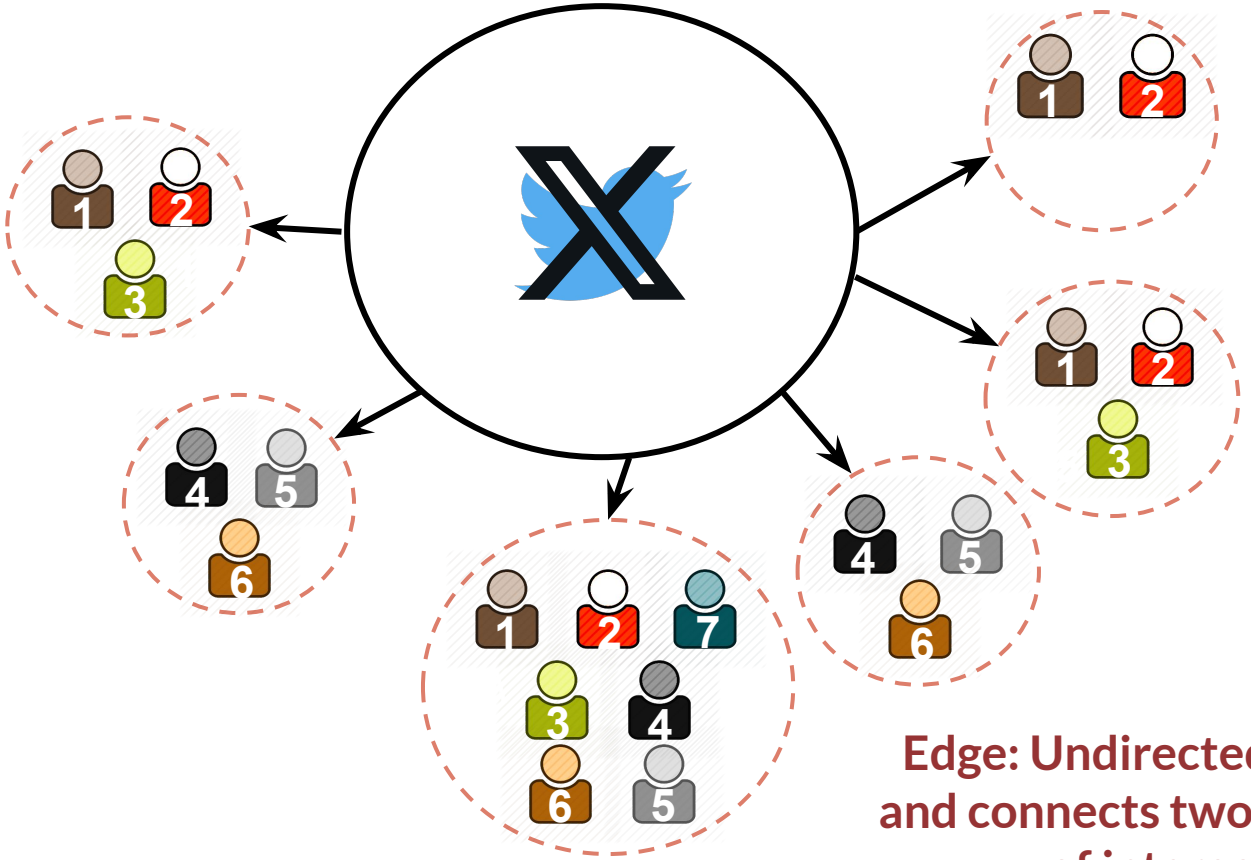
# Community Detection: Challenges

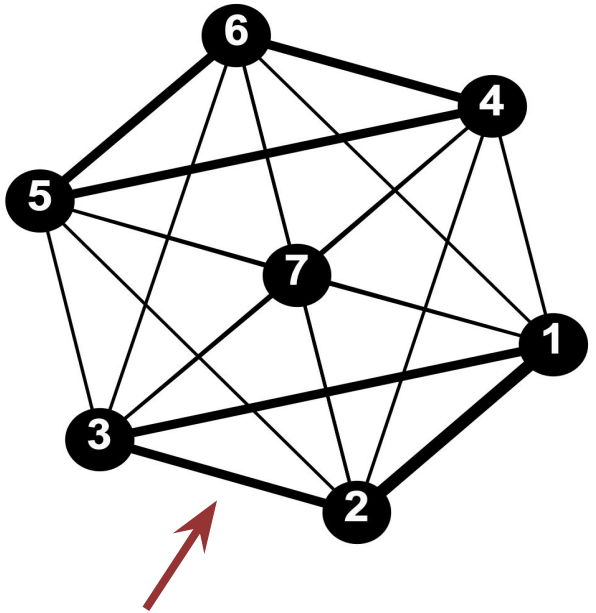




Node: User

# Community Detection: Challenges





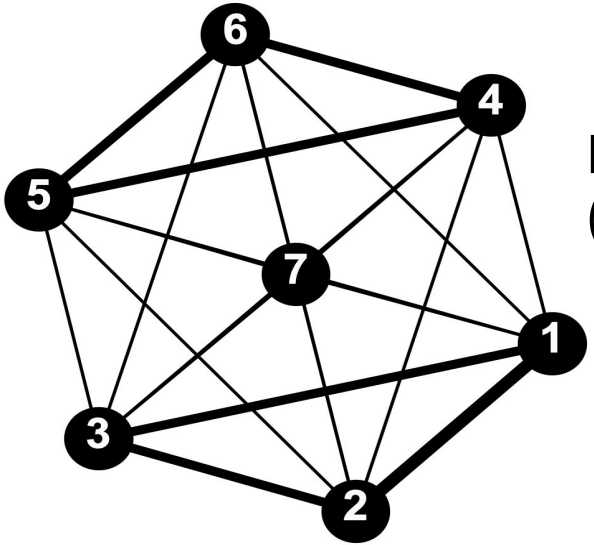
Edge: Undirected and weighted (thickness) and connects two given nodes by the number of interactions they have had

# Community Detection: Challenges

Noise in abundance **hide the real underlying network structure**

# Community Detection: Challenges

Noise in abundance **hide the real underlying network structure**



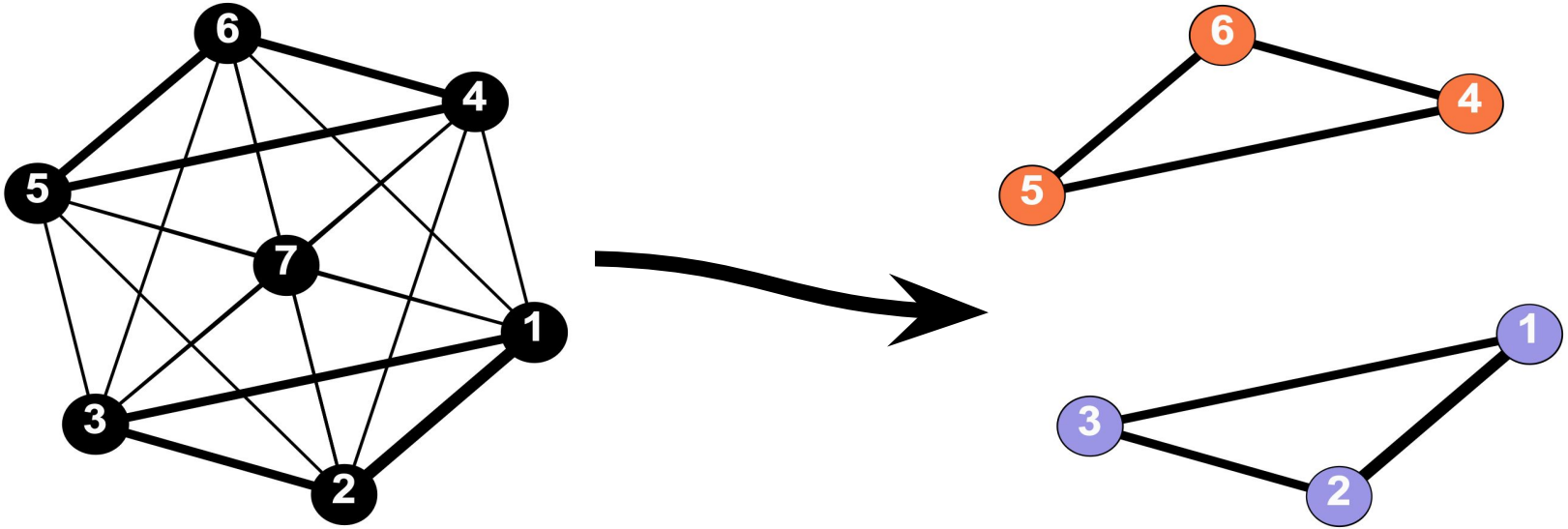

**Noisy Network**

Need to extract the truly relevant connections (**salient**) representing the phenomenon under study

- Forming the **network backbone**

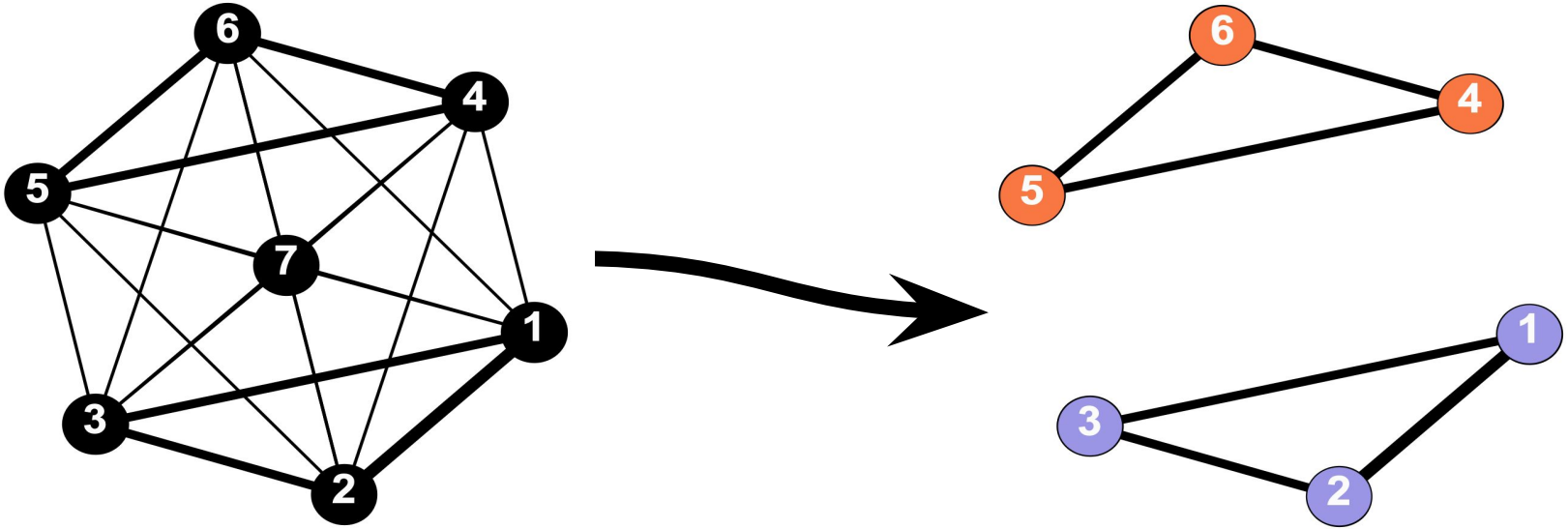
# Community Detection: Challenges

Noise in abundance **hide the real underlying network structure**



# Community Detection: Challenges

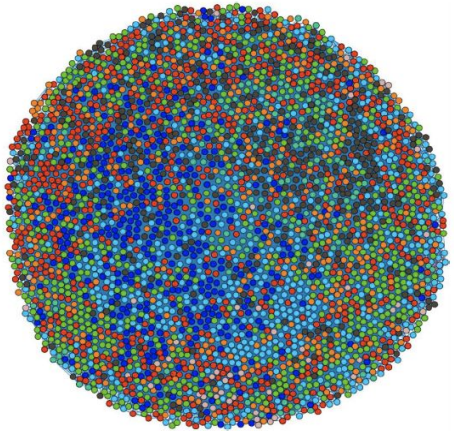
Noise in abundance **hide the real underlying network structure**




**Noisy Network** **Backbone**

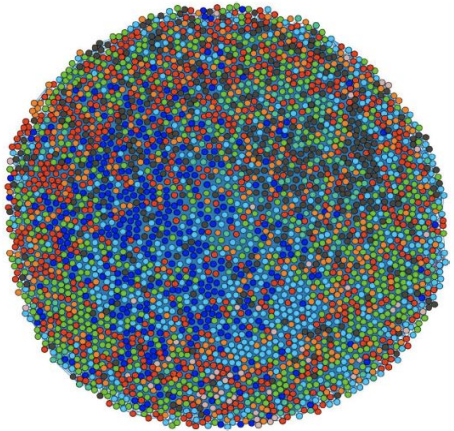
**In practice, it requires the adoption/development of more elaborate models**

# Impact of Backbone Extraction Models

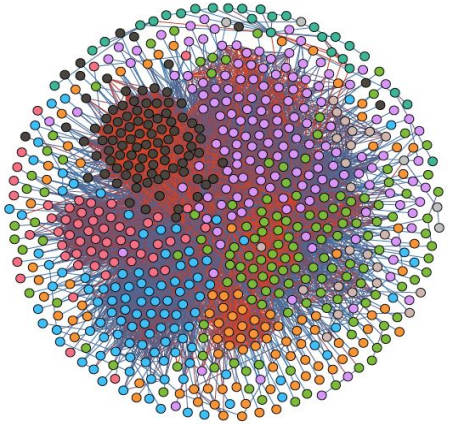


Original network

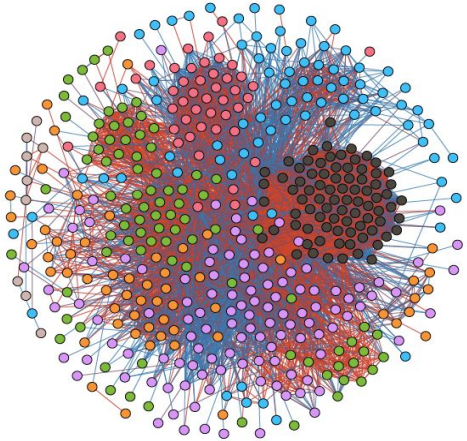
# Impact of Backbone Extraction Models



Original network

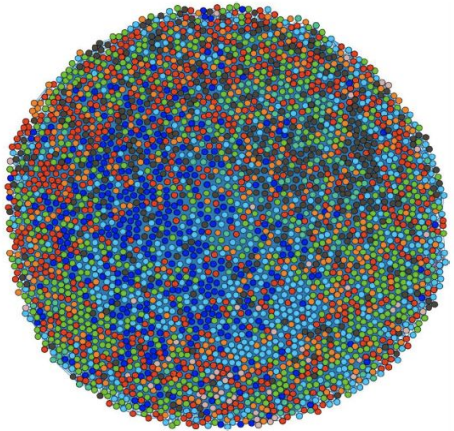


Disparity Filter's backbone

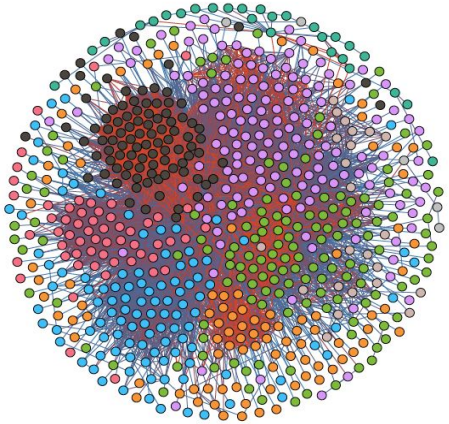


Threshold's backbone

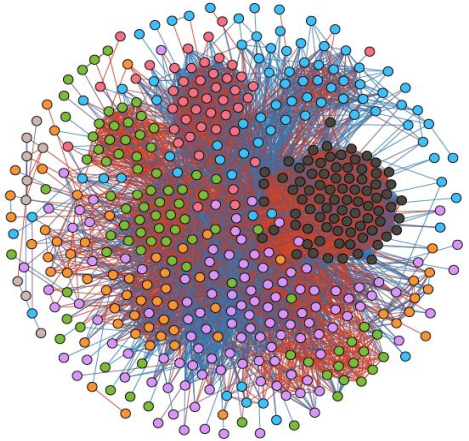
# Impact of Backbone Extraction Models



Original network



Disparity Filter's back-bone



Threshold's backbone

## How does it work?

# To be continued!

# To be continued!

**VISITING PROFESSOR DEL CENTENARIO – The Power of Social Media Platformsin Modern Society: Understanding Users'Behavior via Network Models**

**Condividi**

# To be continued!

## VISITING PROFESSOR DEL CENTENARIO – The Power of Social Media Platformsin Modern Society: Understanding Users'Behavior via Network Models

Condividi

15 Novembre ore 11:00

Il Dipartimento di INGEGNERIA E ARCHITETTURA – DIA presenta **venerdì 15 novembre 2024 – h 11.00, in**

**Aula 1-A Edificio D**, **Piazzale Europa 1,** *The Power of Social Media Platforms in Modern Society: Understanding Users'Behavior via Network Models.*

Interviene il **Prof. Carlos Henrique Gomes Ferreira**, Assistant Professor, Department of Computing and Systems, Federal University of Ouro Preto, Brazil.

# Roadmap

Graph Modeling Fundamentals

Structural and Centrality Analysis

Community Detection

**Final Considerations**

# Final Considerations

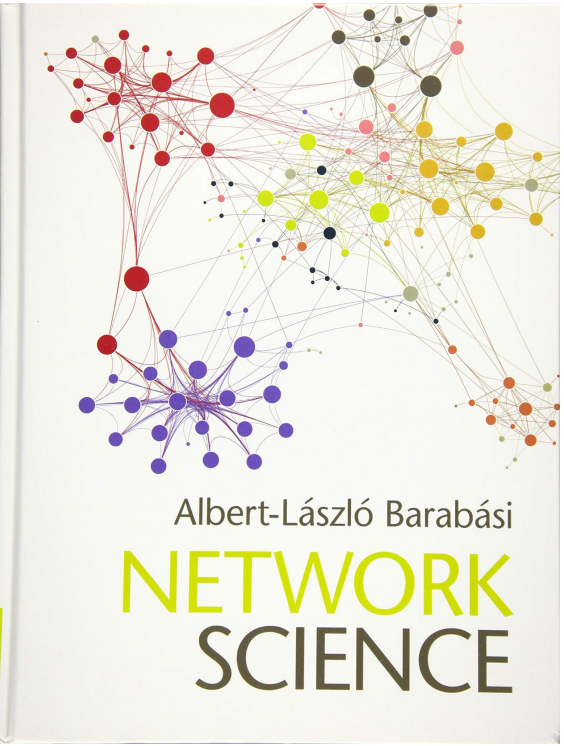
The study of networks offers valuable insights into complex systems

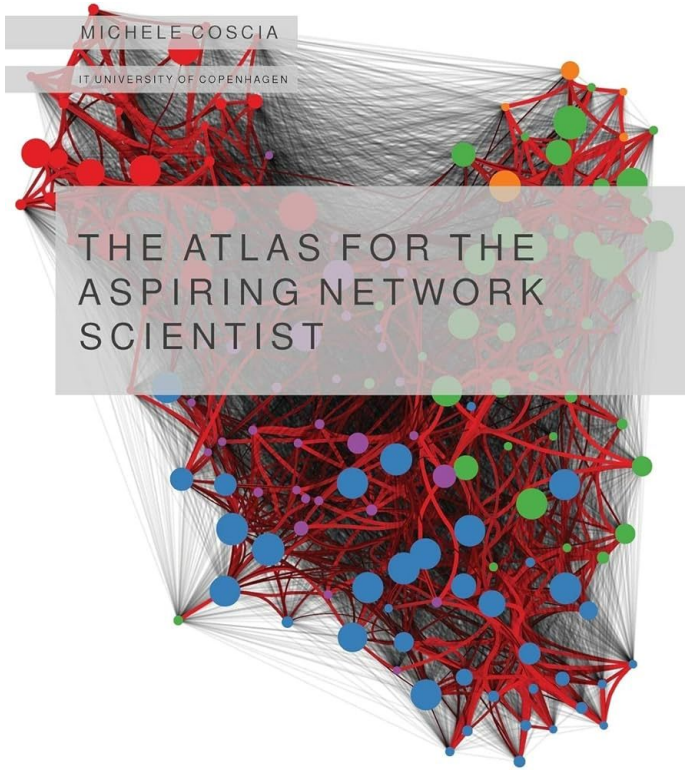
It is a multidisciplinary topic both within and outside of computing

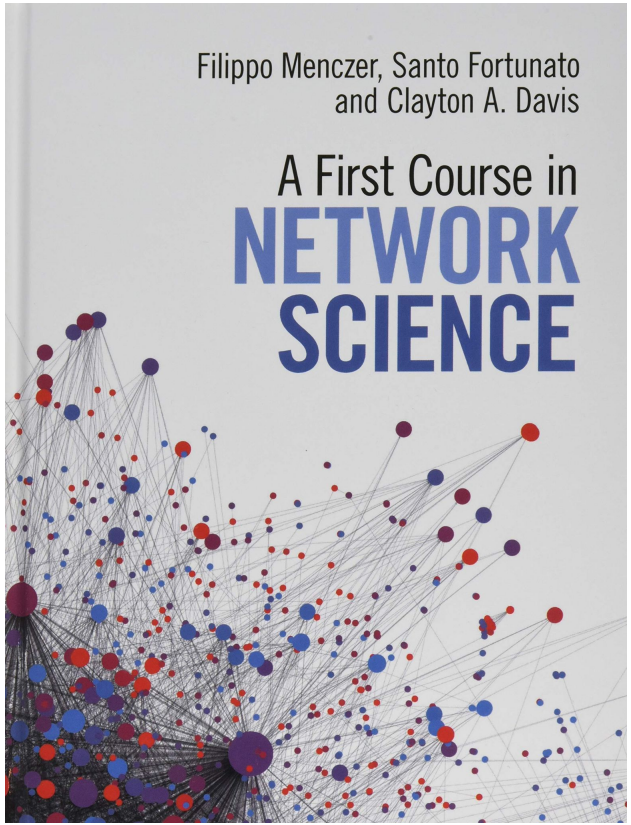
As technology evolves, the importance of this area will continue to increase, with a social impact

- Ex: Information dissemination on OSN, Biological Systems, etc

# Where to begin?



# If you would like to work a bit with this...



# Thanks!

E-mail: chgferreira@ufop.edu.br